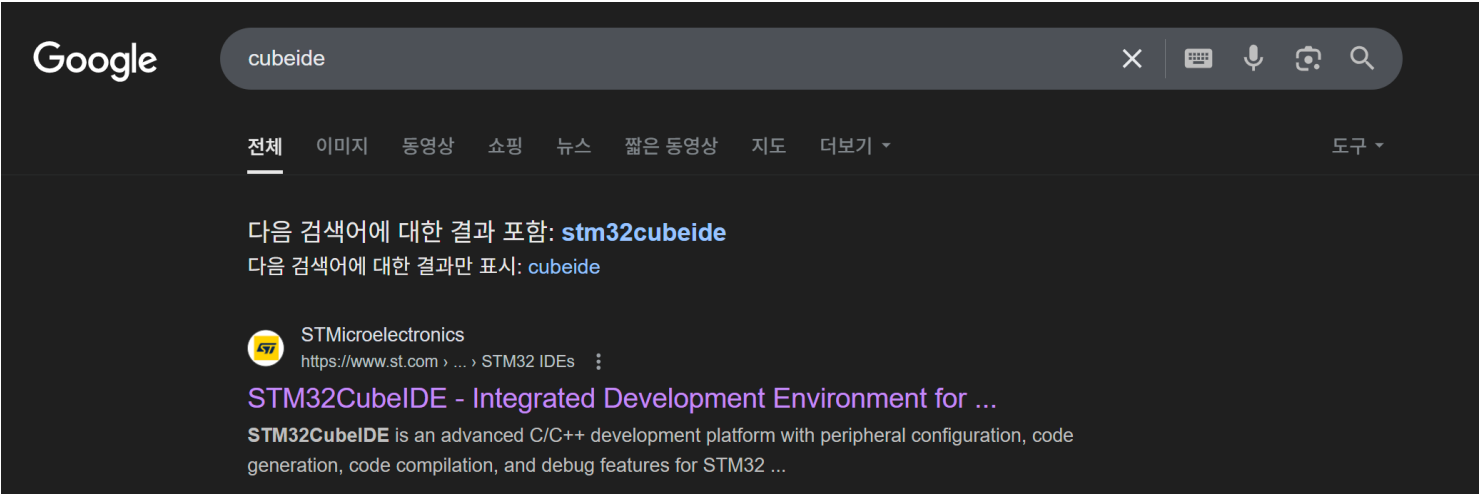


■ 개발환경 구성 (CubeIDE 설치)



Get Software

Part Number	General Description	Latest version	Download	All versions
+ STM32CubeIDE-DEB	STM32CubeIDE Debian Linux Installer	1.18.1	Get latest	Select version
+ STM32CubeIDE-Lnx	STM32CubeIDE Generic Linux Installer	1.18.1	Get latest	Select version
+ STM32CubeIDE-Mac	STM32CubeIDE macOS Installer	1.18.1	Get latest	Select version
+ STM32CubeIDE-RPM	STM32CubeIDE RPM Linux Installer	1.18.1	Get latest	Select version
+ STM32CubeIDE-Win	STM32CubeIDE Windows Installer	1.18.1	Get latest	Select version

STMicroelectronics recommends always keeping your software up to date

1.18.1

1.18.0

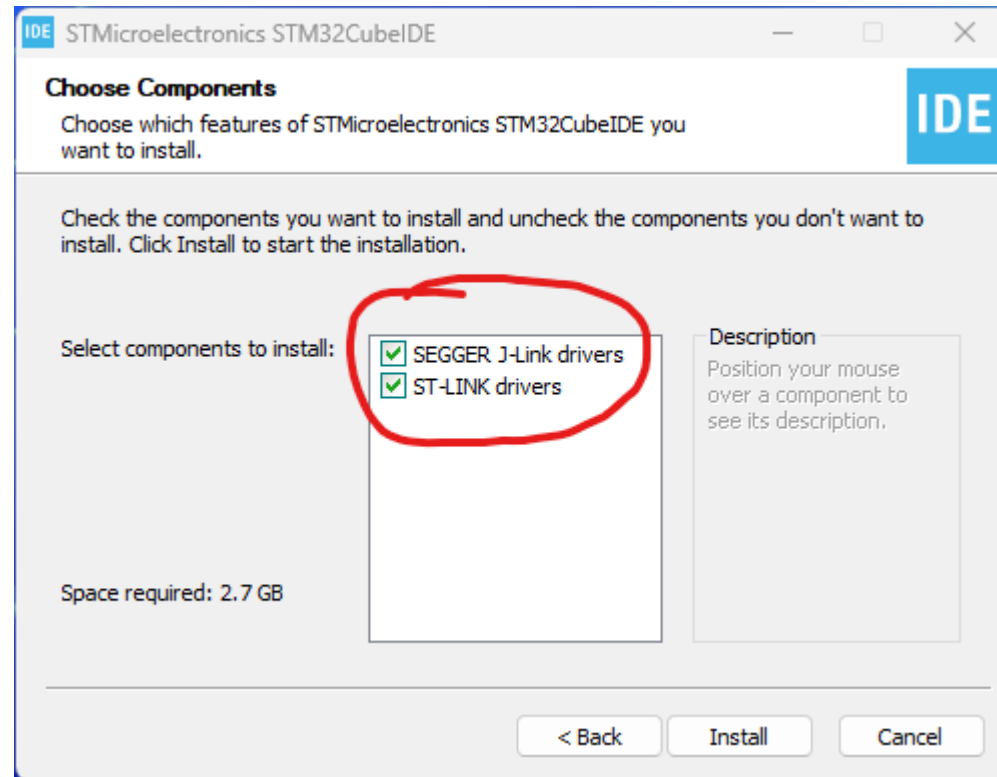
1.17.0



1.17 버전 다운로드 추천

■ 개발환경 구성 (CubeIDE 설치)

- 설치 경로에 한국어가 없도록 진행
- ST-LINK, J-LINK 모두 설치



■ 개발환경 구성

- 만일 ST link 연결 안되면 st link 드라이버 따로 설치

Google

st link v2 driver



전체

이미지

동영상

쇼핑

짧은 동영상

뉴스

웹

더보기 ▾

도구 ▾



STMicroelectronics

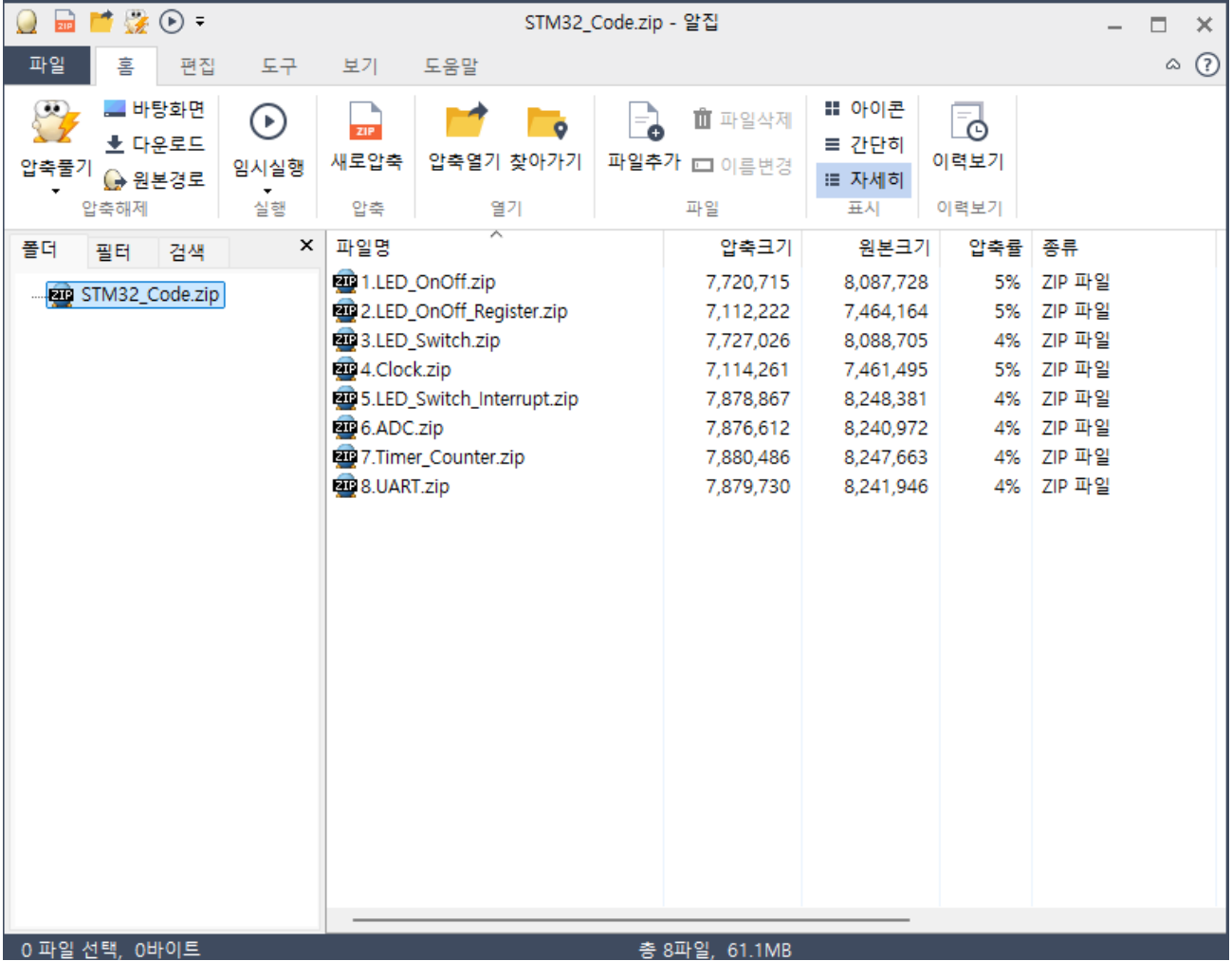
<https://www.st.com> > ... > STM32 utilities

STSW-LINK009

This USB driver (STSW-LINK009) is for **ST-LINK/V2**, ST-LINK/V2-1, and STLINK-V3 boards and derivatives (STM8/STM32 Discovery kits, STM8/STM32 Evaluation boards, ...)

■ 예제코드

➤ STM32_Code 압축파일 내부에 실습 코드 존재



예제코드 사용방법

프로젝트 생성

STM32 Project

Target Selection

Select STM32 target or STM32Cube example

MCU/MPU Selector

Board Selector

Example Selector

Cross Selector

MCU/MPU Filters

Commercial Part Number

stm32f767vit6

PRODUCT INFO

Segment

Series

Line

Marketing Status

Price

Package

Core

Coprocessor

MEMORY

Flash = 2048 (kBytes)

2048

EEPROM = 0 (Bytes)

0

RAM Total = 512 (kBytes)

512

RAM = 512 (kBytes)

Features

Block Diagram

Docs & Resources

CAD Resources

Datasheet

Buy

STM32F7 Series

STM32F767VIT6

High-performance and DSP with FPU, Arm Cortex-M7 MCU with 2 Mbytes of Flash memory, 216 MHz CPU, Art Accelerator, L1 cache, SDRAM, TFT, JPEG codec, DFSDM

Unit Price for 10kU (US\$) : 10.3813

LQFP 100 14x14x1.4 mm

The STM32F765xx, STM32F767xx, STM32F768Ax, and STM32F769xx devices are based on the high-performance Arm® Cortex®-M7 32-bit RISC core operating at up to 216 MHz frequency. The Cortex®-M7 core features a floating point unit (FPU), which supports Arm® double-precision and single-precision data-processing instructions and data types. It also implements a full set of DSP instructions and a memory protection unit (MPU), which enhances the application security. The STM32F765xx, STM32F767xx, STM32F768Ax, and STM32F769xx devices incorporate high-speed embedded memories with a flash up to 2 Mbytes, 512 Kbytes of SRAM (including 128 Kbytes of Data TCM RAM for critical real-time data), 16 Kbytes of instruction TCM RAM (for critical real-time routines), 4 Kbytes of backup SRAM available in the lowest power modes, and an extensive range of enhanced I/Os and peripherals connected to two APB buses, two AHB buses, a 32-bit multi-processor bus, and a multi-processor bus.

MCUs/MPUs List: 2 items

Export

	Commercial Part No	Reference	Marketing Sta...	Unit Price for ...	Board	Package	Flash	RAM	I/O	Frequency
☆	STM32F767VIT6	STM32F767V...	Active	10.3813		LQFP 100 14x...	2048 kBytes	512 kBytes	82	216 MHz
☆	STM32F767VIT6TR	STM32F767V...	Active	10.3813		LQFP 100 14x...	2048 kBytes	512 kBytes	82	216 MHz

< Back

Next >

Finish

Cancel

STM32 Project

IDE

Directory with specified name already exists

Project

Project Name: LED_ONOFF

☒ Use default location

Location: C:/Users/ljh/Desktop/A/ExampleCode

Browse...

Options

Targeted Language

☒ C ☐ C++

Targeted Device Usage

Targeted Binary Type

☒ Executable ☐ Static Library

Targeted Project Type

☒ STM32Cube ☐ Empty

?

< Back

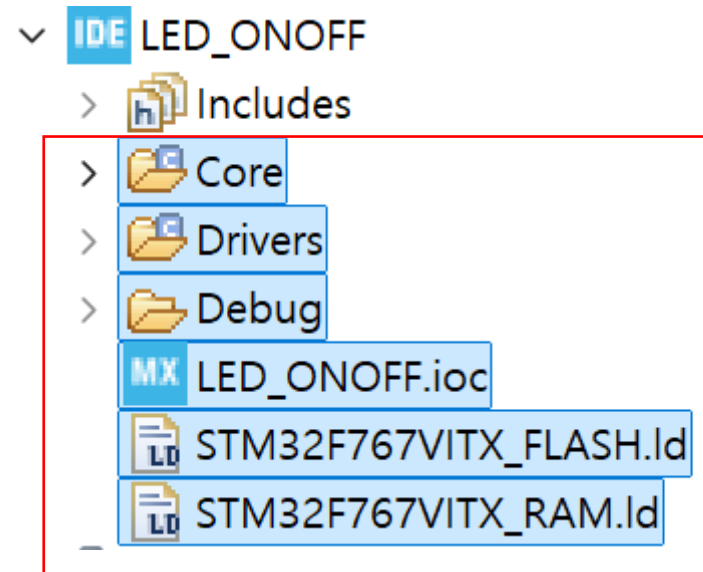
Next >

Finish

Cancel

■ 예제코드 사용방법

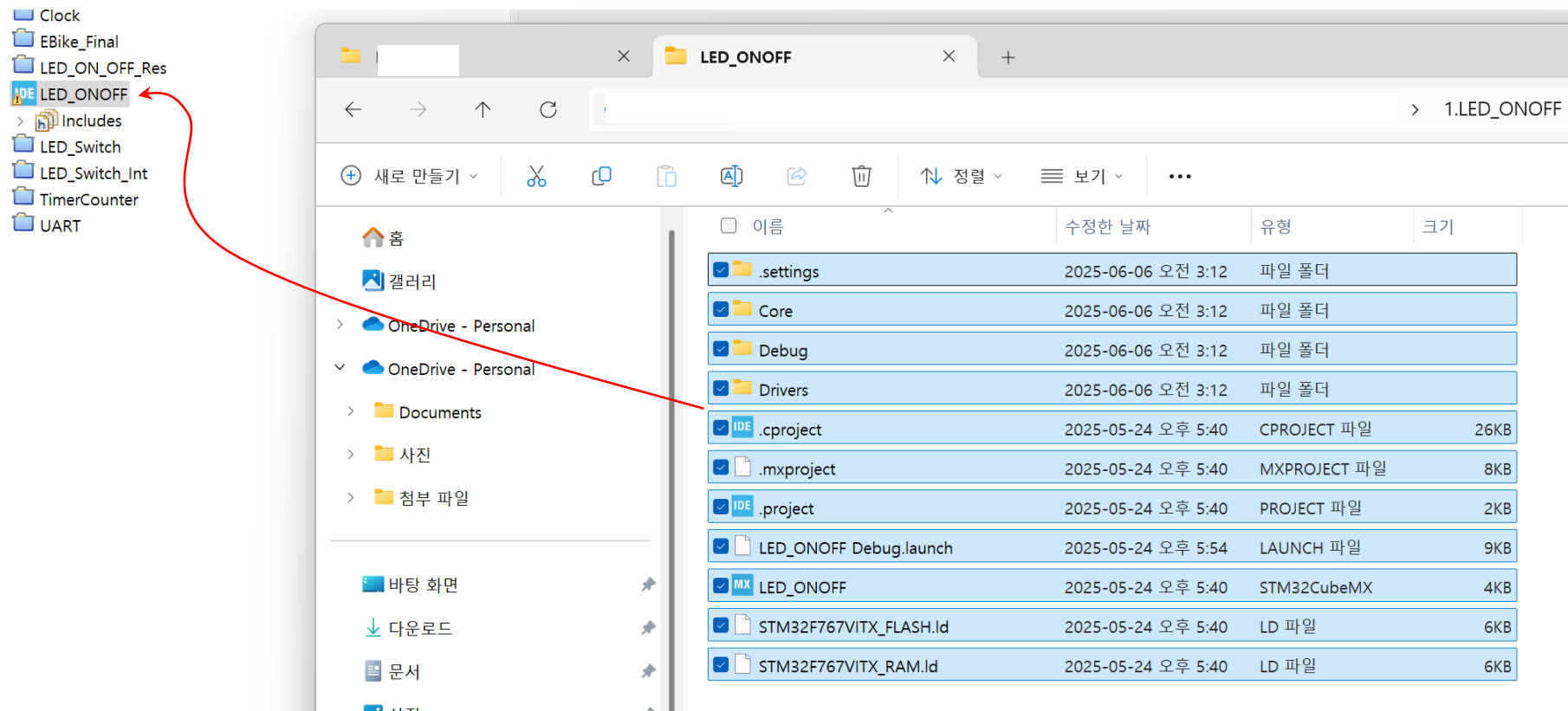
- 프로젝트 생성 후 Includes를 제외한 나머지 폴더,파일 제거



삭제

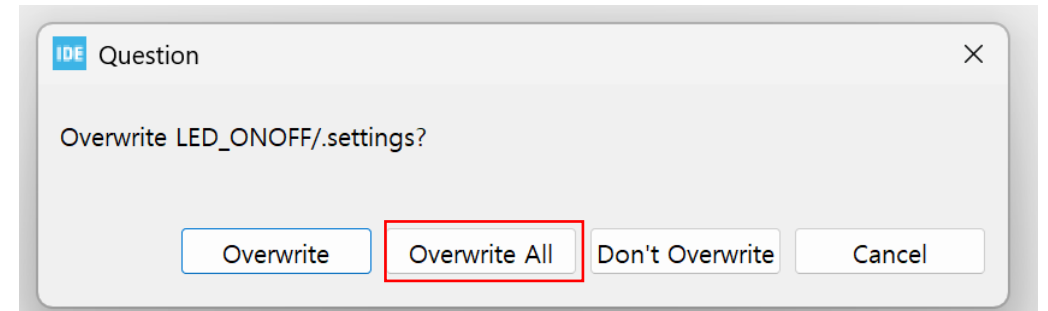
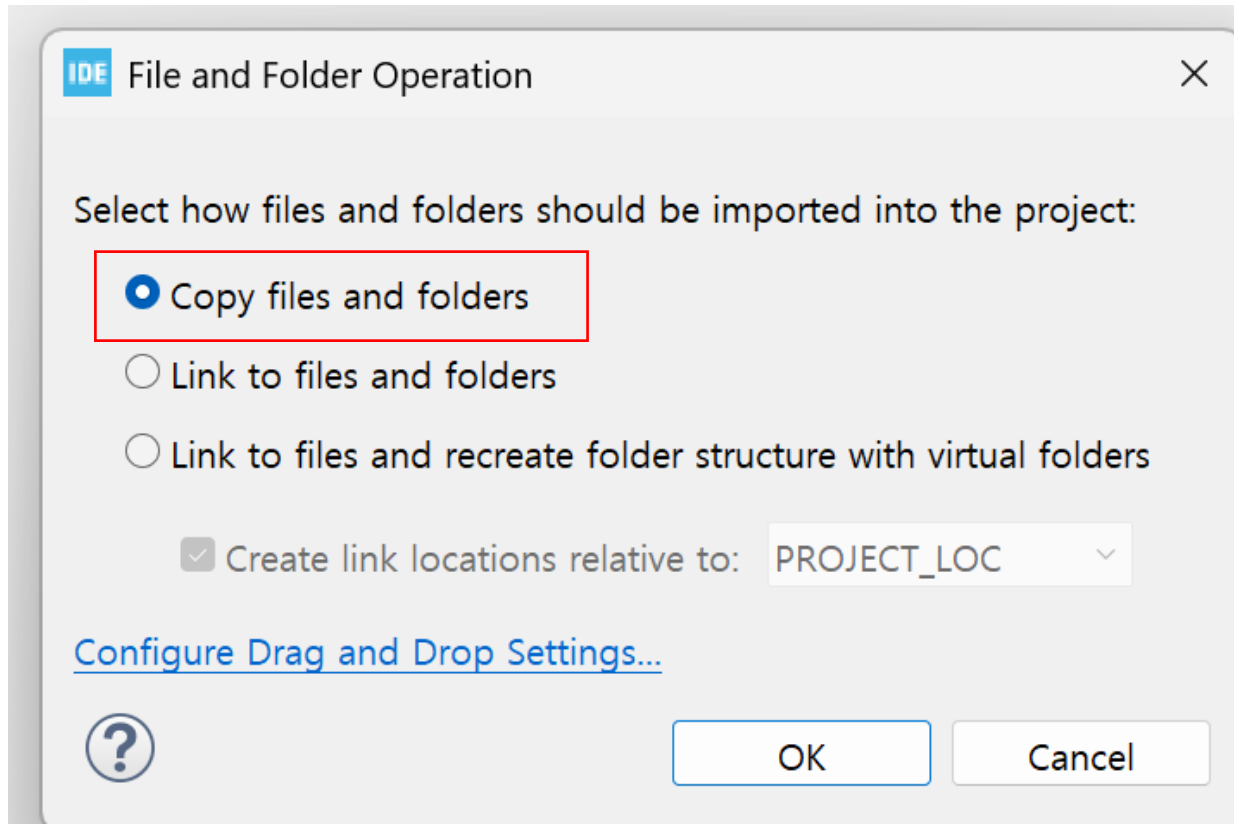
■ 예제코드 사용방법

➤ 제공한 실습 코드 압축해제하고, 내부에 있는 파일 드래그하여 새로 만든 프로젝트로 이동



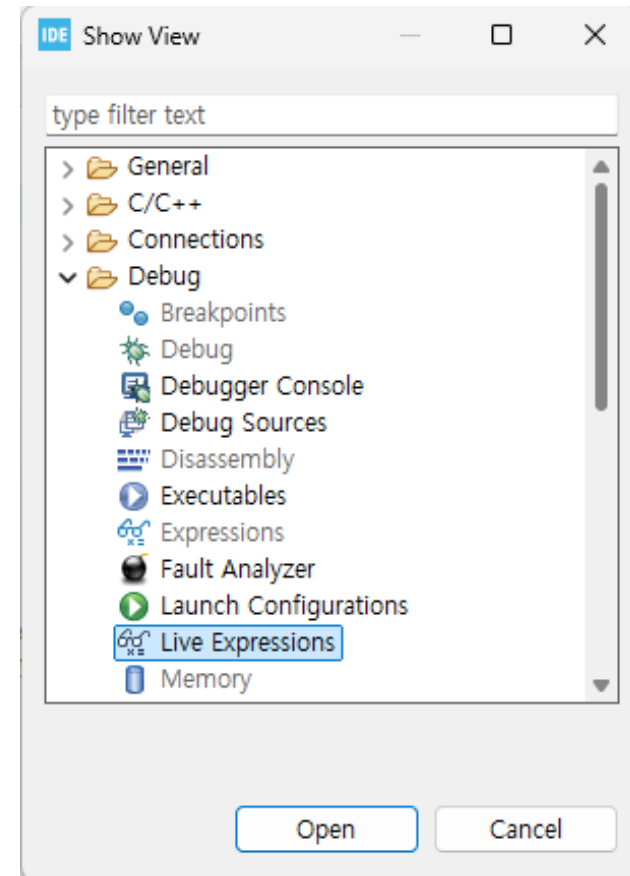
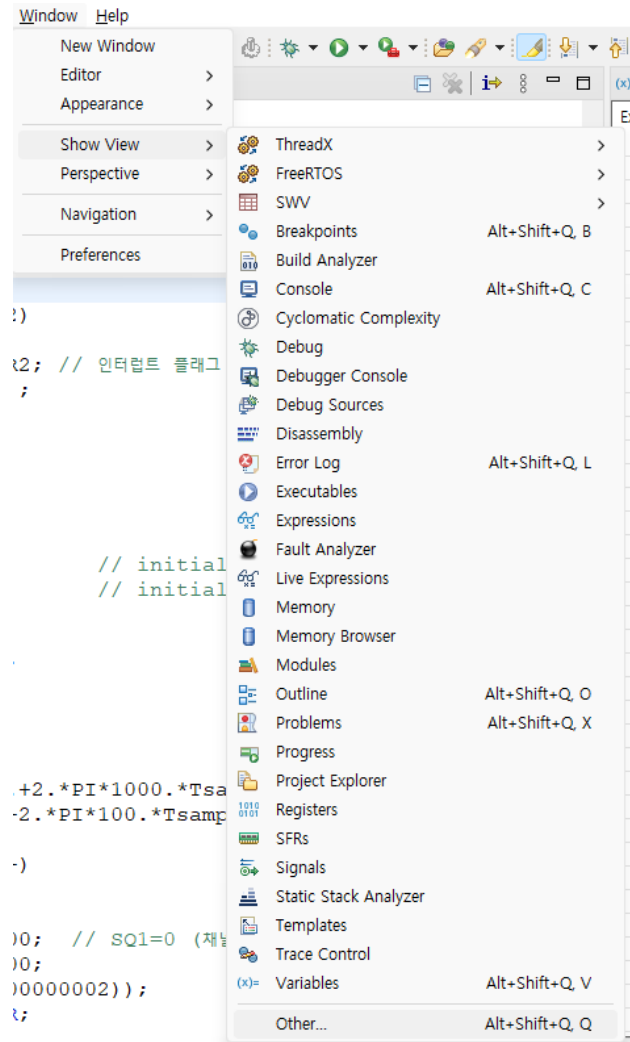
■ 예제코드 사용방법

- Copy files and folders 클릭 → Overwrite All 클릭



■ Live Expressions 사용법

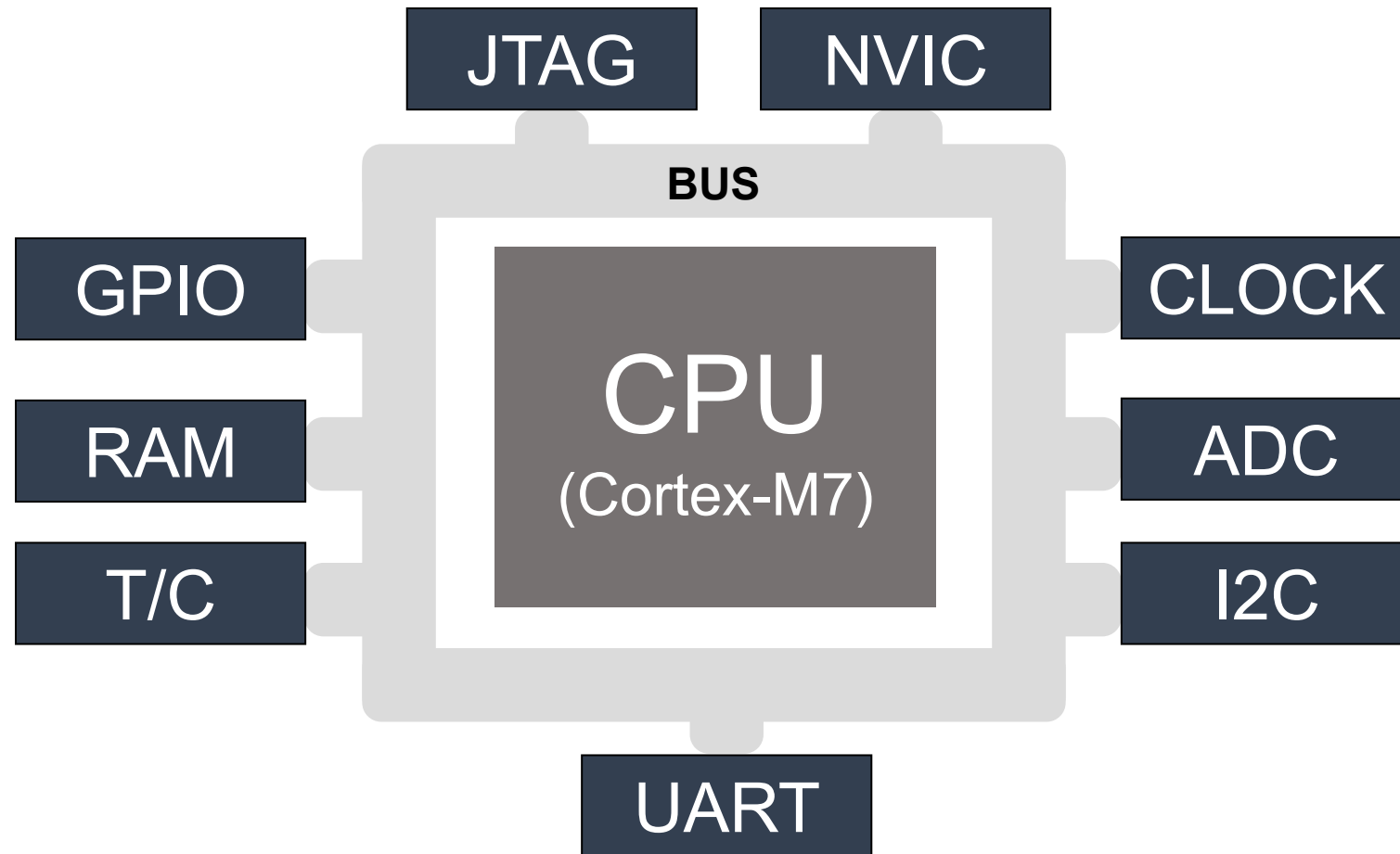
➤ Window → Show View → Other



1. ARM 마이크로프로세서 개요

■ MCU(Micro Controller Unit)이란?

- MCU란 CPU에 디지털 신호(On/Off), PWM, 통신과 같이 임베디드 시스템에서 많이 사용되는 주변장치(Pheripheral)을 연결하여 하나의 칩으로 제작된 형태
- CPU와 주변 장치는 데이터를 주고 받는데, 이러한 길을 버스라고 부름



1. ARM 마이크로프로세서 개요

■ Arm과 STM32

- Arm에서는 코어를 제공하고, 각 반도체 업체(ST, TI, NXP, Infineon 등)에서 코어와 주변장치를 추가하여 MCU 제작

Cortex-A

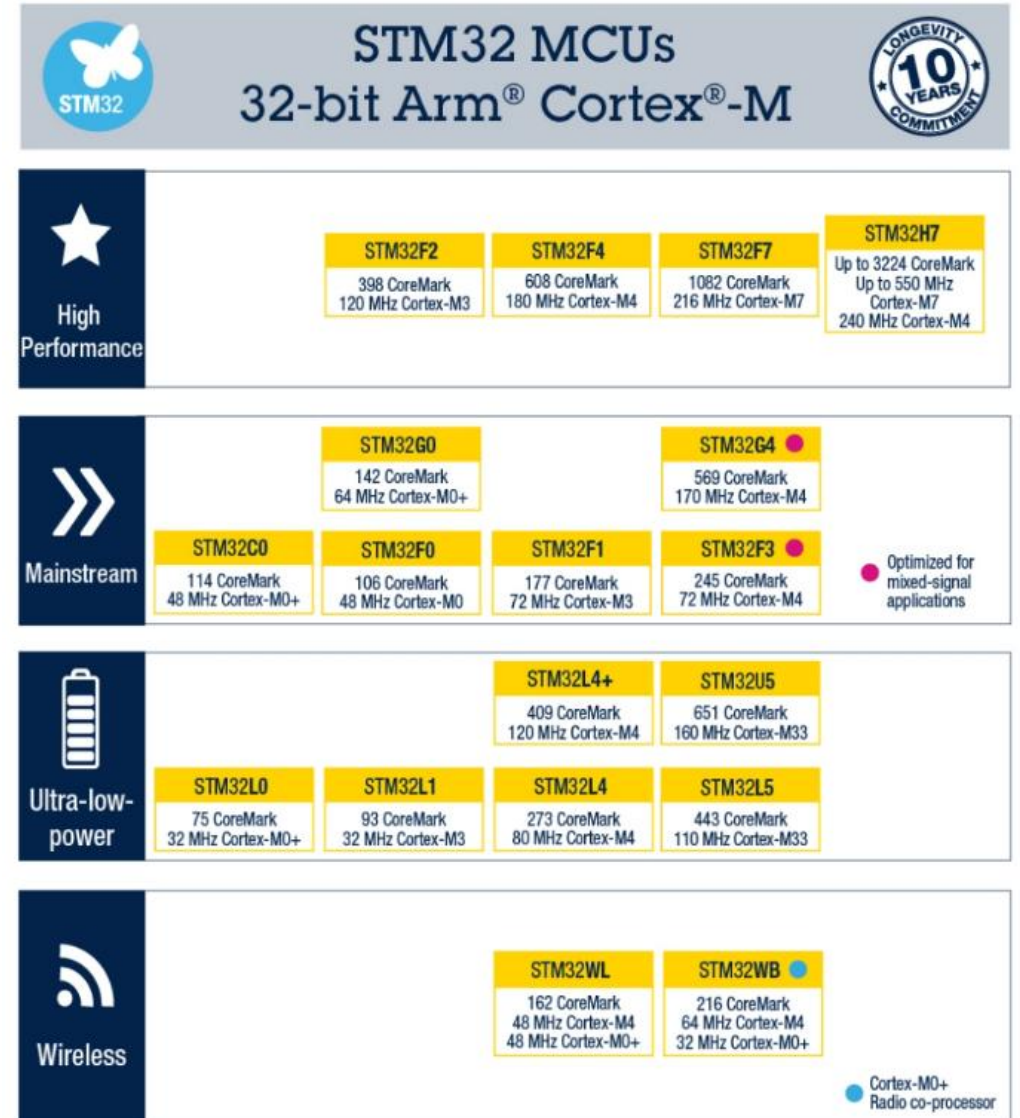
- ✓ 스마트폰, 태블릿, PC, 고성능 임베디드 프로세서
- ✓ 멀티 코어, 높은 성능

Cortex-R

- ✓ 실시간 프로세싱을 위한 고성능 임베디드 프로세서
- ✓ 의료기기, 항공우주 자동차 등

Cortex-M

- ✓ MCU 기반 임베디드 시스템을 위해 설계
- ✓ 초저전력, 실시간 처리 성능
- ✓ 범용적으로 많이 사용



1. ARM 마이크로프로세서 개요

Google

stm32f767 datasheet

전체 쇼핑 이미지 동영상 짧은 동영상 도서 웹 더보기 ▾

도움말: 결과를 **한국어**로 표시합니다. 언어별 필터링에 대해서도 자세히 알아보세요.

 STMicroelectronics
https://www.st.com › resource › datasheet PDF ⋮

STM32F765xx STM32F767xx STM32F768Ax STM32F769xx

Includes **ST state-of-the-art patented technology**. • Core: Arm® 32-bit Cortex®-M7 CPU with. DPFPU, ART Accelerator and L1-cache: 16 Kbytes I/D cache, ...

페이지 256개

Google

stm32f76xxx reference manual

전체 이미지 동영상 쇼핑 짧은 동영상 뉴스 도서 더보기 ▾

도움말: 결과를 **한국어**로 표시합니다. 언어별 필터링에 대해서도 자세히 알아보세요.

 STMicroelectronics
https://www.st.com › resource › reference_manual PDF ⋮

RM0410 Reference manual - -based 32-bit MCUs

2024. 7. 1. — It provides complete information on **how to use the STM32F76xxx** and STM32F77xxx microcontroller memory and peripherals. The STM32F76xxx and ...

페이지 1,942개

1. ARM 마이크로프로세서 개요

■ Arm과 STM32

➤ Bus 구조

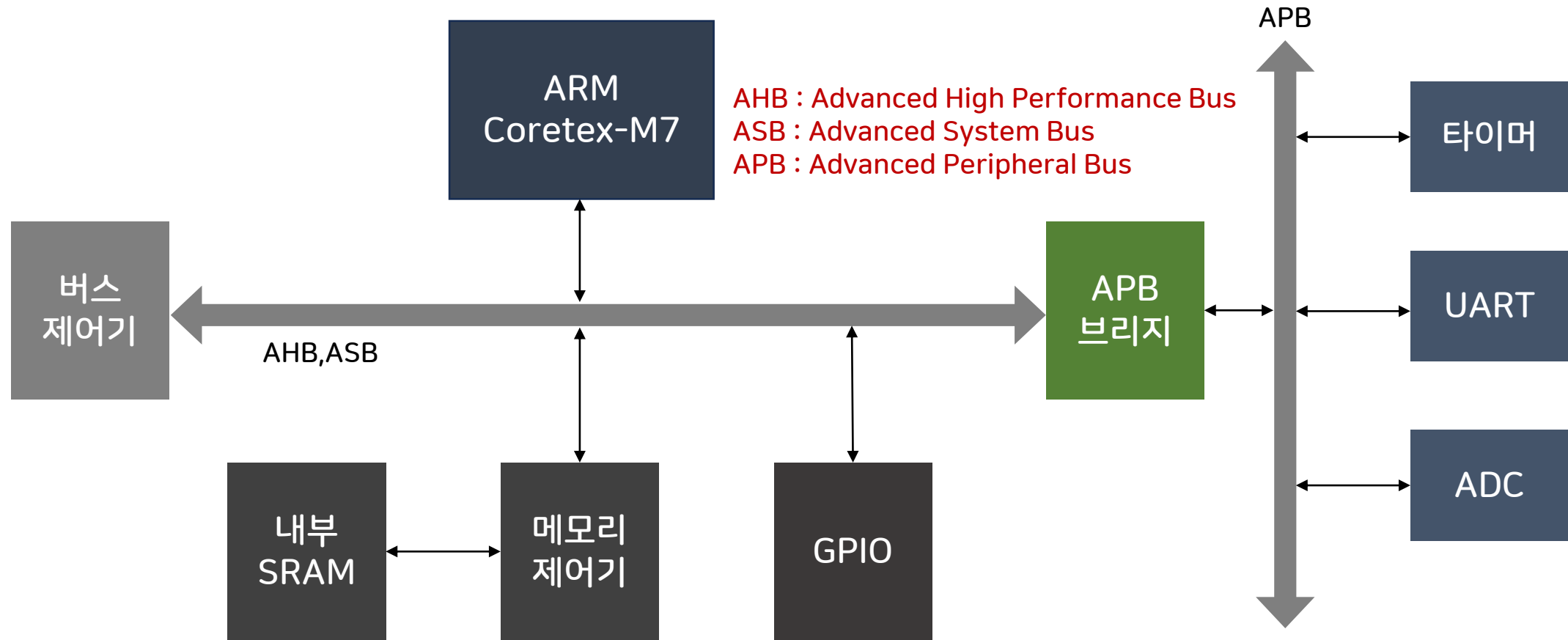
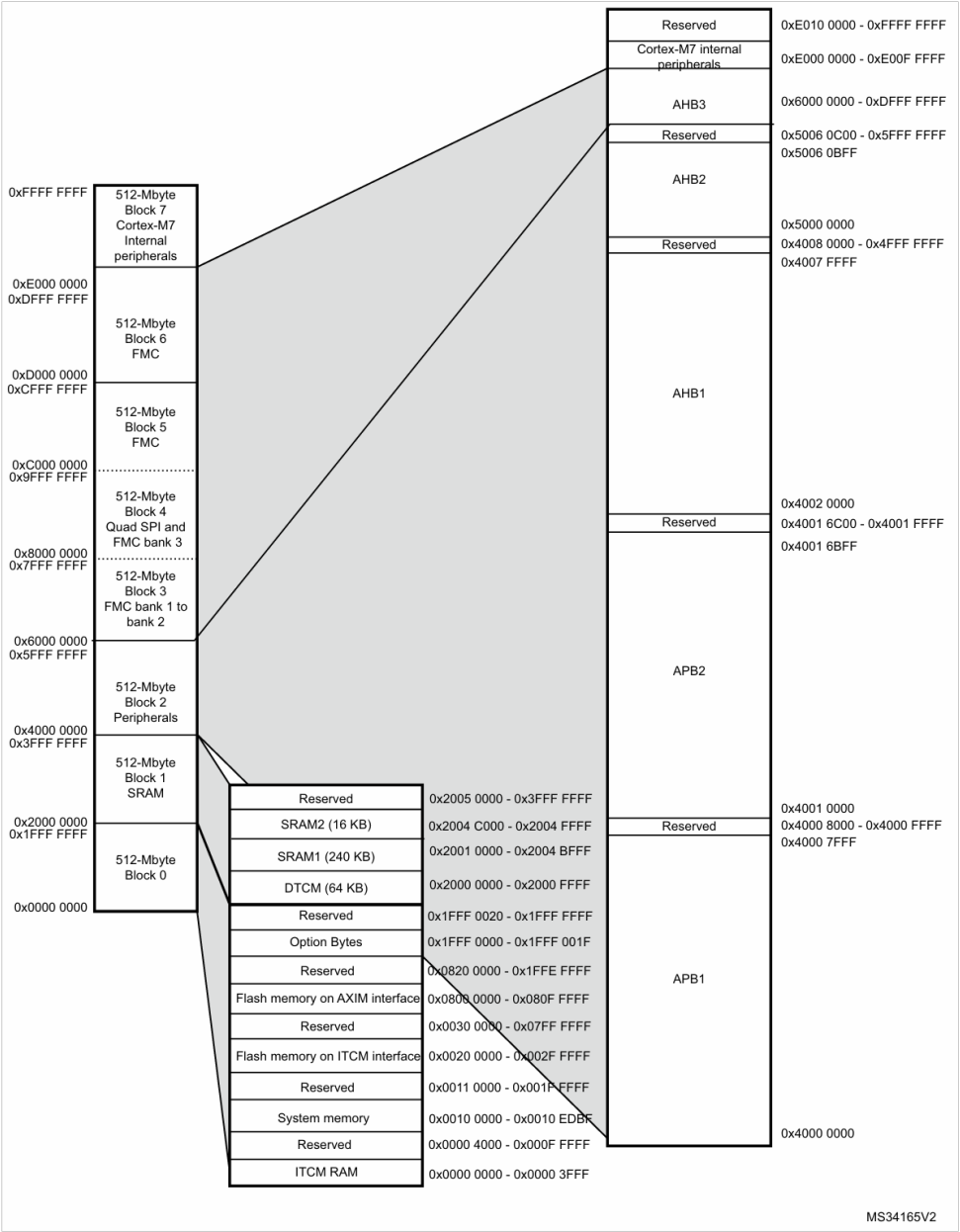


Figure 2. Memory map

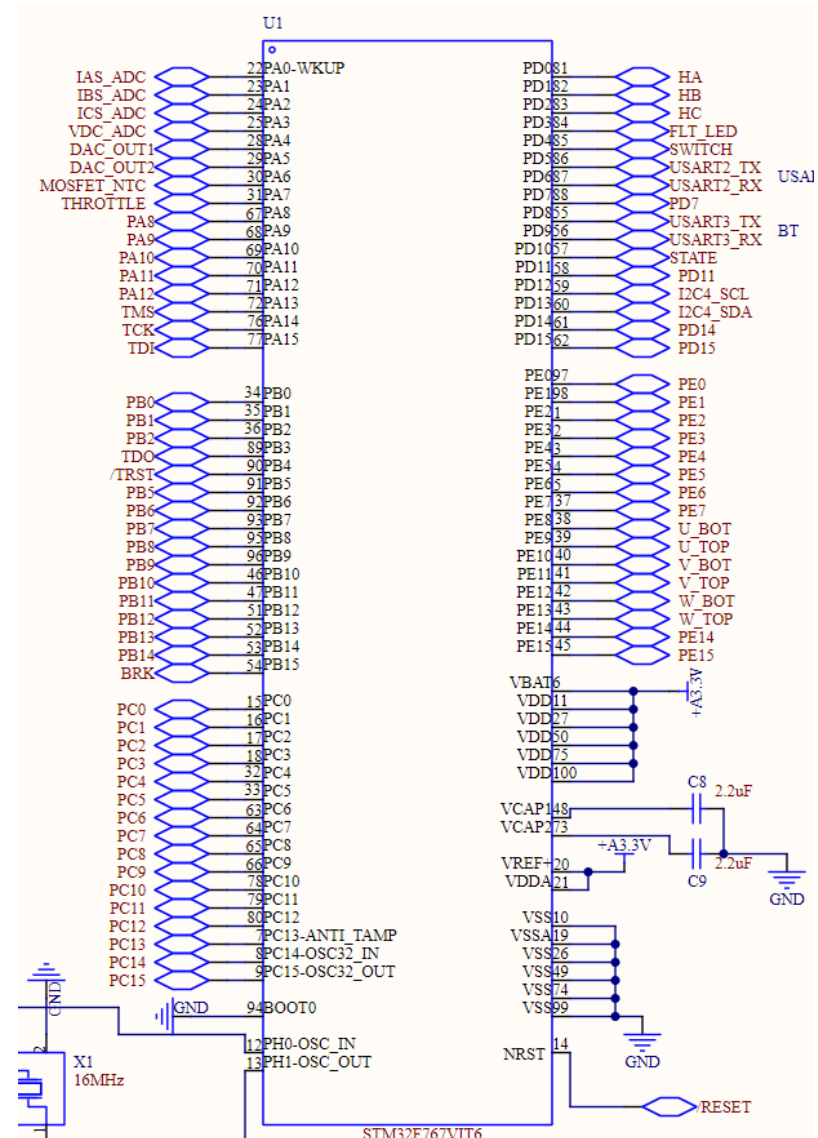
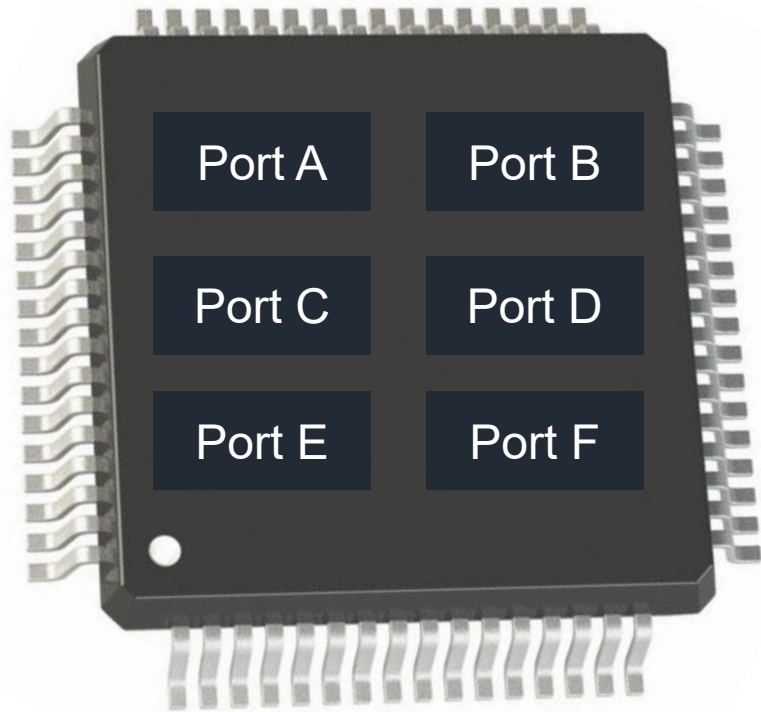


LED On/OFF

2. I/O 포트(GPIO)

■ Hello LED

- MCU 종류, 패키지에 따라 PORT의 수가 다름
- 각 PORT마다 최대 16개의 PIN 존재
- 교육용 보드(Stm32F767VIT6)는 PA~PE15 까지 지원함



<마이크로 컨트롤러의 포트>

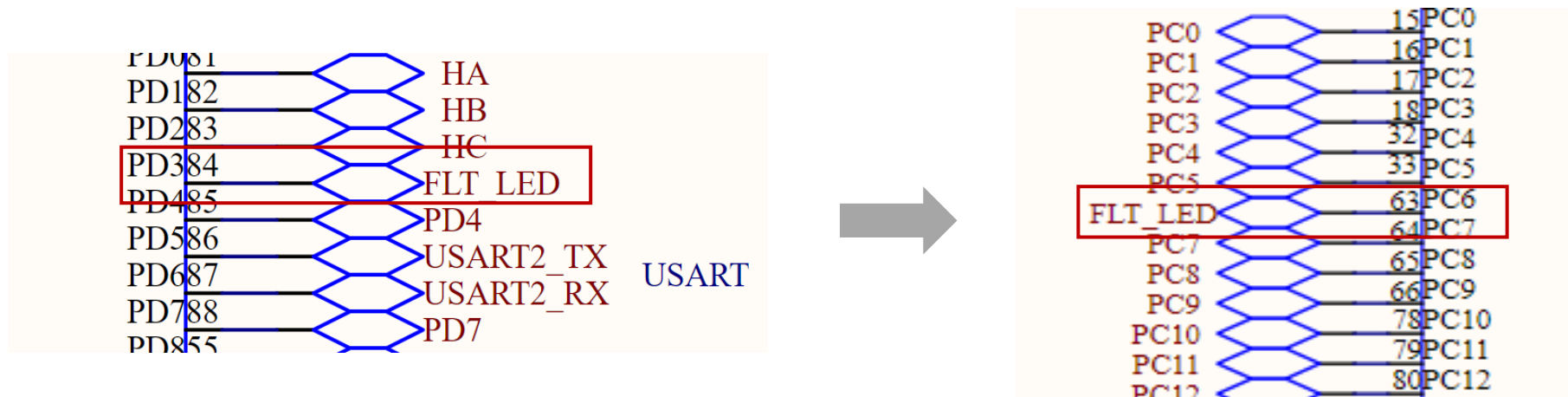
하드웨어 핀 변경으로 PD3 → PC6으로 변경되었습니다.

강의영상에는 코드 구현과 설명이 PD3로 설명 되어있습니다.

포트만 바뀐 것이므로 코드는 GPIO모듈을 PORTD를 사용할지 PORTC를 사용할 지의 차이만 존재합니다.

혹시 코드 구현에 있어서 어려움이 있으시면 질문 남겨 주시면 상세히 설명 드리도록 하겠습니다.

죄송합니다.

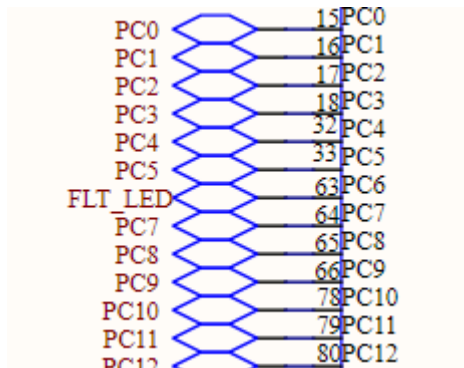
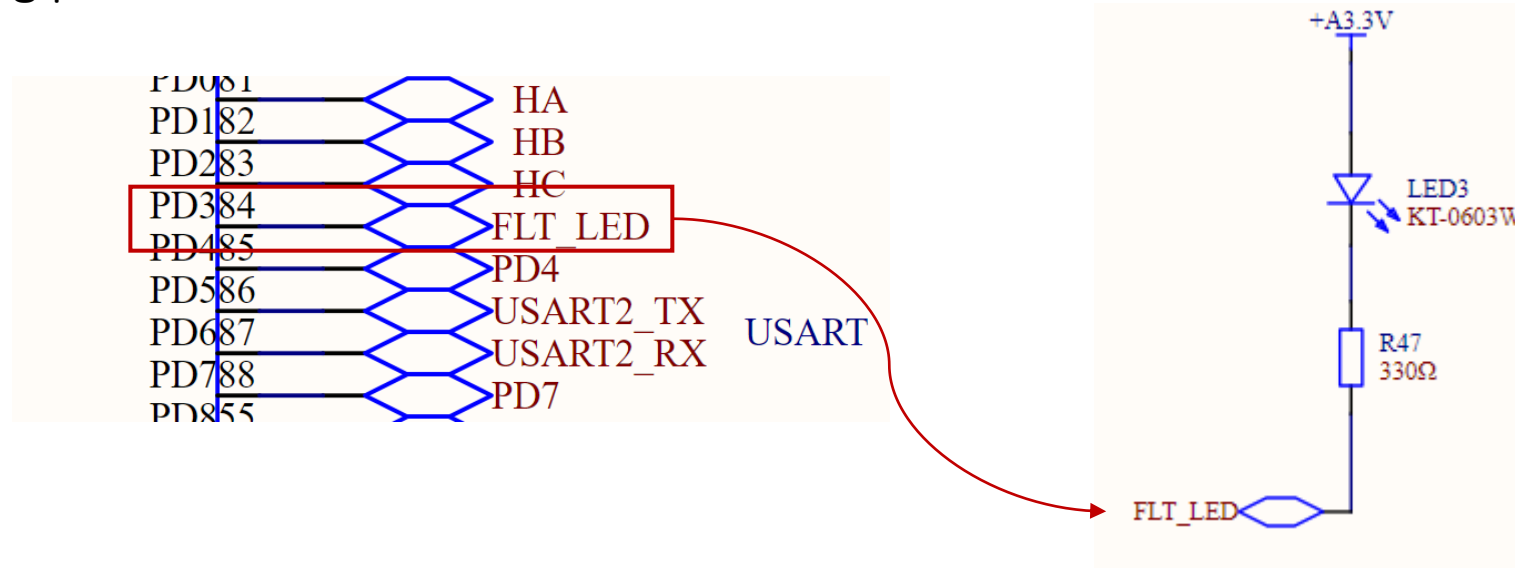


<PD3 → PC6으로 LED 핀 변경>

2. I/O 포트(GPIO)

- Hello LED

- PORTD 3 을 On/Off하여 LED 컨트롤, 포트 내부의 구조는 Push-Pull 형태로 되어 있음
- PD3이 High(3.3V)인 경우 LED OFF
- PD3이 Low(0V)인 경우 LED On



PORTD 3	LED
High	Off
Low	On

하드웨어 핀 변경으로 PD3 → PC6으로 변경되었습니다.

<PORT 상태에 따른 LED 동작>

2. I/O 포트(GPIO)

■ MCU 레지스터 설정

- MCU 레지스터를 설정한다는 것은 MCU의 주변기기의 기능을 사용하기 위한 작업
- 레지스터 설정 시 세가지의 기준을 가지고 설정을 할 수 있음
- 사용자가 Reference Manual을 보고 특정한 모듈을 설정(Write) 해주기 위한 레지스터 → Control Register
- 특정 모듈의 상태를 읽을(Read) 수 있는 레지스터 → Status Register
- 데이터를 읽을 수 있는 레지스터 → Data Register

Control Register

통신 속도, 클럭 속도, 인터럽트
설정 등

Status Register

통신 완료, 클럭 설정 완료, 인터
럽트 발생

Data Register

ADC 변환데이터, 통신 데이터

2. I/O 포트(GPIO)

■ MCU 레지스터 설정 예시

- GPIOx_MODER 레지스터는 사용하려는 PORT를 어떤 모드로 사용할지 정할 수 있음
- 각 포트는 단순 On/Off 기능 뿐만 아니라 Alternate functions, ADC로 사용할 수 있음
- Ex) PA6는 여러가지 기능을 수행할 수 있음
- 레지스터는 32bit로 이루어져 있으며, 각 비트를 0 또는 1로 설정하여 레지스터 설정을 할 수 있음

Pin Number											Pin name (function after reset)	Pin type	I/O structure	Notes	Alternate functions	Additional functions
STM32F765xx STM32F767xx							STM32F768Ax STM32F769xx									
TFBGA100	LQFP100	LQFP144	UFBGA176	LQFP176	LQFP208	TFBGA216	WLCSP180 ⁽¹⁾	LQFP176	LQFP208	TFBGA216						
H3	29	41	P4	51	54	P4	P11	51	54	P4	PA5	I/O	TTa	-	TIM2_CH1/TIM2_ETR, TIM8_CH1N, SPI1_SCK/I2S1_CK, SPI6_SCK, OTG_HS_ULPI_CK, LCD_R4, EVENTOUT	ADC1_IN5, ADC2_IN5, DAC_OUT2
J3	30	42	P3	52	55	P3	N10	52	55	P3	PA6	I/O	FT	-	TIM1_BKIN, TIM3_CH1, TIM8_BKIN, SPI1_MISO, SPI6_MISO, TIM13_CH1, MDIOS_MDC, DCMI_PIXCLK, LCD_G2, EVENTOUT	ADC1_IN6, ADC2_IN6

6.4.1 GPIO port mode register (GPIOx_MODER) (x =A to K)

Address offset:0x00

Reset value:

- 0xA800 0000 for port A
- 0x0000 0280 for port B
- 0x0000 0000 for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **MODER[15:0][1:0]:** Port x configuration I/O pin y (y = 15 to 0)

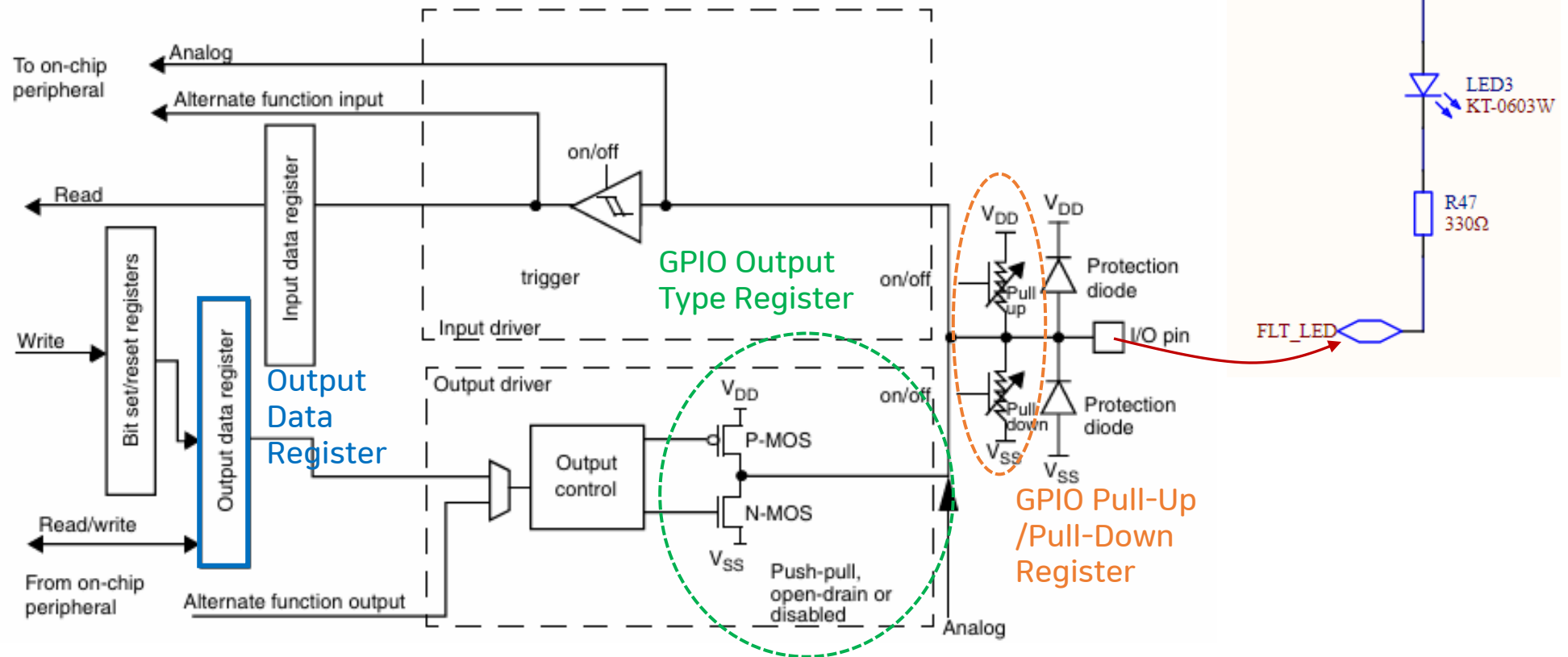
These bits are written by software to configure the I/O mode.

- 00: Input mode (reset state)
- 01: General purpose output mode
- 10: Alternate function mode
- 11: Analog mode

2. I/O 포트(GPIO)

■ GPIO 내부 블록도

- 아래의 그림은 GPIO의 내부 블록도이다.
- Register 설정을 통해 PORT를 On/Off(Output Control)할 수 있음 → Push-Pull형태, 오픈 드레인 선택 가능, 내부 풀업/풀 저항 설정 가능
- Protection Diode를 내장하고 있어 입력 신호의 과전압을 보호 할 수 있음



<GPIO 내부 블록도-출력>

2. I/O 포트(GPIO)

- AHB1 peripheral clock register (RCC_AHB1ENR)
 - MCU는 클럭을 기반으로 동작하므로 클럭 주파수 설정을 해줘야 함
 - PORTD3에 Clock enabled

5.3.10 RCC AHB1 peripheral clock register (RCC_AHB1ENR)

Address offset: 0x30

Reset value: 0x0010 0000

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	OTGHS ULPIEN	OTGHS EN	ETHM ACPTP EN	ETHM ACRX EN	ETHM ACTX EN	ETHMA CEN	Res.	DMA2D EN	DMA2 EN	DMA1 EN	DTCMRA MEN	Res.	BKPSR AMEN	Res.	Res.
	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw		rw		
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	CRC EN	Res.	GPIOK EN	GPIOJ EN	GPIOI EN	GPIOH EN	GPIOG EN	GPIOF EN	GPIOE EN	GPIOD EN	GPIOC EN	GPIOB EN	GPIOA EN
			rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 3 **GPIODEN**: IO port D clock enable

This bit is set and cleared by software.

0: IO port D clock disabled

1: IO port D clock enabled

2. I/O 포트(GPIO)

- GPIO port mode register (GPIOx_MODER) (x=A to K)
 - PORTD 3 을 Output mode로 설정

6.4.1 GPIO port mode register (GPIOx_MODER) (x =A to K)

Address offset:0x00

Reset value:

- 0xA800 0000 for port A
- 0x0000 0280 for port B
- 0x0000 0000 for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
								Pin 3		Pin 2		Pin 1		Pin 0	

Bits 31:0 **MODER[15:0][1:0]**: Port x configuration I/O pin y (y = 15 to 0)

These bits are written by software to configure the I/O mode.

00: Input mode (reset state)

01: General purpose output mode

10: Alternate function mode

11: Analog mode

2. I/O 포트(GPIO)

- GPIO Output Type Register (OTYPE)
 - PORTD 3의 출력을 Push-pull로 설정

6.4.2 GPIO port output type register (GPIOx_OTYPER) (x = A to K)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Pin 3

Bits 15:0 **OT[15:0]**: Port x configuration I/O pin y (y = 15 to 0)
These bits are written by software to configure the I/O output type.
0: Output push-pull (reset state)
1: Output open-drain

2. I/O 포트(GPIO)

- GPIO port output speed register (GPIOx_OSPEEDR)
 - PORTD 3의 출력 On/Off 속도 설정
 - GPIO의 속도가 빠르다고 항상 좋은 것은 아님 → EMI 문제 발생

6.4.3 GPIO port output speed register (GPIOx_OSPEEDR) (x = A to K)

Address offset: 0x08
Reset value: 0x0C00 0000 (for port A)
Reset value: 0x0000 00C0 (for port B)
Reset value: 0x0000 0000 (for other ports)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
OSPEEDR15 [1:0]		OSPEEDR14 [1:0]		OSPEEDR13 [1:0]		OSPEEDR12 [1:0]		OSPEEDR11 [1:0]		OSPEEDR10 [1:0]		OSPEEDR9 [1:0]		OSPEEDR8 [1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OSPEEDR7 [1:0]		OSPEEDR6 [1:0]		OSPEEDR5 [1:0]		OSPEEDR4 [1:0]		OSPEEDR3 [1:0]		OSPEEDR2 [1:0]		OSPEEDR1 [1:0]		OSPEEDR0 [1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Pin 3

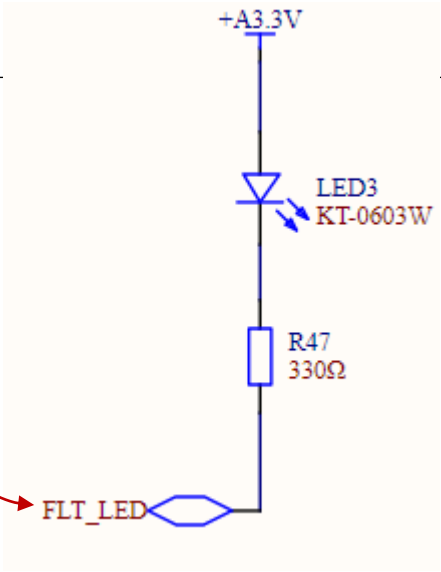
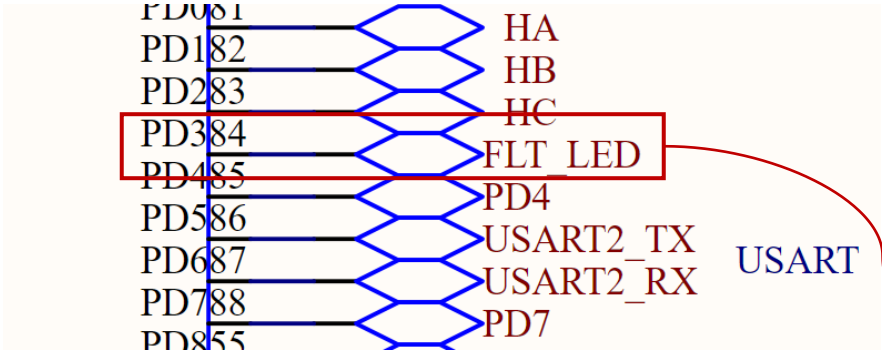
Bits 31:0 **OSPEEDR[15:0][1:0]**: Port x configuration I/O pin y (y = 15 to 0)
These bits are written by software to configure the I/O output speed.
00: Low speed
01: Medium speed 저속(8Mhz), 중속(50Mhz), 고속(100Mhz)
10: High speed
11: Very high speed

Note: Refer to the product datasheets for the values of OSPEEDRy bits versus V_{DD} range and external load.

2. I/O 포트(GPIO)

GPIO Output Data Register (ODR)

- PORTD 3의 출력 On/Off 설정
- ODR비트를 1로 설정 하면 출력 High
- ODR비트를 0로 설정 하면 출력 Low



6.4.6 GPIO port output data register (GPIOx_ODR) (x = A to K)

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Pin 3

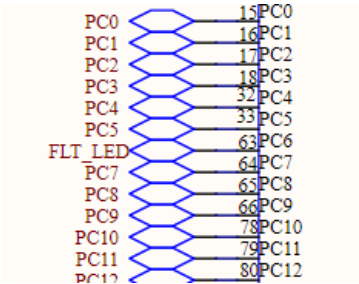
PORTD 3	LED
High	Off
Low	On

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **ODR[15:0]**: Port output data I/O pin y (y = 15 to 0)

These bits can be read and written by software.

Note: For atomic bit set/reset, the ODR bits can be individually set and/or reset by writing to the GPIOx_BSRR or GPIOx_BRR registers (x = A..F).



하드웨어 핀 변경으로 PD3 → PC6으로 변경되었습니다.

2. I/O 포트(GPIO)

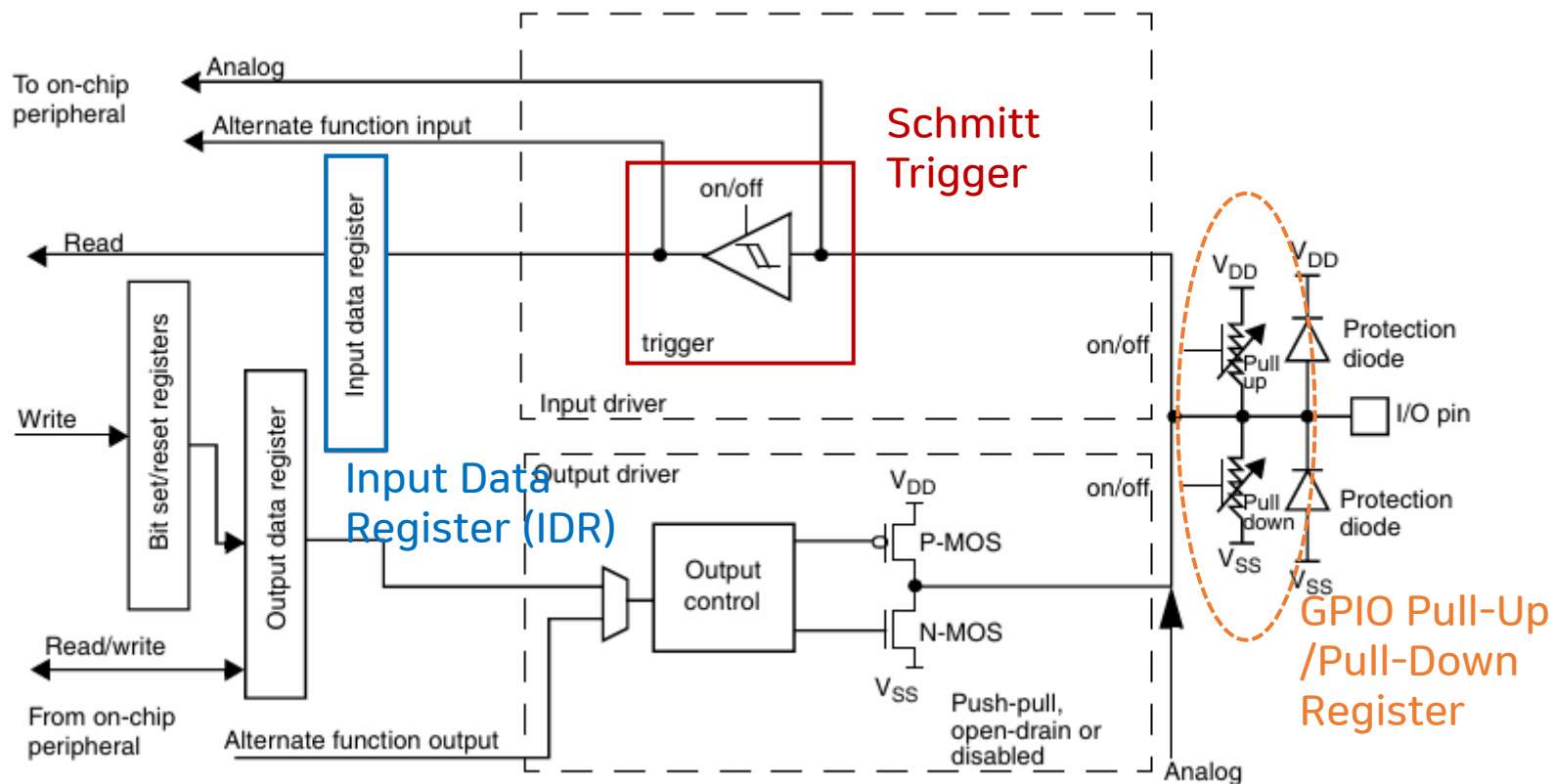
```
1 // PORTD3 LED를 On/Off
2 #include "stm32f7xx.h" // CMSIS 디바이스 헤더
3
4 // 간단 딜레이 (~100ms)
5 static void delay(void) {
6     for (volatile uint32_t i = 0; i < 2000000; i++);
7 }
8
9 int main(void) {
10     // 1) RCC_AHB1ENR: GPIOD 클럭 활성화
11     RCC->AHB1ENR |= 0x00000008; // bit3 = 1 → GPIODEN
12
13     // 2) GPIOD_MODER: PD3을 일반 출력 모드로 설정
14     GPIOD->MODER |= 0x00000040; // set MODER3 = 01 → general-purpose output
15
16     // 4) GPIOD_OSPEEDR: PD3 Medium speed 설정
17     GPIOD->OSPEEDR |= 0x00000040; // set OSPEEDR3 = 01 → Medium speed
18
19     // 6) 무한 루프: PD3 토글 → LED On/Off
20     while (1) {
21         GPIOD->ODR |= 0x00000008; // set ODR3 = 1 (bit3) → LED On
22         delay();
23
24         GPIOD->ODR &= ~0x00000008; // clear ODR3 = 0 (bit3) → LED Off
25         delay();
26     }
27 }
```

Switch + LED

2. I/O 포트(GPIO)

■ GPIO Input

- 채널은 입력, 출력으로 설정하여 사용할 수 있음.
- 외부의 신호(0/3.3V)를 받아들이는 경우 입력
- 슈미트 트리거 회로가 내장 되어 있어서 노이즈에 강인

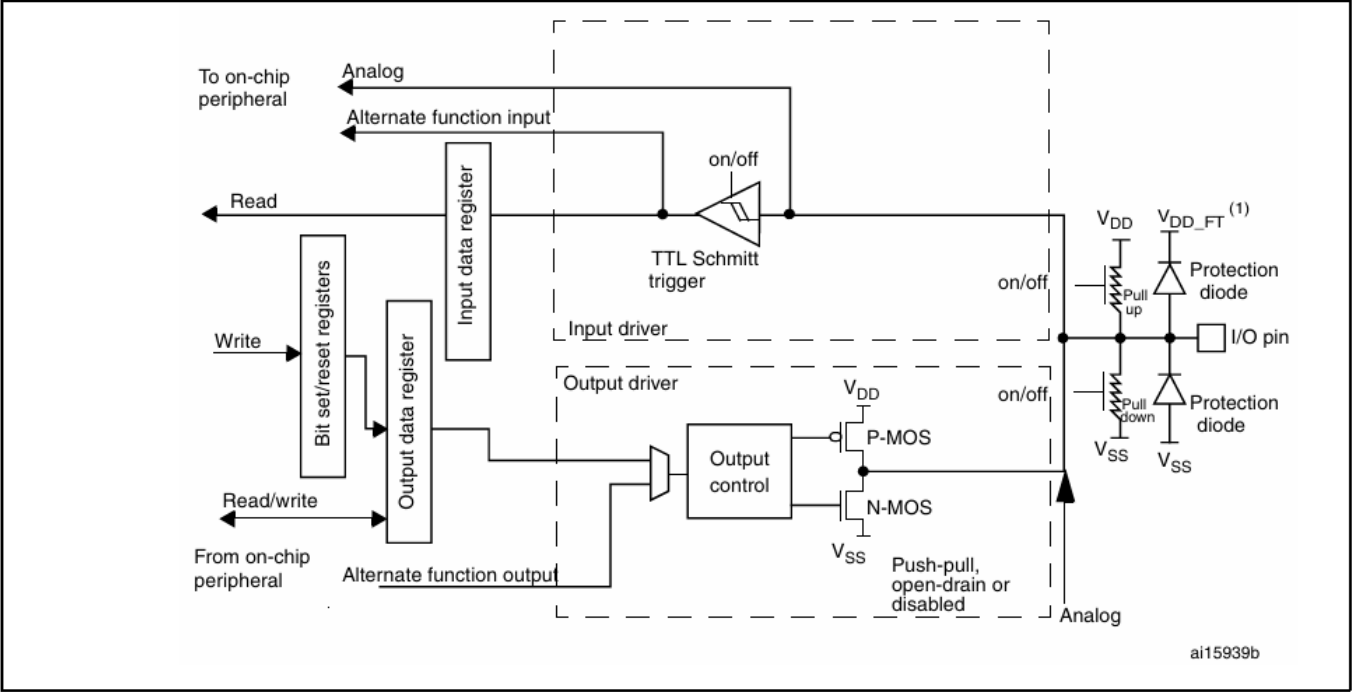


<GPIO Input 설정 블록>

2. I/O 포트(GPIO)

- GPIO Input
 - 5V로 입력을 받을 수 있는 핀들이 있음(ADC는 3.3V만 가능)
 - 데이터시트를 꼭 확인해야 함

Figure 18. Basic structure of a 5-Volt tolerant I/O port bit



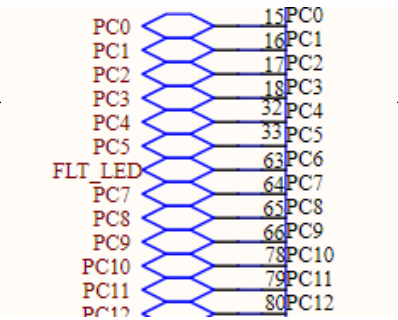
1. V_{DD_FT} is a potential specific to five-volt tolerant I/Os and different from V_{DD} .

Pin Number											Pin name (function after reset)	Pin type	I/O structure	Notes	Alternate functions	Additional functions
STM32F765xx STM32F767xx							STM32F768Ax STM32F769xx									
TFBGA100	LQFP100	LQFP144	UFBGA176	LQFP176	LQFP208	TFBGA216	WLCSP180 ⁽¹⁾	LQFP176	LQFP208	TFBGA216						
C3	3	3	B1	3	3	A1	C12	3	3	A1	PE4	I/O	FT	-	TRACED1, SPI4_NSS, SAI1_FS_A, DFSDM1_DATIN3, FMC_A20, DCMI_D4, LCD_B0, EVENTOUT	-
D3	4	4	B2	4	4	B1	D12	4	4	B1	PE5	I/O	FT	-	TRACED2, TIM9_CH1, SPI4_MISO, SAI1_SCK_A, DFSDM1_CKIN3, FMC_A21, DCMI_D6, LCD_G0, EVENTOUT	-
E3	5	5	B3	5	5	B2	E11	5	5	B2	PE6	I/O	FT	-	TRACED3, TIM1_BKIN2, TIM9_CH2, SPI4_MOSI, SAI1_SD_A, SAI2_MCLK_B, FMC_A22, DCMI_D7, LCD_G1, EVENTOUT	-

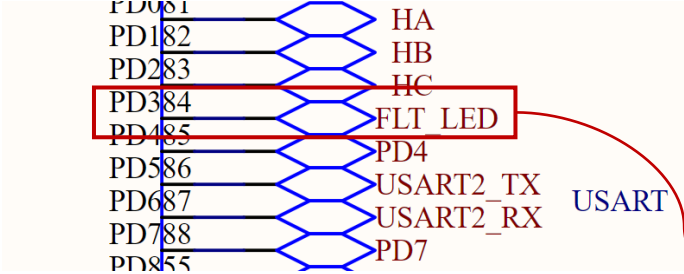
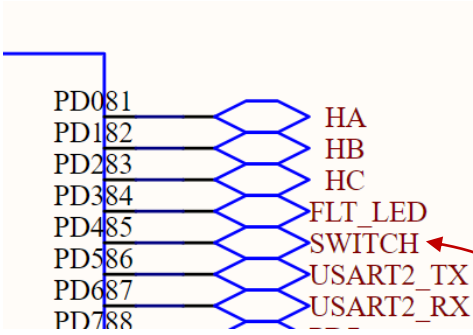
<GPIO Input 설정 블록(5V)>

2. I/O 포트(GPIO)

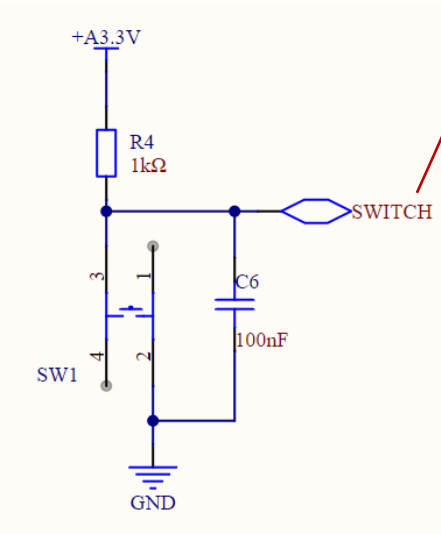
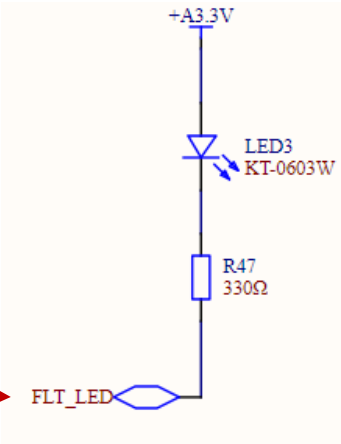
- 실습
 - 스위치 입력 상태에 따라 LED On/Off 실습



하드웨어 핀 변경으로 PD3 → PC6으로 변경되었습니다.



USART



스위치	LED
1(High, No-Push)	OFF
0(Low, Push)	On

<스위치 입력 상태에 따라 LED On-Off>

- AHB1 peripheral clock register (RCC_AHB1ENR)
 - PORTD에 Clock enabled

5.3.10 **RCC AHB1 peripheral clock register (RCC_AHB1ENR)**

Address offset: 0x30
Reset value: 0x0010 0000
Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	OTGHS ULPIEN	OTGHS EN	ETHM ACPTP EN	ETHM ACRX EN	ETHM ACTX EN	ETHMA CEN	Res.	DMA2D EN	DMA2 EN	DMA1 EN	DTCMRA MEN	Res.	BKPSR AMEN	Res.	Res.
	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw		rw		
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	CRC EN	Res.	GPIOK EN	GPIOJ EN	GPIOI EN	GPIOH EN	GPIOG EN	GPIOF EN	GPIOE EN	GIOD EN	GPIOC EN	GPIO BEN	GPIO AEN
			rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 3 **GIODEN**: IO port D clock enable
This bit is set and cleared by software.
0: IO port D clock disabled
1: IO port D clock enabled

2. I/O 포트(GPIO)

GPIO port mode register (GPIOx_MODER) (x=A to K)

- PORTD4 을 Input mode로 설정
- PORTD3 을 Output mode로 설정

6.4.1 GPIO port mode register (GPIOx_MODER) (x =A to K)

Address offset:0x00

Reset value:

- 0xA800 0000 for port A
- 0x0000 0280 for port B
- 0x0000 0000 for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
						Pin 4		Pin 3		Pin 2		Pin 1		Pin 0	

Bits 31:0 **MODER[15:0][1:0]**: Port x configuration I/O pin y (y = 15 to 0)

These bits are written by software to configure the I/O mode.

00: Input mode (reset state)

01: General purpose output mode

10: Alternate function mode

11: Analog mode

2. I/O 포트(GPIO)

- GPIO port pull-up/pull-down register (GPIOx_PUPDR)
 - PORTD 4에 내부 풀업 저항 사용

6.4.4 GPIO port pull-up/pull-down register (GPIOx_PUPDR) (x = A to K)

Address offset: 0x0C

Reset value: 0x6400 0000 (for port A)

Reset value: 0x0000 0100 (for port B)

Reset value: 0x0000 0000 (for other ports)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PUPDR15[1:0]		PUPDR14[1:0]		PUPDR13[1:0]		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PUPDR7[1:0]		PUPDR6[1:0]		PUPDR5[1:0]		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
						Pin 4		Pin 3		Pin 2		Pin 1		Pin 0	

Bits 31:0 **PUPDR[15:0][1:0]**: Port x configuration I/O pin y (y = 15 to 0)

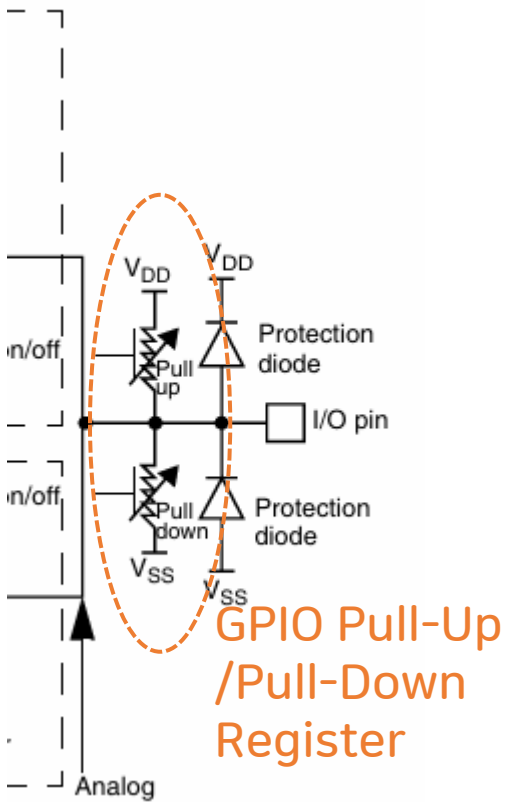
These bits are written by software to configure the I/O pull-up or pull-down

00: No pull-up, pull-down

01: Pull-up

10: Pull-down

11: Reserved



2. I/O 포트(GPIO)

- GPIO port input data register (GPIOx_IDR) (x = A to K)
 - PORTD 4의 입력 데이터 레지스터 -> 신호(3.3V) 들어오면 1(High), 아닌(0V) 경우(Low) 0

6.4.5 GPIO port input data register (GPIOx_IDR) (x = A to K)

Address offset: 0x10

Reset value: 0x0000 XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Pin 4

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **IDR[15:0]**: Port x input data I/O pin y (y = 15 to 0)

These bits are read-only. They contain the input value of the corresponding I/O port.

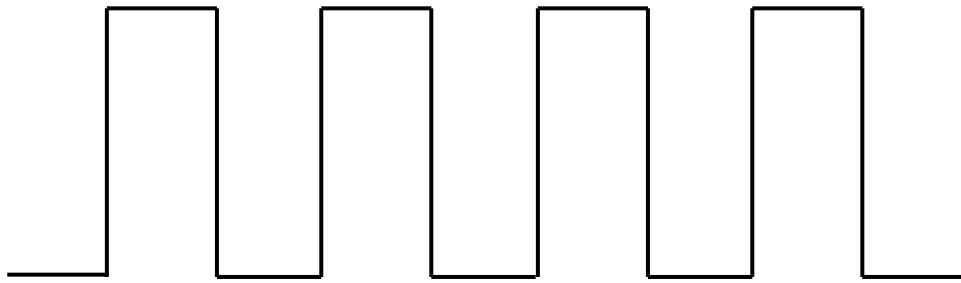
2. I/O 포트(GPIO)

```
1 #include "stm32f7xx.h"
2
3 // 메인 루프에서 LED 제어용 GPIO 입력 읽기 (No blocking delay)
4 int main(void)
5 {
6     // 1) GPIOD 클럭 활성화
7     RCC->AHB1ENR |= 0x00000008;    // bit3 = 1 → GPIODEN
8
9     // — PD3: LED 출력 설정 —
10    // 2) MODE3[1:0] = 01 (general-purpose output)
11    GPIOD->MODER  |= 0x00000040; // set  MODER3 = 01
12
13    // 3) OT3 = 0 (Push-Pull)
14    GPIOD->OTYPER  &= ~0x00000008; // clear OT3 bit 3
15
16    // 4) OSPEEDR3[1:0] = 01 (Medium speed)
17    GPIOD->OSPEEDR |= 0x00000040; // set  OSPEEDR3 = 01
18
19    // — PD4: 스위치 입력 설정 —
20    // Reset 값이 00(input)이지만, 명시적으로 설정
21    GPIOD->MODER   &= ~0x00000300; // clear MODER4 bits 9:8 → 00 = input
22
23    // — 무한 루프: 스위치 상태에 따라 LED On/Off —
24    //   PD4 = 0 (Low) → 스위치 눌림 → LED On
25    //   PD4 = 1 (High) → 스위치 떼짐 → LED Off
26    while (1)
27    {
28        if ((GPIOD->IDR & 0x00000010) == 0)
29        {
30            GPIOD->ODR &= ~0x00000008; // clear ODR3 = 0 → LED On
31        }
32        else
33        {
34            GPIOD->ODR |= 0x00000008; // set ODR3 = 1 → LED Off
35        }
36    }
37 }
```

3. Clock

■ 클럭이란?

- MCU는 내부적으로 많은 일들을 수행하는데 클럭에 동기가 되어 실행함
- 즉 클럭 주파수가 높으면 더 빠르게 명령어 처리가 가능함
- STM32F767은 최대 216MHz로 동작이 가능

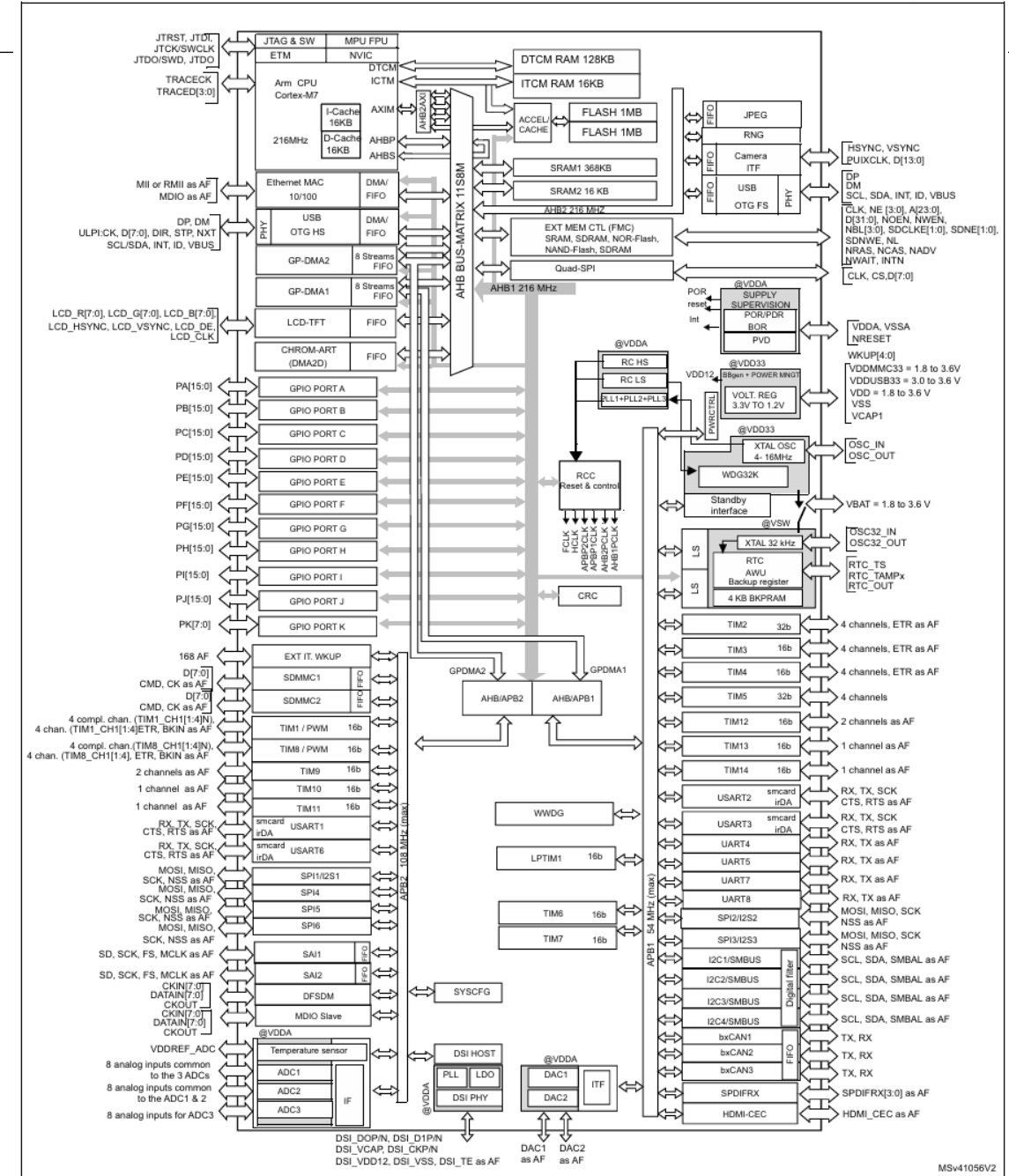


3. Clock

■ STM32F767의 내부 블록도

- STM32F767은 AHB, APB1/APB2 버스를 통해 주변 장치와 클럭에 동기화되어 데이터를 주고 받음
- 각 모듈마다 프리스케일러를 이용해서 클럭 주파수를 분주할 수 있음

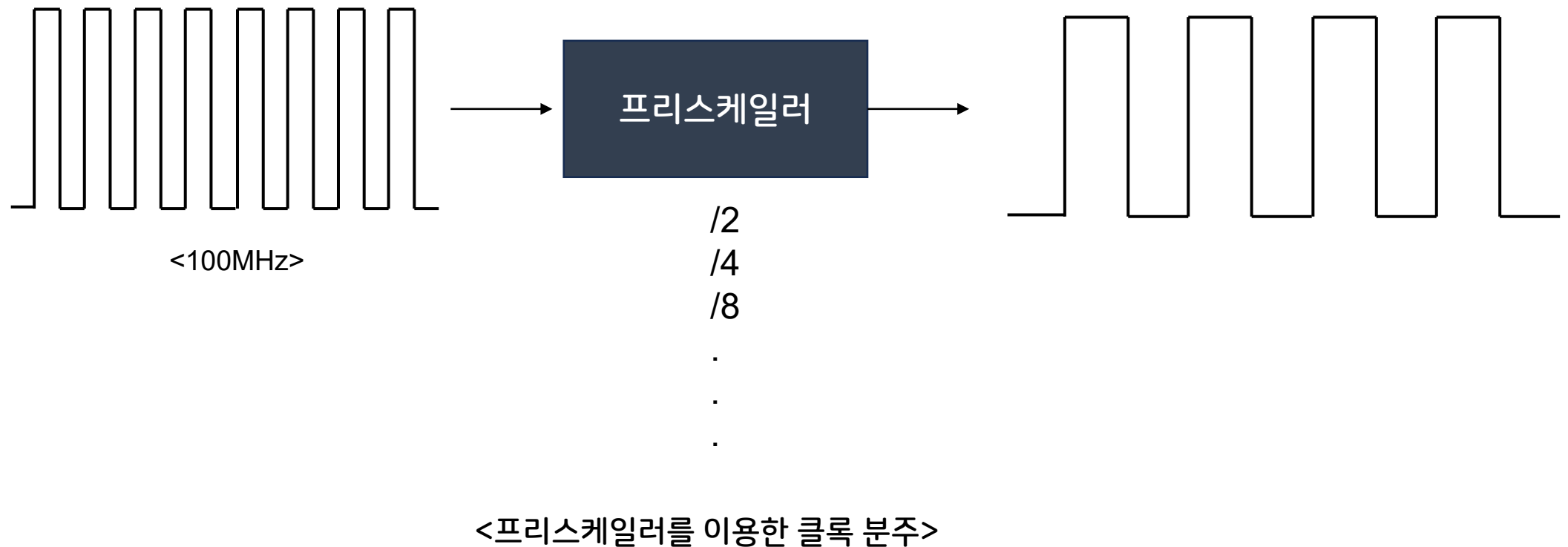
Figure 2. STM32F765xx, STM32F767xx, STM32F768Ax, and STM32F769xx block diagram



3. Clock

■ STM32F767의 내부 블록도

- 각 모듈마다 프리스케일러를 이용해서 클럭 주파수를 분주할 수 있음
- 입력 주파수가 100MHz이고 프리스케일러의 분주비를 /2로 하는 경우 50MHz로 동작
- 적절한 주파수를 얻기 위하여 프리스케일러 사용



3. Clock

■ HSE (High Speed External) 클럭

- HSE 클럭 신호는 아래와 같이 OSC_IN, OSC_OUT에 4~26Mhz 크리스탈 or 세라믹 레조네이터를 접속하여 만들어줄 수 있음
- 또는 외부의 구형파 클럭 주파수으로도 동작 가능
- 외부 크리스탈을 접속하는 경우 커패시터 C_{L1} , C_{L2} 의 세라믹 커패시터를 사용
- 회로설계 강의에서 자세히 다룸

■ HSI (High Speed Internal) 클럭

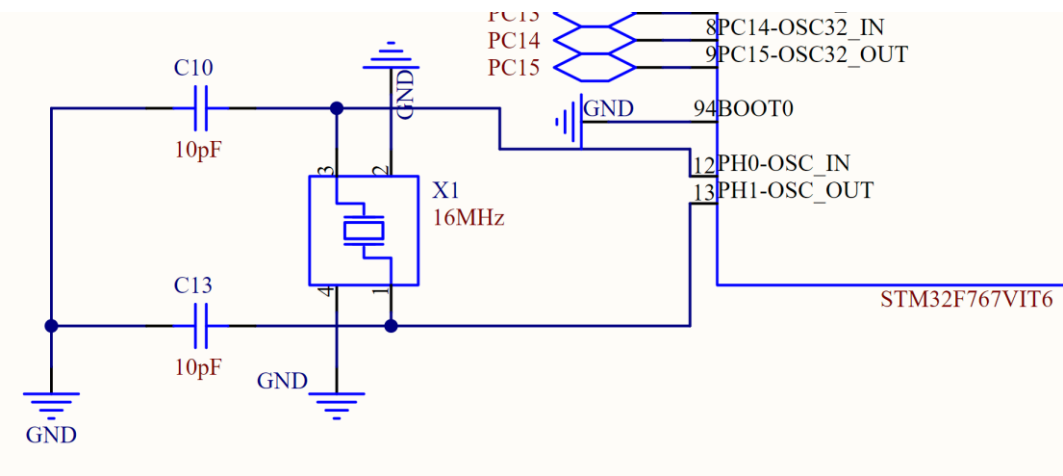
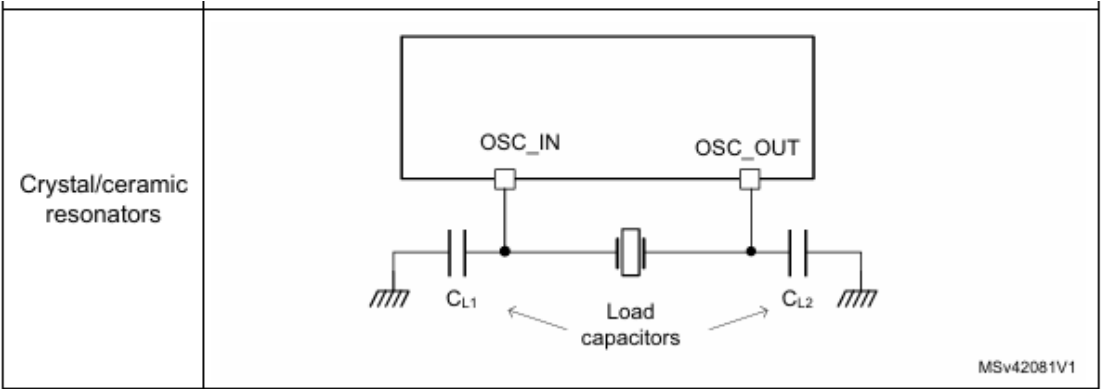
- 내부 RC 오실레이터에 의하여 발생 → 16Mhz
- 직접 SYSCLK으로 공급 & PLL 클럭 소스로 사용
- 외부 소자를 사용하지 않고 내부 회로로 동작하므로 **원가 절감**을 할 수 있지만 정밀도가 떨어짐
- MCU가 리셋된 후에는 디폴트로 HSI 클럭을 SYSCLK으로 사용함
- HSE 오실레이터가 동작 실패하거나 정지 및 대기모드에서도 HIS 사용

■ LSE (Low Speed External) 클럭

- LSE는 OSC32_IN , OSC32_OUT에 32.768kHz의 낮은 주파수의 크리스탈 또는 레조네이터를 연결하여 사용
- 저전력으로 RTC에 공급하기 위함

■ LSI (Low Speed Internal) 클럭

- 내부 RC 오실레이터로 발생되는 32kHz의 저주파 클럭 신호
- 정지 모드나 대기 모드에서 동작
- IWDG, RTC에 클럭 공급

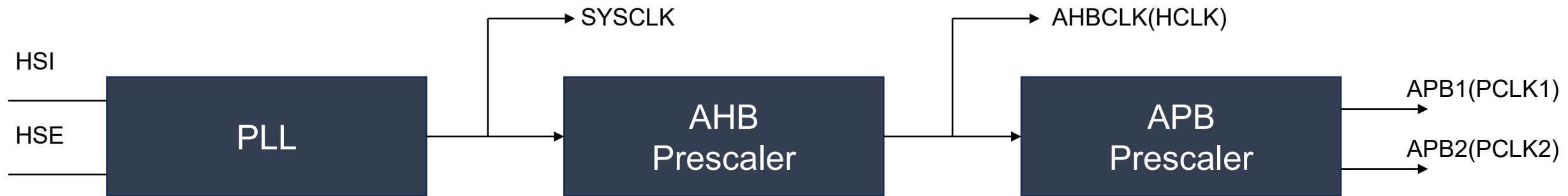
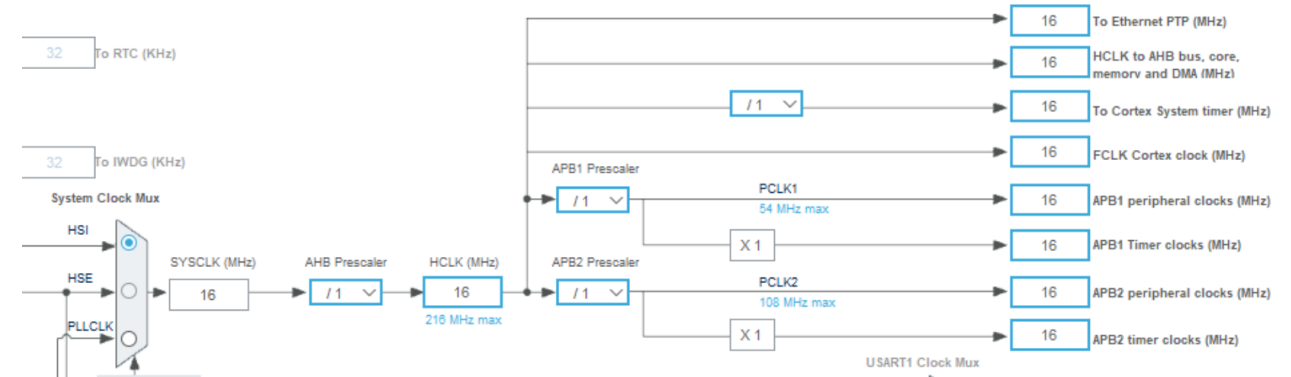


< HSE 클럭 사용을 위한 외부 크리스탈 사용 >

3. Clock

■ PLL(Phase-Locked Loop) 회로

- PLL은 HSI나 HSE 클럭 신호를 입력으로 받아서 이보다 더 높은 출력 주파수를 발생 시키는 주파수 체배 회로이다.
- PLL의 출력 주파수 범위 → 24~216MHz
- RCC_CR 레지스터의 PLLON 비트를 0으로 설정하여 PLL 회로 정지
- RCC_CR 레지스터의 PLLRDY 비트가 0으로 되기까지 대기
- RCC_PLLCFGR 레지스터를 사용하여 PLL 설정
- RCC_CR 레지스터의 PLLON 비트를 1로 설정하여 PLL 회로 동작
- RCC_CR 레지스터의 PLLRDY 비트가 1로 되기까지 대기



<PLL 블록 다이어그램>

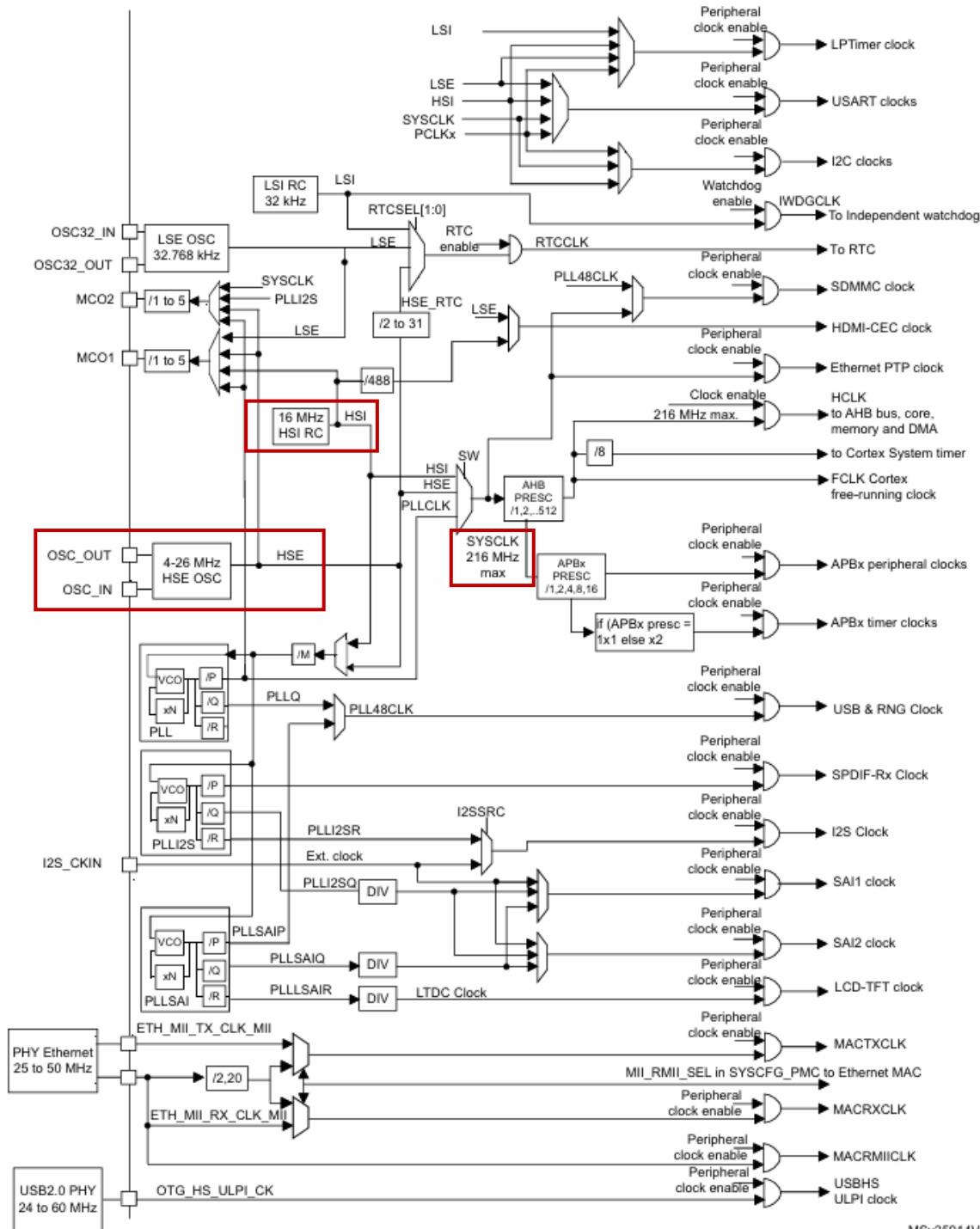
3. Clock

■ Clock 제어기

- HSI(16Mhz), HSE(4~26Mhz), PLL : SYSCLK을 발생하는 소스
- LSI(32kHz) : RTC, 위치독 타이머에서 사용
- LSE(32.768kHz) : RTC

❖ 클럭 셋팅 과정

1. 리셋 된 후 RCC_CFGR 레지스터에서 디폴트로 HSI 클럭이 SYSCLK으로 선택 됨, HCLK 프리스케일도 1로 설정되므로 CPU 코어는 16Mhz의 HCLK에 의하여 동작
2. SYSCLK을 HSI가 아닌 다른 클럭으로 변경하려면 현재 클럭의 동작이 안정화 되어야 함(RCC_CR 레지스터에서 안정 상태 표시 비트로 확인)
3. 이후 클럭 레지스터 설정을 변경하여 SYSCLK을 바꿀 수 있음
4. 교육용 키트에서는 AHB, APB1, APB2 버스 클럭을 각각 216Mhz, 54MHz, 54Mhz로 사용 SysTick은 SysTick_CTRL 레지스터 비트 설정에 따라 HCLK를 사용
5. 타이머의 경우 APB 프리스케일러가 1로 설정된 경우에는 PCLK 클럭이 그대로 타이머에 공급되지만, 프리스케일러가 1이 아닌 경우, PCLK 클럭 주파수의 2배가 타이머에 공급됨



3. Clock

- Clock 제어기

Table 5. Number of wait states according to CPU clock (HCLK) frequency

Wait states (WS) (LATENCY)	HCLK (MHz)			
	Voltage range 2.7 V - 3.6 V	Voltage range 2.4 V - 2.7 V	Voltage range 2.1 V - 2.4 V	Voltage range 1.8 V - 2.1 V
0 WS (1 CPU cycle)	$0 < \text{HCLK} \leq 30$	$0 < \text{HCLK} \leq 24$	$0 < \text{HCLK} \leq 22$	$0 < \text{HCLK} \leq 20$
1 WS (2 CPU cycles)	$30 < \text{HCLK} \leq 60$	$24 < \text{HCLK} \leq 48$	$22 < \text{HCLK} \leq 44$	$20 < \text{HCLK} \leq 40$
2 WS (3 CPU cycles)	$60 < \text{HCLK} \leq 90$	$48 < \text{HCLK} \leq 72$	$44 < \text{HCLK} \leq 66$	$40 < \text{HCLK} \leq 60$
3 WS (4 CPU cycles)	$90 < \text{HCLK} \leq 120$	$72 < \text{HCLK} \leq 96$	$66 < \text{HCLK} \leq 88$	$60 < \text{HCLK} \leq 80$
4 WS (5 CPU cycles)	$120 < \text{HCLK} \leq 150$	$96 < \text{HCLK} \leq 120$	$88 < \text{HCLK} \leq 110$	$80 < \text{HCLK} \leq 100$
5 WS (6 CPU cycles)	$150 < \text{HCLK} \leq 180$	$120 < \text{HCLK} \leq 144$	$110 < \text{HCLK} \leq 132$	$100 < \text{HCLK} \leq 120$
6 WS (7 CPU cycles)	$180 < \text{HCLK} \leq 210$	$144 < \text{HCLK} \leq 168$	$132 < \text{HCLK} \leq 154$	$120 < \text{HCLK} \leq 140$
7 WS (8 CPU cycles)	$210 < \text{HCLK} \leq 216$	$168 < \text{HCLK} \leq 192$	$154 < \text{HCLK} \leq 176$	$140 < \text{HCLK} \leq 160$
8 WS (9 CPU cycles)	-	$192 < \text{HCLK} \leq 216$	$176 < \text{HCLK} \leq 198$	$160 < \text{HCLK} \leq 180$
9 WS (10 CPU cycles)	-	-	$198 < \text{HCLK} \leq 216$	-

3. Clock

▪ Clock 제어기

5.3.1 **RCC clock control register (RCC_CR)**

Address offset: 0x00

Reset value: 0x0000 XX83 where X is undefined.

Access: no wait state, word, half-word and byte access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	PLLSAI RDY	PLLSAI ON	PLLI2S RDY	PLLI2S ON	PLLRD Y	PLLON	Res.	Res.	Res.	Res.	CSS ON	HSE BYP	HSE RDY	HSE ON
		r	rw	r	rw	r	rw					rw	rw	r	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
HSICAL[7:0]								HSITRIM[4:0]					Res.	HSI RDY	HSION
r	r	r	r	r	r	r	r	rw	rw	rw	rw	rw		r	rw

Bit 0 **HSION**: Internal high-speed clock enable

Set and cleared by software.
Set by hardware to force the HSI oscillator ON when leaving the Stop or Standby mode or in case of a failure of the HSE oscillator used directly or indirectly as the system clock. This bit cannot be cleared if the HSI is used directly or indirectly as the system clock.
0: HSI oscillator OFF
1: HSI oscillator ON

Bit 1 **HSIRDY**: Internal high-speed clock ready flag

Set by hardware to indicate that the HSI oscillator is stable. After the HSION bit is cleared, HSIRDY goes low after 6 HSI clock cycles.
0: HSI oscillator not ready
1: HSI oscillator ready

Bit 16 **HSEON**: HSE clock enable

Set and cleared by software.
Cleared by hardware to stop the HSE oscillator when entering Stop or Standby mode. This bit cannot be reset if the HSE oscillator is used directly or indirectly as the system clock.
0: HSE oscillator OFF
1: HSE oscillator ON

3. Clock

■ Clock 제어기

5.3.3 RCC clock configuration register (RCC_CFGR)

Address offset: 0x08

Reset value: 0x0000 0000

Access: 0 ≤ wait state ≤ 2, word, half-word and byte access

1 or 2 wait states inserted only if the access occurs during a clock source switch.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MCO2		MCO2 PRE[2:0]			MCO1 PRE[2:0]			I2SSC R	MCO1		RTCPRE[4:0]				
rw		rw	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PPRE2[2:0]			PPRE1[2:0]			Res.	Res.	HPRE[3:0]				SWS1	SWS0	SW1	SW0
rw	rw	rw	rw	rw	rw			rw	rw	rw	rw	r	r	rw	rw

Bits 3:2 **SWS**: System clock switch status

Set and cleared by hardware to indicate which clock source is used as the system clock.

00: HSI oscillator used as the system clock

01: HSE oscillator used as the system clock

10: PLL used as the system clock

11: not applicable

Bits 1:0 **SW**: System clock switch

Set and cleared by software to select the system clock source.

Set by hardware to force the HSI selection when leaving the Stop or Standby mode or in case of failure of the HSE oscillator used directly or indirectly as the system clock.

00: HSI oscillator selected as system clock

01: HSE oscillator selected as system clock

10: PLL selected as system clock

11: not allowed

3. Clock

■ Clock 제어기

5.3.2 RCC PLL configuration register (RCC_PLLCFGR)

Address offset: 0x04

Reset value: 0x2400 3010

Access: no wait state, word, half-word and byte access.

This register is used to configure the PLL clock outputs according to the formulas:

- $f_{(\text{VCO clock})} = f_{(\text{PLL clock input})} \times (\text{PLL}N / \text{PLL}M)$
- $f_{(\text{PLL general clock output})} = f_{(\text{VCO clock})} / \text{PLL}P$
- $f_{(\text{USB OTG FS, SDMMC, RNG clock output})} = f_{(\text{VCO clock})} / \text{PLL}Q$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	PLLQ[3:0]				Res.	PLLSR C	Res.	Res.	Res.	Res.	PLL[P:1:0]	
				rw	rw	rw	rw		rw					rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	PLL[N:8:0]								PLL[M:5:0]						
	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

1001 0100 0000 0011 0110 0000 1000

▪ Clock 제어기

5.3.13 **RCC APB1 peripheral clock enable register (RCC_APB1ENR)**

Address offset: 0x40

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
UART8 EN	UART7 EN	DAC EN	PWR EN	CEC EN	CAN2 EN	CAN1 EN	I2C4 EN	I2C3 EN	I2C2 EN	I2C1 EN	UART5 EN	UART4 EN	USART 3 EN	USART 2 EN	SPDIFRX EN
rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw	rw

Bit 28 **PWREN:** Power interface clock enable

This bit is set and cleared by software.

0: Power interface clock disabled

1: Power interface clock enable

3. Clock

■ Clock 제어기

4.4.1 PWR power control register (PWR_CR1)

Address offset: 0x00

Reset value: 0x0000 C000 (reset by wakeup from Standby mode)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	UDEN[1:0]		ODSWEN	ODEN
												rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VOS[1:0]		ADCD1	Res.	MRUDS	LPUDS	FPDS	DBP	PLS[2:0]			PVDE	CSBF	Res.	PDDS	LPDS
rw	rw	rw		rw	rw	rw	rw	rw	rw	rw	rw	rc_w1		rw	rw

Bit 16 **ODEN**: Over-drive enable

This bit is set by software. It is cleared automatically by hardware after exiting from Stop mode. It is used to enabled the Over-drive mode in order to reach a higher frequency. To set or reset the ODEN bit, the HSI or HSE must be selected as system clock. When the ODEN bit is set, the application must first wait for the Over-drive ready flag (ODRDY) to be set before setting the ODSWEN bit.

- 0: Over-drive disabled
- 1: Over-drive enabled

Entering Over-drive mode

It is recommended to enter Over-drive mode when the application is not running critical tasks and when the system clock source is either HSI or HSE. To optimize the configuration time, enable the Over-drive mode during the PLL lock phase.

To enter Over-drive mode, follow the sequence below:

1. Select HSI or HSE as system clock.
2. Configure RCC_PLLCFGR register and set PLLON bit of RCC_CR register.
3. Set ODEN bit of PWR_CR1 register to enable the Over-drive mode and wait for the ODRDY flag to be set in the PWR_CSR1 register.
4. Set the ODSW bit in the PWR_CR1 register to switch the voltage regulator from Normal mode to Over-drive mode. The System will be stalled during the switch but the PLL clock system will be still running during locking phase.
5. Wait for the ODSWRDY flag in the PWR_CSR1 to be set.
6. Select the required Flash latency as well as AHB and APB prescalers.
7. Wait for PLL lock.
8. Switch the system clock to the PLL.
9. Enable the peripherals that are not generated by the System PLL (I2S clock, LCD-TFT clock, SAI1 clock, USB_48MHz clock....).

The PLLI2S and PLLSAI can be configured at the same time as the system PLL.

During the Over-drive switch activation, no peripheral clocks should be enabled. The peripheral clocks must be enabled once the Over-drive mode is activated.

Entering Stop mode disables the Over-drive mode, as well as the PLL. The application software has to configure again the Over-drive mode and the PLL after exiting from Stop mode.

3. Clock

■ Clock 제어기

5.3.25 RCC dedicated clocks configuration register (RCC_DCKCFGR1)

Address offset: 0x8C

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access.

This register allows to configure the timer clock prescalers and the PLLSAI and PLLI2S output clock dividers for SAI and LTDC peripherals according to the following formula:

$$f_{(\text{PLLSAIDIVQ clock output})} = f_{(\text{PLLSAI_Q})} / \text{PLLSAIDIVQ}$$

$$f_{(\text{PLLSAIDIVR clock output})} = f_{(\text{PLLSAI_R})} / \text{PLLSAIDIVR}$$

$$f_{(\text{PLLI2SDIVQ clock output})} = f_{(\text{PLLI2S_Q})} / \text{PLLI2SDIVQ}$$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	TIMPRE	SAI2SEL[1:0]		SAI1SEL[1:0]		Res.	Res.	PLLSAIDIVR[1:0]	
							rw	rw	rw	rw	rw			rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	PLLSAIDIVQ[4:0]					Res.	Res.	Res.	PLLI2SDIVQ[4:0]				
			rw	rw	rw	rw	rw				rw	rw	rw	rw	rw

Bits 31:25 Reserved, must be kept at reset value.

Bit 24 **TIMPRE**: Timers clocks prescalers selection

This bit is set and reset by software to control the clock frequency of all the timers connected to APB1 and APB2 domain.

0: If the APB prescaler (PPRE1, PPRE2 in the RCC_CFGR register) is configured to a division factor of 1, TIMxCLK = PCLKx. Otherwise, the timer clock frequencies are set to twice to the frequency of the APB domain to which the timers are connected:

$$\text{TIMxCLK} = 2 \times \text{PCLKx}.$$

1: If the APB prescaler (PPRE1, PPRE2 in the RCC_CFGR register) is configured to a division factor of 1, 2 or 4, TIMxCLK = HCLK. Otherwise, the timer clock frequencies are set to four times to the frequency of the APB domain to which the timers are connected:

$$\text{TIMxCLK} = 4 \times \text{PCLKx}.$$

▪ Clock 제어기

5.3.14 **RCC APB2 peripheral clock enable register (RCC_APB2ENR)**

Address offset: 0x44

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	LTDC EN	Res.	Res.	SAI2EN	SAI1EN	SPI6EN	SPI5EN	Res.	TIM11 EN	TIM10 EN	TIM9 EN
					rw			rw	rw	rw	rw		rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	SYSCFG EN	SPI4 EN	SPI1 EN	SDMMC1 EN	ADC3 EN	ADC2 EN	ADC1 EN	Res.	Res.	USART6 EN	USART1 EN	Res.	Res.	TIM8 EN	TIM1 EN
	rw	rw	rw	rw	rw	rw	rw			rw	rw			rw	rw

Bit 14 **SYSCFGEN**: System configuration controller clock enable

This bit is set and cleared by software.

0: System configuration controller clock disabled

1: System configuration controller clock enabled

▪ Clock 제어기

7.2.7 Compensation cell control register (SYSCFG_CMPCR)

Address offset: 0x20

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	READY	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CMP_PD
							r								rw

Bit 0 **CMP_PD**: Compensation cell power-down
0: I/O compensation cell power-down mode
1: I/O compensation cell enabled

3. Clock & Reset

```
void Initialize_MCU(void)          /* initialize STM32F767VGT6 MCU */
{
    // (1) 명령 캐시 및 데이터 캐시 설정
    SCB_EnableICache();            // enable L1 instruction cache
    SCB_EnableDCache();            // enable L1 data cache

    // (2) ART 가속기, 프리페치 버퍼, 웨이트 사이클 설정
    FLASH->ACR = 0x00000307;        // 7 waits, enable ART accelerator and prefetch

    // (3) HSE 및 PLL 설정(시스템 클럭 SYSCLK = 216MHz)
    RCC->CR |= 0x00010001;          // HSE on, HSI on
    while((RCC->CR & 0x00000002) == 0); // wait until HSIRDY = 1
    RCC->CFGR = 0x00000000;          // SYSCLK = HSI
    while((RCC->CFGR & 0x0000000C) != 0); // wait until SYSCLK = HSI

    RCC->CR = 0x00010001;           // PLL off, HSE on, HSI on
    RCC->PLLCFGR = 0x09403608;      // SYSCLK = HSE*PLLN/PLLM/PLLP = 16MHz*216/8/2 = 216MHz
                                   // PLL48CK = HSE*PLLN/PLLM/PLLQ = 16MHz*216/8/9 = 48MHz
    RCC->CR = 0x01010001;           // PLL on, HSE on, HSI on
    while((RCC->CR & 0x02000000) == 0); // wait until PLLRDY = 1

    // (4) 오버드라이브 설정
    RCC->APB1ENR |= 0x10000000;      // 전원모듈 클럭(PWREN = 1)
    PWR->CR1 |= 0x00010000;          // over-drive enable(ODEN = 1)
    while((PWR->CSR1 & 0x00010000) == 0); // ODRDY = 1 ?
    PWR->CR1 |= 0x00020000;          // over-drive switching enable(ODSWEN = 1)
    while((PWR->CSR1 & 0x00020000) == 0); // ODSRDY = 1 ?

    // (5) 주변장치 클럭 설정(APB1CLK = APB2CLK = 54MHz)
    RCC->CFGR = 0x3040B402;          // SYSCLK = PLL, AHB = 216MHz, APB1 = APB2 = 54MHz
    RCC->DCKCFGR1 = 0x01000000;      // TIMxCLK = 216MHz
    while((RCC->CFGR & 0x0000000C) != 0x00000008); // wait until SYSCLK = PLL
    RCC->CR |= 0x00080000;           // CSS on
}
```

캐시를 사용하여 코드 실행 속도 향상

- HSE, HSI 모두 On
- HSI 안정 확인
- 임시로 HSI를 SYSCLK으로 사용
- While문으로 SYSCLK 설정 완료 확인
- HSE, HSI ON 유지 후 PLL 설정
- HSE를 PLL입력으로 사용하여 216MHz를 만들어 SYSCLK으로 사용
- PLL 동작 대기

- 216MHz 사용을 위해선 내부 LDO 전압을 더 높여서 사용해야 함
- 대신 소비 전류가 증가함

- 블록 최종 분주비 결정하여
- SYSCLK, AHB = 216MHz
- APB1/APB2 = 54MHz

Interrupt

4. 인터럽트(NVIC, EXTI)

■ Polling vs Interrupt

- ✓ Polling 방식 : 치킨 배달을 시킨 뒤 배달이 올때까지 계속 문 앞에서 기다린다.

```
/*Polling*/  
While(1)  
{  
    문 앞에서 계속 기다리기  
    if(치킨도착)  
        exit;  
}
```

- ✓ Interrupt 방식 : 현재 하고 있는 일을 진행하고 있다가 벨이 울리면 치킨을 받으러 간다.

```
/*Interrupt*/  
Interrupt_Handler()  
{  
    치킨수령  
    printf("감사합니다.");  
}
```

4. 인터럽트(NVIC, EXTI)

■ 인터럽트 개념

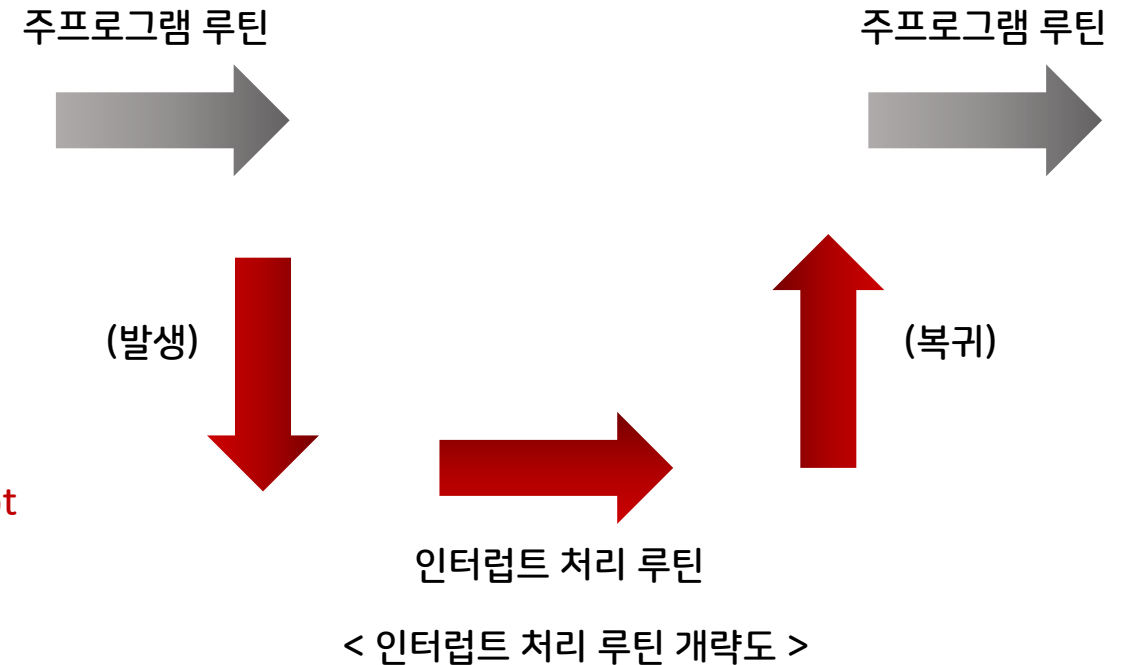
- MCU 코어는 외부 소자와 연결하기 위해 다양한 주변 장치와 연결
- 이러한 주변 장치에서 인터럽트가 발생하면 현재 하던 작업을 잠시 멈추고 인터럽트를 처리함

■ NVIC(Nested Vectored Interrupt Controller)

- Cortex-M 시리즈를 사용하는 경우 ARM에서 개발한 NVIC라는 Interrupt Controller를 공통적으로 사용

■ 인터럽트 처리 방법

- 기본적으로 인터럽트가 발생하면, 발생한 **인터럽트가 호출할 함수인 ISR(Interrupt Service Routine)**을 미리 코딩해둠
- 인터럽트 발생 시 호출될 함수 ISR의 시작 번지들을 **우선순위에 따라** 저장해 놓음 (**Interrupt Vector Table**)
- 인터럽트가 발생하면 NVIC가 **우선순위에 따라서** Core의 현재 작업을 일시 중단하고, ISR을 수행함

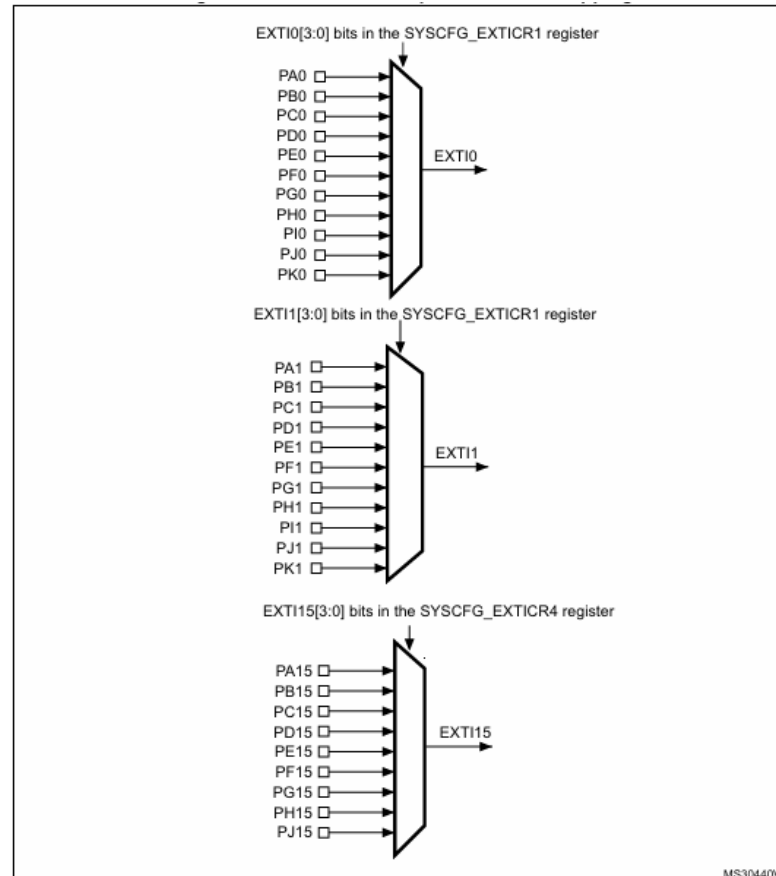


벡터테이블 확인 레퍼런스 메뉴얼

4. 인터럽트(NVIC, EXTI)

■ EXTI(Extended Interrupt and Event Controller)

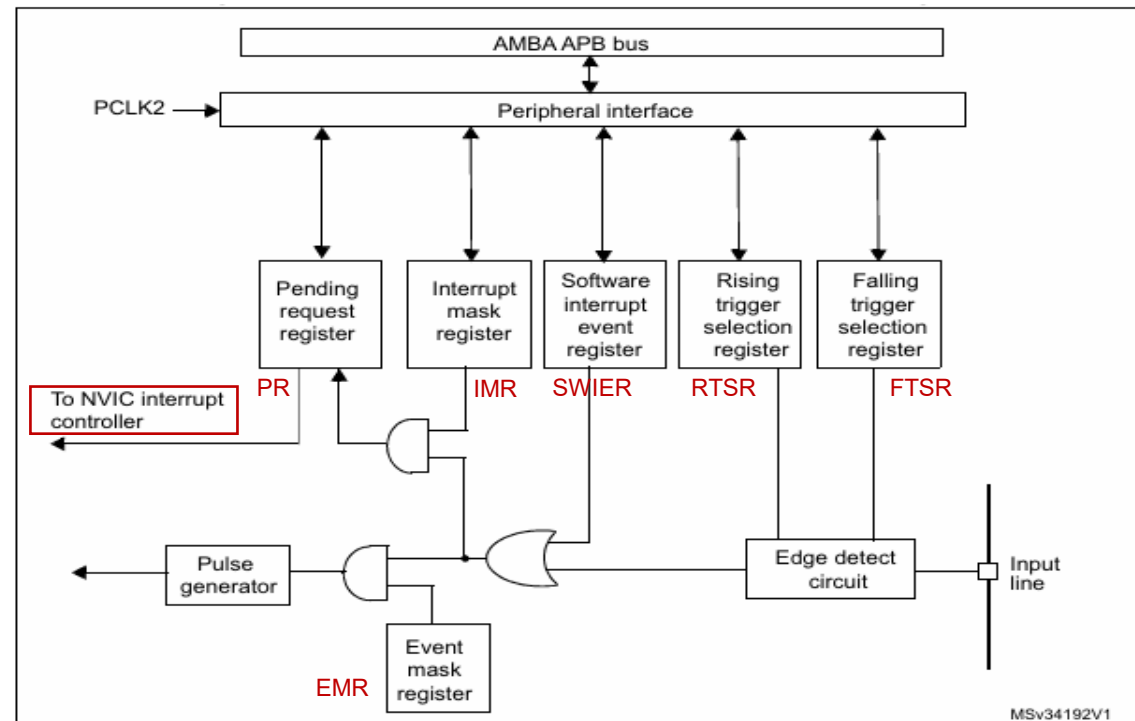
- GPIO로 입력되는 외부 인터럽트 EXTI0~EXTI15는 SYSCFG_EXTICRx 레지스터를 사용하여 인터럽트 신호 단자 선택 가능
- EXTI0~EXTI4는 각각 개별적인 인터럽트 벡터 존재
- EXTI9_5, EXTI15_10과 같이 여러 개의 인터럽트가 하나의 인터럽트 벡터를 중복하여 사용



4. 인터럽트(NVIC, EXTI)

■ EXTI(Extended Interrupt and Event Controller)

- EXTI는 CPU 코어가 아닌, STM32F767 마이크로컨트롤러의 주변장치에 해당하는 인터럽트 제어기
- 내부 또는 외부에서 입력되는 인터럽트 신호를 인식하여 CPU의 인터럽트 제어기인 NVIC에게 전달
- EXTI가 처리하는 모든 인터럽트/이벤트는 각각 인터럽트 마스크, 이벤트 마스크 레지스터를 이용하여 개별적으로 허용&금지 설정 가능
- 신호의 상승 에지 또는 하강 에지 입력 선택 가능
- 인터럽트 신호가 발생하면 인터럽트 대기 레지스터의 해당 비트가 1로 설정됨

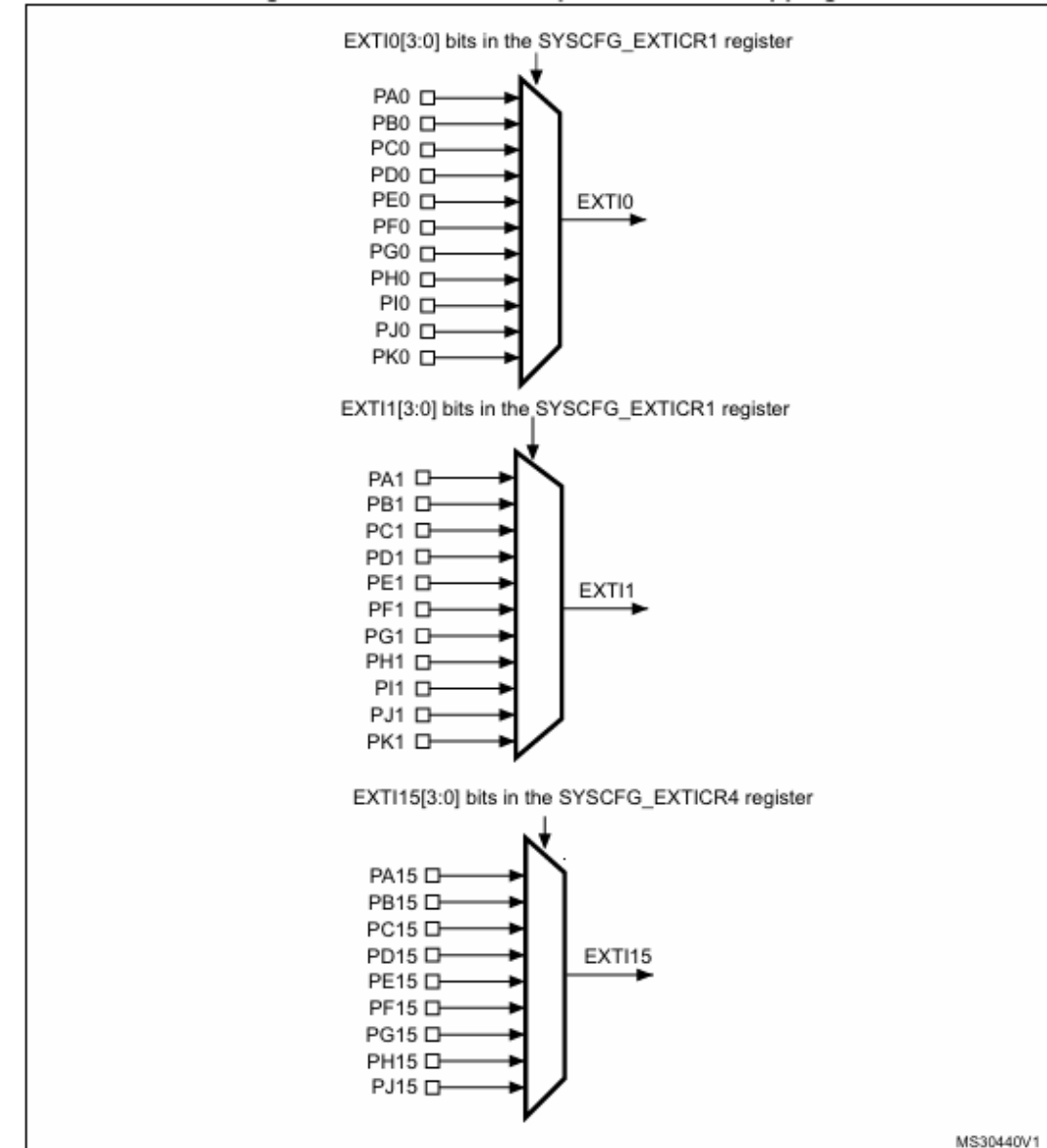
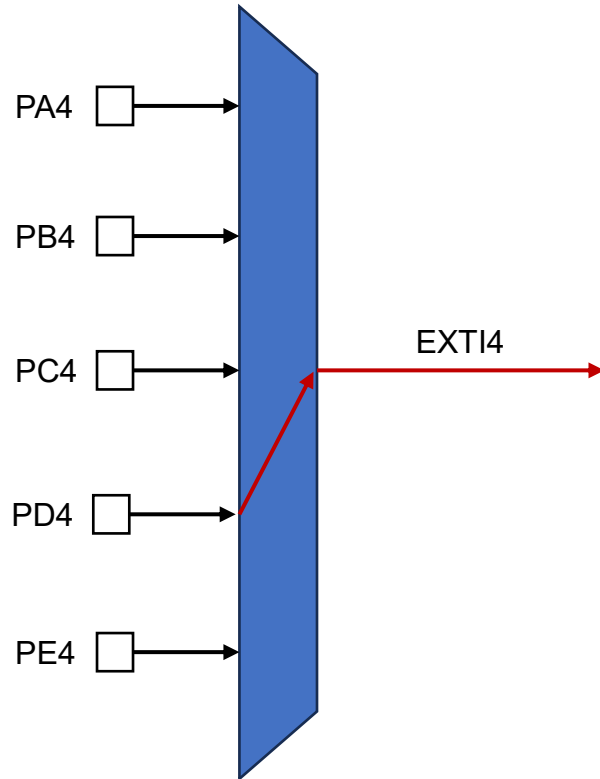


< EXTI 내부 블록도 >

4. 인터럽트(NVIC, EXTI)

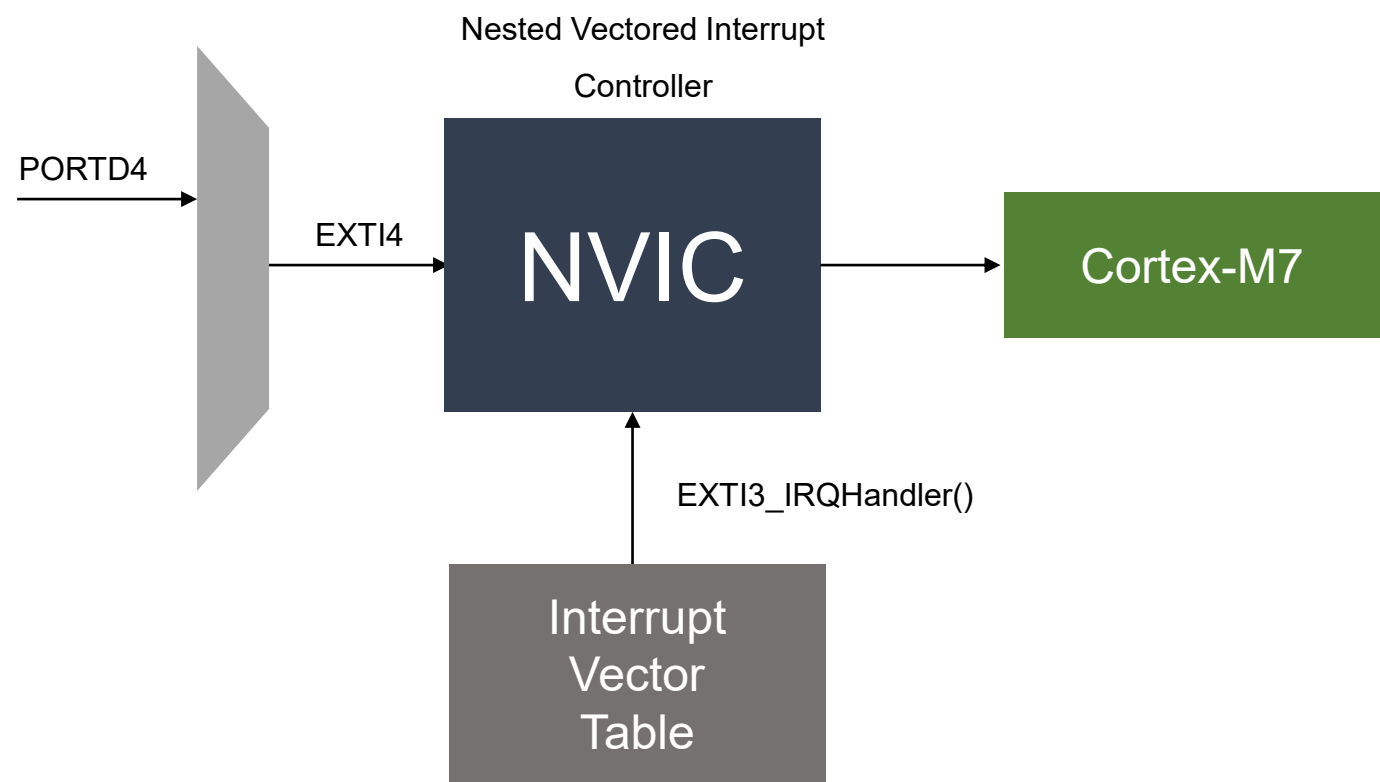
■ EXTI(Extended Interrupt and Event Controller)

- PD4의 스위치 신호를 검출하여 외부 인터럽트 발생



4. 인터럽트(NVIC, EXTI)

- PORTD 4 스위치 On/Off (인터럽트 사용)
 - PORTD 4의 스위치 신호를 이용하여 인터럽트 발생

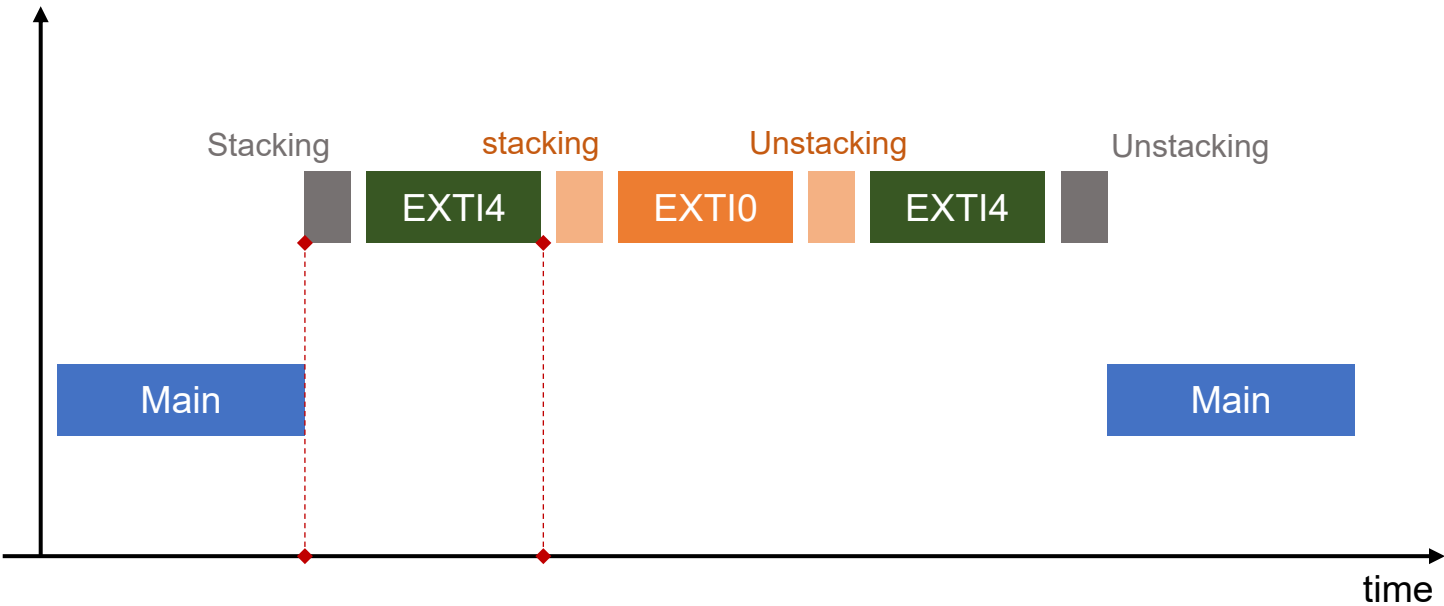


<인터럽트 처리를 위한 NVIC>

Position	Priority	Type of priority	Acronym	Description	Address
-	3	settable	SVCall	System service call via SWI instruction	0x0000 002C
-	4	settable	Debug Monitor	Debug Monitor	0x0000 0030
-	-	-	-	Reserved	0x0000 0034
-	5	settable	PendSV	Pendable request for system service	0x0000 0038
-	6	settable	SysTick	System tick timer	0x0000 003C
0	7	settable	WWDG	Window Watchdog interrupt	0x0000 0040
1	8	settable	PVD	PVD through EXTI line detection interrupt	0x0000 0044
2	9	settable	TAMP_STAMP	Tamper and TimeStamp interrupts through the EXTI line	0x0000 0048
3	10	settable	RTC_WKUP	RTC Wakeup interrupt through the EXTI line	0x0000 004C
4	11	settable	FLASH	Flash global interrupt	0x0000 0050
5	12	settable	RCC	RCC global interrupt	0x0000 0054
6	13	settable	EXTI0	EXTI Line0 interrupt	0x0000 0058
7	14	settable	EXTI1	EXTI Line1 interrupt	0x0000 005C
8	15	settable	EXTI2	EXTI Line2 interrupt	0x0000 0060
9	16	settable	EXTI3	EXTI Line3 interrupt	0x0000 0064
10	17	settable	EXTI4	EXTI Line4 interrupt	0x0000 0068
11	18	settable	DMA1_Stream0	DMA1 Stream0 global interrupt	0x0000 006C
12	19	settable	DMA1_Stream1	DMA1 Stream1 global interrupt	0x0000 0070
13	20	settable	DMA1_Stream2	DMA1 Stream2 global interrupt	0x0000 0074
14	21	settable	DMA1_Stream3	DMA1 Stream3 global interrupt	0x0000 0078
15	22	settable	DMA1_Stream4	DMA1 Stream4 global interrupt	0x0000 007C
16	23	settable	DMA1_Stream5	DMA1 Stream5 global interrupt	0x0000 0080
17	24	settable	DMA1_Stream6	DMA1 Stream6 global interrupt	0x0000 0084
18	25	settable	ADC	ADC1, ADC2 and ADC3 global interrupts	0x0000 0088
19	26	settable	CAN1_TX	CAN1 TX interrupts	0x0000 008C
20	27	settable	CAN1_RX0	CAN1 RX0 interrupts	0x0000 0090
21	28	settable	CAN1_RX1	CAN1 RX1 interrupt	0x0000 0094

4. 인터럽트(NVIC, EXTI)

- PORTD 4 스위치 On/Off (인터럽트 사용)
 - PORTD 4의 스위치 신호를 이용하여 인터럽트 발생



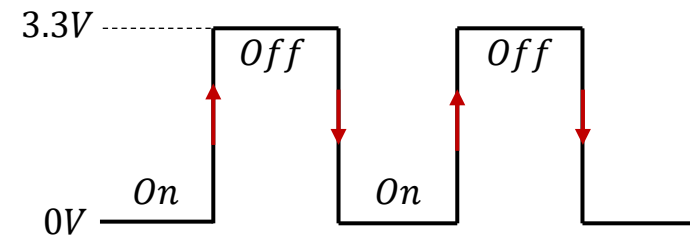
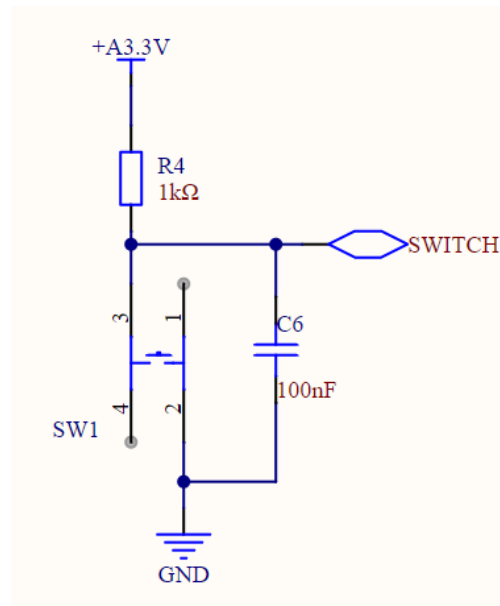
<다중 인터럽트 발생 시 우선 순위에 따른 인터럽트 처리>

Position	Priority	Type of priority	Acronym	Description	Address
-	3	settable	SVCall	System service call via SWI instruction	0x0000 002C
-	4	settable	Debug Monitor	Debug Monitor	0x0000 0030
-	-	-	-	Reserved	0x0000 0034
-	5	settable	PendSV	Pendable request for system service	0x0000 0038
-	6	settable	SysTick	System tick timer	0x0000 003C
0	7	settable	WWDG	Window Watchdog interrupt	0x0000 0040
1	8	settable	PVD	PVD through EXTI line detection interrupt	0x0000 0044
2	9	settable	TAMP_STAMP	Tamper and TimeStamp interrupts through the EXTI line	0x0000 0048
3	10	settable	RTC_WKUP	RTC Wakeup interrupt through the EXTI line	0x0000 004C
4	11	settable	FLASH	Flash global interrupt	0x0000 0050
5	12	settable	RCC	RCC global interrupt	0x0000 0054
6	13	settable	EXTI0	EXTI Line0 interrupt	0x0000 0058
7	14	settable	EXTI1	EXTI Line1 interrupt	0x0000 005C
8	15	settable	EXTI2	EXTI Line2 interrupt	0x0000 0060
9	16	settable	EXTI3	EXTI Line3 interrupt	0x0000 0064
10	17	settable	EXTI4	EXTI Line4 interrupt	0x0000 0068
11	18	settable	DMA1_Stream0	DMA1 Stream0 global interrupt	0x0000 006C
12	19	settable	DMA1_Stream1	DMA1 Stream1 global interrupt	0x0000 0070
13	20	settable	DMA1_Stream2	DMA1 Stream2 global interrupt	0x0000 0074
14	21	settable	DMA1_Stream3	DMA1 Stream3 global interrupt	0x0000 0078
15	22	settable	DMA1_Stream4	DMA1 Stream4 global interrupt	0x0000 007C
16	23	settable	DMA1_Stream5	DMA1 Stream5 global interrupt	0x0000 0080
17	24	settable	DMA1_Stream6	DMA1 Stream6 global interrupt	0x0000 0084
18	25	settable	ADC	ADC1, ADC2 and ADC3 global interrupts	0x0000 0088
19	26	settable	CAN1_TX	CAN1 TX interrupts	0x0000 008C
20	27	settable	CAN1_RX0	CAN1 RX0 interrupts	0x0000 0090
21	28	settable	CAN1_RX1	CAN1 RX1 interrupt	0x0000 0094

4. 인터럽트(NVIC, EXTI)

■ 인터럽트 신호

- STM32F767은 외부 인터럽트 신호로 에지 트리거를 이용 → 상승/하강 엣지
- 상승 에지는 EXTI_RTSR, 하강 에지는 EXTI_FTSR 레지스터로 설정



<스위치 On/Off에 따른 외부 인터럽트 신호>

4. 인터럽트(NVIC, EXTI)

- SYSCFG_EXTICR2
 - EXTI4를 사용하기 위해서 아래의 레지스터 설정을 해줘야 함

7.2.4 SYSCFG external interrupt configuration register 2 (SYSCFG_EXTICR2)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI7[3:0]				EXTI6[3:0]				EXTI5[3:0]				EXTI4[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **EXTIx[3:0]**: EXTI x configuration (x = 4 to 7)

These bits are written by software to select the source input for the EXTIx external interrupt.

0000: PA[x] pin

0001: PB[x] pin

0010: PC[x] pin

0011: PD[x] pin

0100: PE[x] pin

0101: PF[x] pin

0110: PG[x] pin

0111: PH[x] pin

1000: PI[x] pin

1001: PJ[x] pin

1010: PK[x] pin

4. 인터럽트(NVIC, EXTI)

- EXTI_IMR
 - 외부 인터럽트를 사용하기 위해선 아래의 레지스터를 설정해야 함

11.9.1 Interrupt mask register (EXTI_IMR)

Address offset: 0x00

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	IM23	IM22	IM21	IM20	IM19	IM18	IM17	IM16
								rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IM15	IM14	IM13	IM12	IM11	IM10	IM9	IM8	IM7	IM6	IM5	IM4	IM3	IM2	IM1	IM0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:24 Reserved, must be kept at reset value.

Bits 23:0 **IMx**: Interrupt mask on line x

0: Interrupt request from line x is masked

1: Interrupt request from line x is not masked

4. 인터럽트(NVIC, EXTI)

- EXTI_FTSR
 - 스위치 신호의 Falling Edge를 검출하기 위해서 아래의 레지스터를 설정해야 함

11.9.4 Falling trigger selection register (EXTI_FTSR)

Address offset: 0x0C
Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TR23	TR22	TR21	TR20	TR19	TR18	TR17	TR16
								rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TR15	TR14	TR13	TR12	TR11	TR10	TR9	TR8	TR7	TR6	TR5	TR4	TR3	TR2	TR1	TR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:24 Reserved, must be kept at reset value.

Bits 23:0 **TRx**: Falling trigger event configuration bit of line x

- 0: Falling trigger disabled (for Event and Interrupt) for input line
- 1: Falling trigger enabled (for Event and Interrupt) for input line.

4. 인터럽트(NVIC, EXTI)

- EXTI_PR
 - 외부 인터럽트 신호가 발생하면 1로 Bit가 Set 됨
 - 이를 확인하고 1을 레지스터에 작성하면 비트를 클리어 할 수 있음 → 비트를 클리어 해주어 NVIC로 인터럽트 신호를 전달

11.9.6 Pending register (EXTI_PR)

Address offset: 0x14
Reset value: undefined

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PR23	PR22	PR21	PR20	PR19	PR18	PR17	PR16
								rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PR15	PR14	PR13	PR12	PR11	PR10	PR9	PR8	PR7	PR6	PR5	PR4	PR3	PR2	PR1	PR0
rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1

Bits 31:24 Reserved, must be kept at reset value.

Bits 23:0 **PRx**: Pending bit

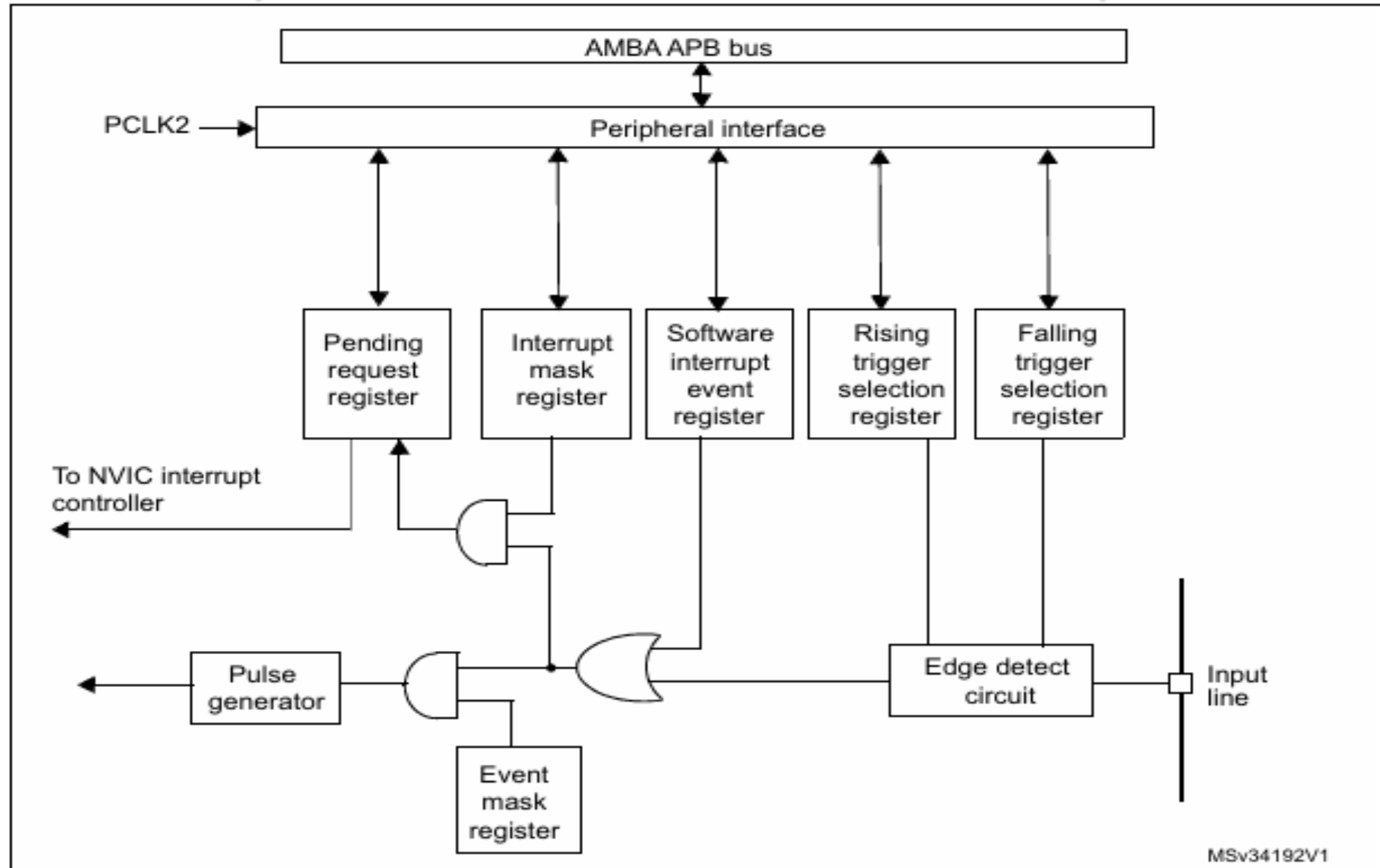
0: No trigger request occurred

1: selected trigger request occurred

This bit is set when the selected edge event arrives on the external interrupt line.

This bit is cleared by programming it to '1'.

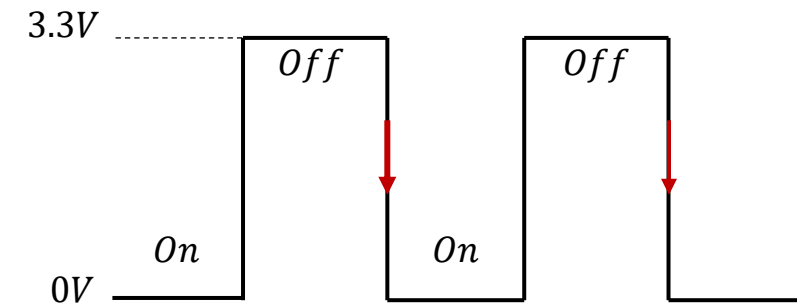
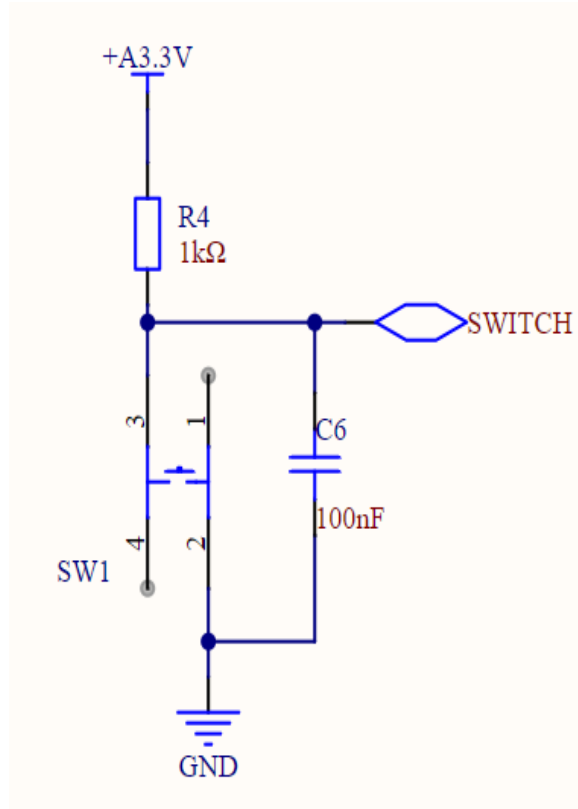
4. 인터럽트(NVIC, EXTI)



4. 인터럽트(NVIC, EXTI)

▪ LED + Switch(인터럽트) 예제

- 스위치의 Falling edge를 검출하여 LED On/Off 하기



4. 인터럽트(NVIC, EXTI)

```
47 /* ----- GPIO & EXTI 초기화 ----- */
48 void GPIO_EXTI_Init(void)
49 {
50     /* 1) GPIOD 클럭 ON (bit3) */
51     RCC->AHB1ENR |= 0x00000008;
52
53     /* --- PD3 : LED (Active-Low) ----- */
54     GPIOD->MODER  |= 0x00000040; // 01 = Output
55     GPIOD->OSPEEDR |= 0x00000040; // Medium speed
56     GPIOD->ODR     |= 0x00000008; // LED OFF(High)로 초기화
57
58     /* --- PD4 : 스위치 입력 / EXTI4 ----- */
59     GPIOD->MODER  &= ~0x00000300; // bits9:8 = 00 (Input)
60
61     /* 2) SYSCFG 클럭 ON (EXTI 라우팅용) */
62     RCC->APB2ENR  |= 0x00004000; // SYSCFGEN
63
64     /* 3) EXTI4 소스 = Port D */
65     SYSCFG->EXTICR[1] |= 0x0003; // 0x3 = Port D
66
67     /* 4) EXTI4 Falling-edge 취소/허용 설정 */
68     EXTI->IMR  |= 0x00000010; // Interrupt request unmask
69     EXTI->FTSR |= 0x00000010; // Falling edge on
70     EXTI->PR   = 0x00000010; // Pending bit clear
71
72     /* 5) NVIC: EXTI4 IRQ Enable (우선순위는 기본 0) */
73     NVIC_EnableIRQ(EXTI4_IRQn);
74 }
```

```
/* ----- EXTI4 인터럽트 서비스 루틴 ----- */
void EXTI4_IRQHandler(void)
{
    if (EXTI->PR & 0x00000010) // 팬딩 확인
    {
        EXTI->PR = 0x00000010; // 팬딩 클리어

        GPIOD->ODR &= ~0x00000008; // LED ON (PD3 = 0)
        delay_1s(); // 1 초간 점등
        GPIOD->ODR |= 0x00000008; // LED OFF (PD3 = 1)
    }
}

/* ----- main ----- */
int main(void)
{
    Initialize_MCU(); // 216 MHz 클럭 설정(앞서 만든 함수)
    GPIO_EXTI_Init(); // GPIO 및 EXTI 설정

    while (1)
    {
        /* 메인 루프는 대기만; 모든 일은 인터럽트에서 처리 */
    }
}
```

5. ADC (Analog to Digital Converter)

■ ADC를 사용하는 이유

- 전압, 전류, 온도와 같은 물리적인 신호는 실제로는 연속 신호이다. → 아날로그
- 이를 측정하기 위해서는 아날로그 신호를 디지털 신호로 변환해야 하고, 이를 ADC 모듈을 이용함
- 또한 센서의 대부분은 물리적인 신호를 전압으로 변환하여 측정함

전압, 전류, 온도, 빛, 토크

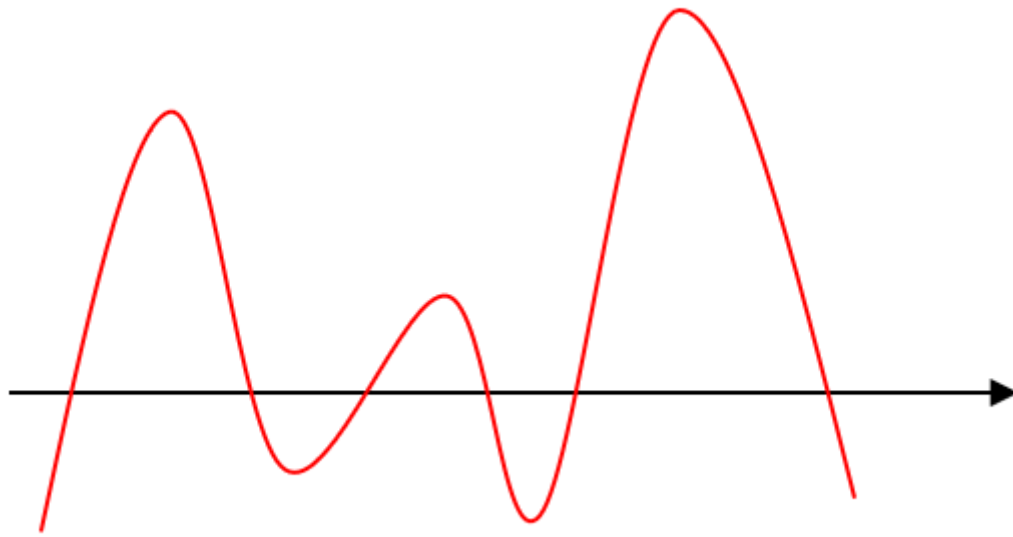
❖ MCU에서 받아들일 수 있는
수준의 전압 신호로 변환됨



5. ADC (Analog to Digital Converter)

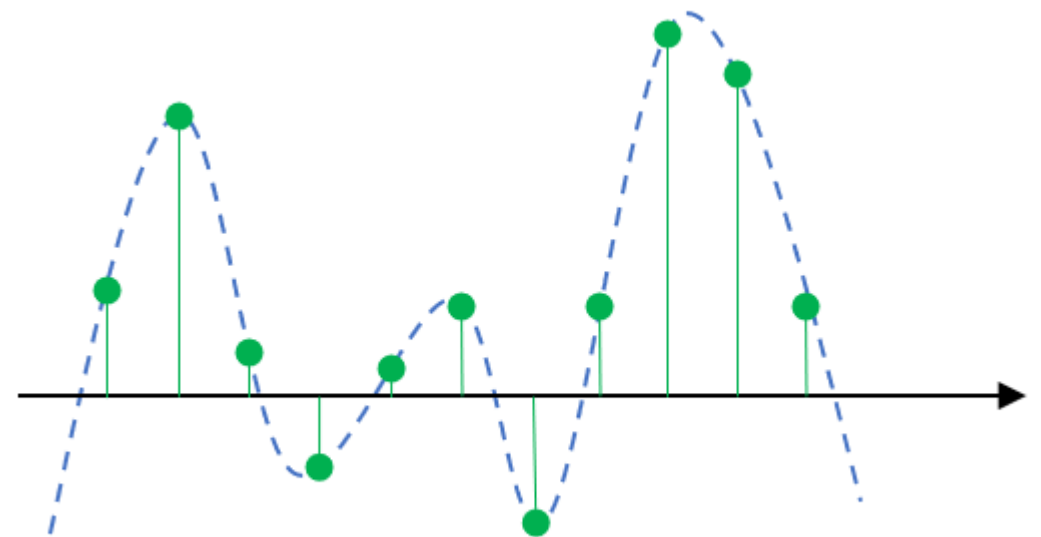
■ STM32F767 ADC

- 연속 신호(아날로그 신호)를 이산 신호로 변환하기 위해서는 샘플링, 양자화, 부호화 3단계를 거친다.
- 이러한 단계는 MCU 내부의 ADC 모듈이 알아서 처리해주지만, 기본적인 원리는 알고 있어야 한다.



연속 신호

VS

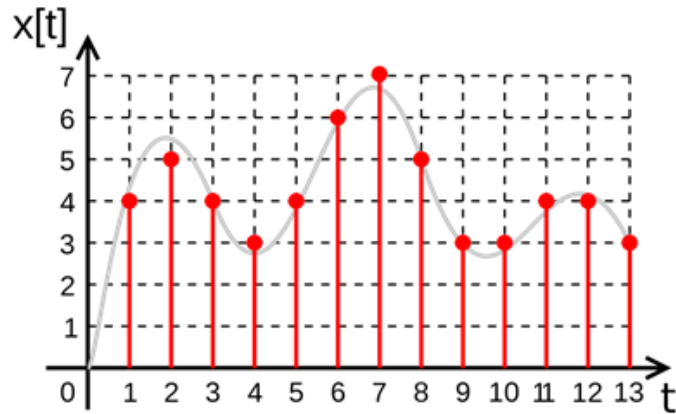


이산 신호

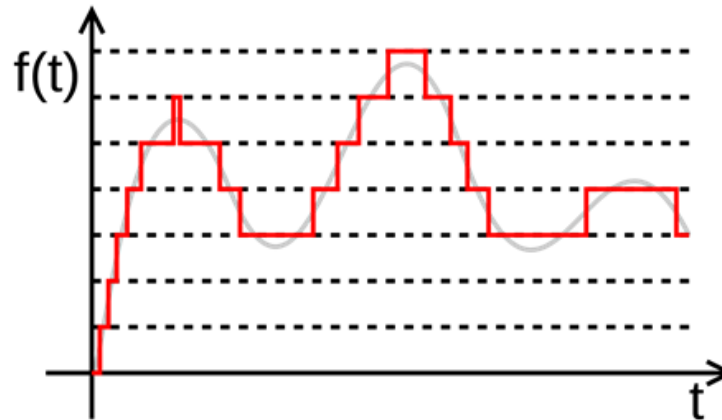
5. ADC (Analog to Digital Converter)

■ STM32F767 ADC 변환 단계

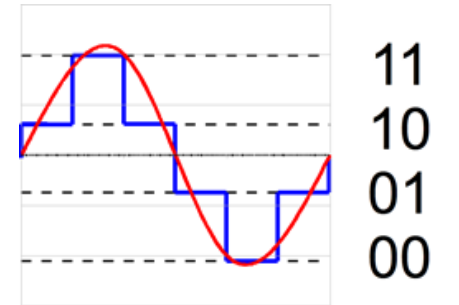
- ADC의 샘플링은 Clock에 동기화되어 수행함
- 나이퀴스트 이론에 따라 샘플링 주파수는 오리지널 신호 주파수보다 2배 이상이어야 함
- 아날로그를 디지털로 변환하는 과정 중, 아날로그 데이터를 수치화 할 수 있는 값으로 근사하여 점으로 표현



➤ 샘플링



➤ 양자화



➤ 부호화

5. ADC (Analog to Digital Converter)

■ STM32F767 ADC 변환 단계

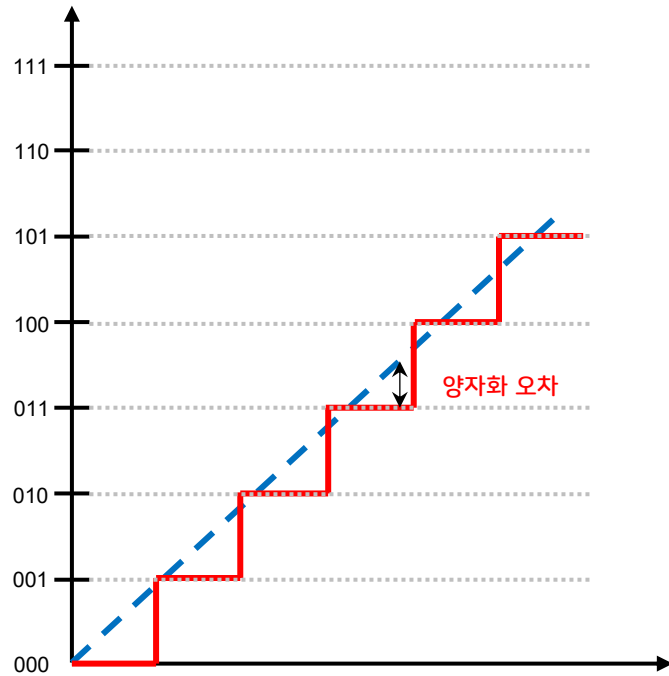
- ADC의 비트가 높을수록 분해능이 높아짐 → 즉 양자화 오차를 줄일 수 있고 정밀한 데이터 취득이 가능함
- STM32f767의 ADC는 12,10,8,6 비트로 설정하여 사용 할 수 있음
- 본 강의에서는 ADC를 12비트로 설정하여 사용

N-Bit	Steps	Step Size
8-Bit	256	$3.3V/256 = 12.89mV$
10-Bit	1024	$3.3V/1024 = 3.22mV$
12-Bit	4096	$3.3V/4096 = 0.8mV$
16-bit	65,533	$3.3V/65536 = 0.05mV$

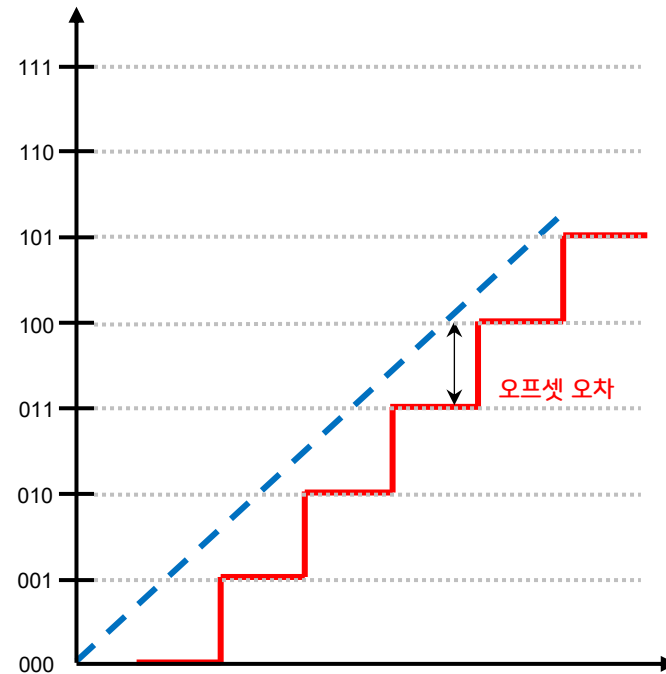
5. ADC (Analog to Digital Converter)

■ STM32F767 ADC 오차

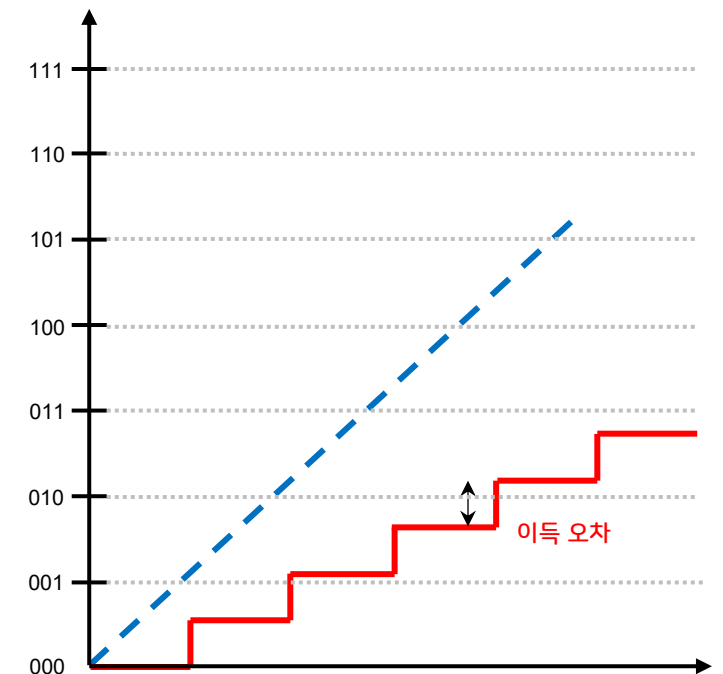
- ADC의 오차는 아래와 같이 3가지의 오차가 존재한다.



(a) 양자화 오차



(b) 오프셋 오차



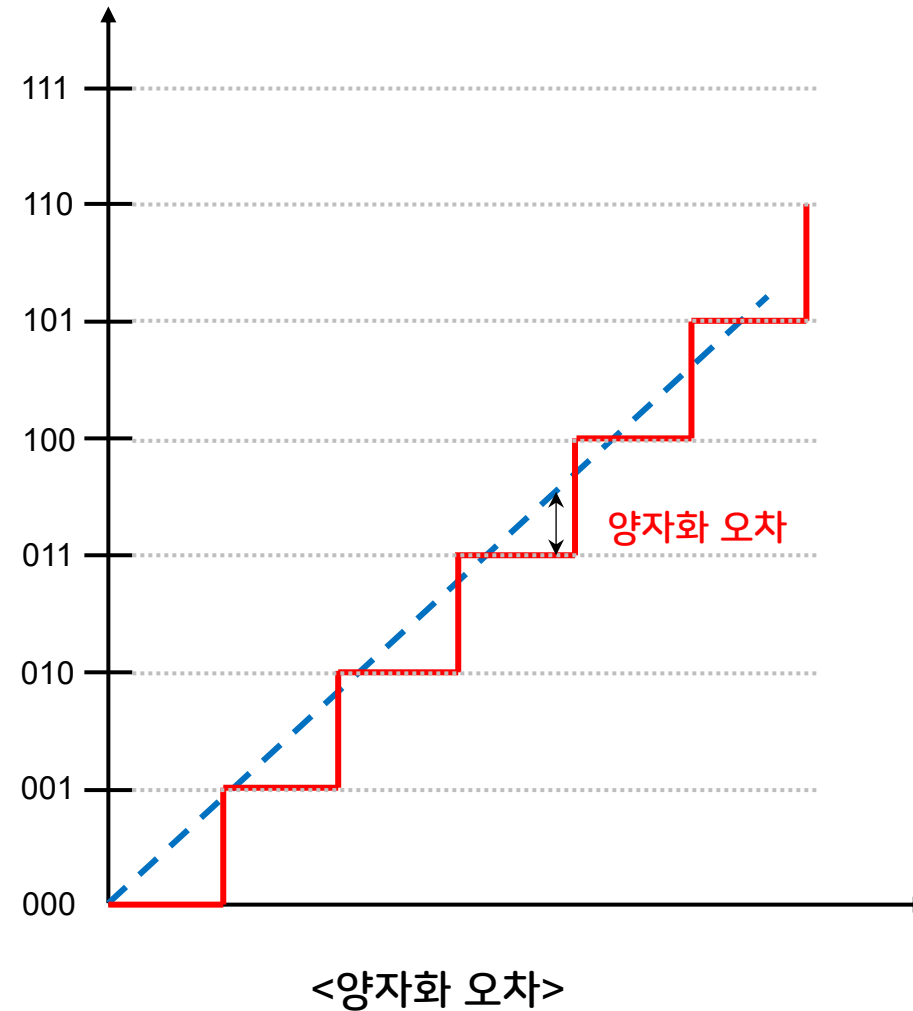
(c) 이득 오차

<STM32f767 ADC 오차>

5. ADC (Analog to Digital Converter)

■ STM32F767 ADC 양자화 오차

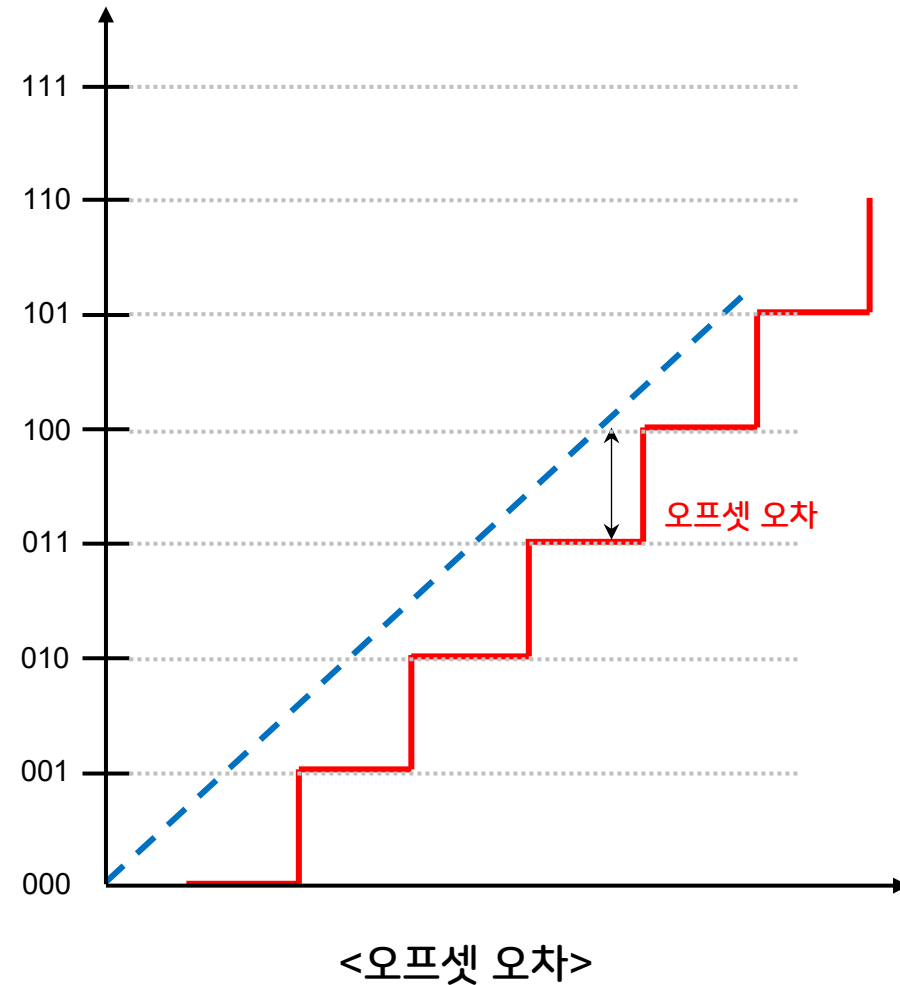
- 양자화 오차는 필연적으로 발생할 수 밖에 없음
- ADC의 비트 수가 높을 수록 양자화 오차가 적음



5. ADC (Analog to Digital Converter)

■ STM32F767 ADC 오프셋 오차

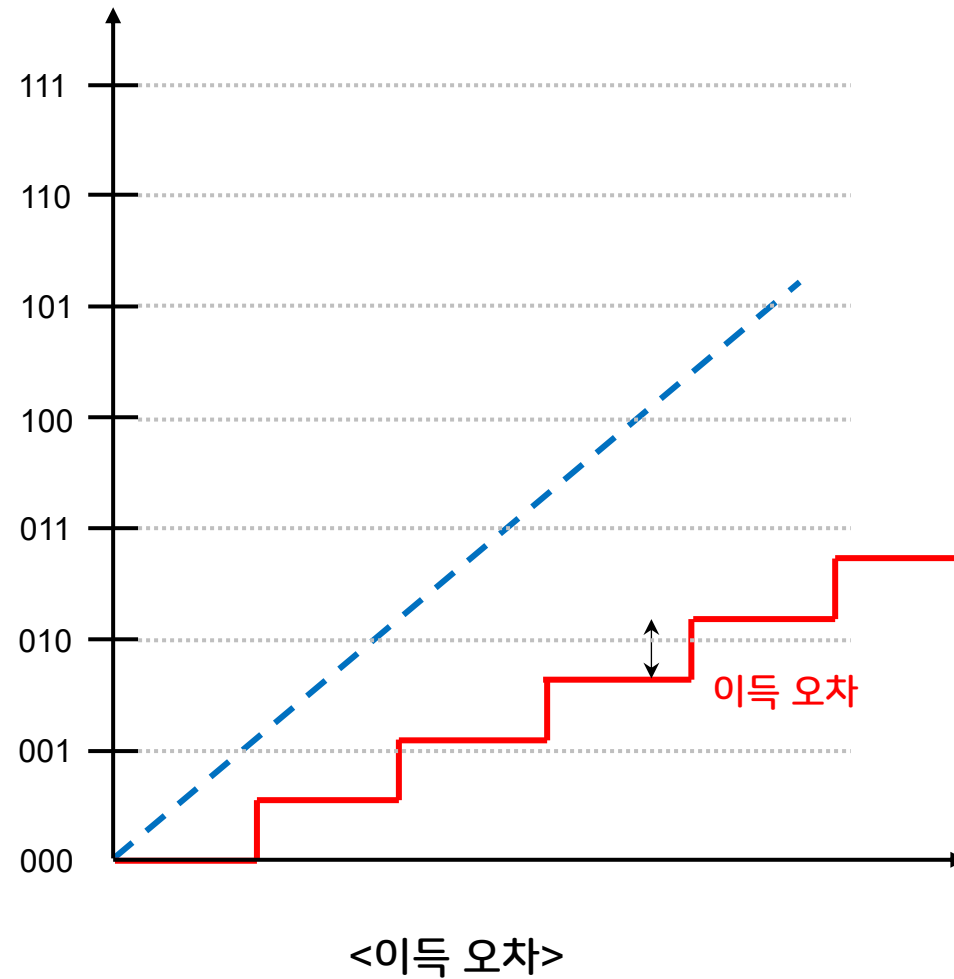
- 내부 회로&외부 회로의 불균형으로 발생됨.
- 일정한 상수 만큼 생기는 오차이며, 소프트웨어적으로 보정 가능 → 일정한 값을 더해주거나 빼서 보상



5. ADC (Analog to Digital Converter)

- STM32F767 ADC 이득 오차

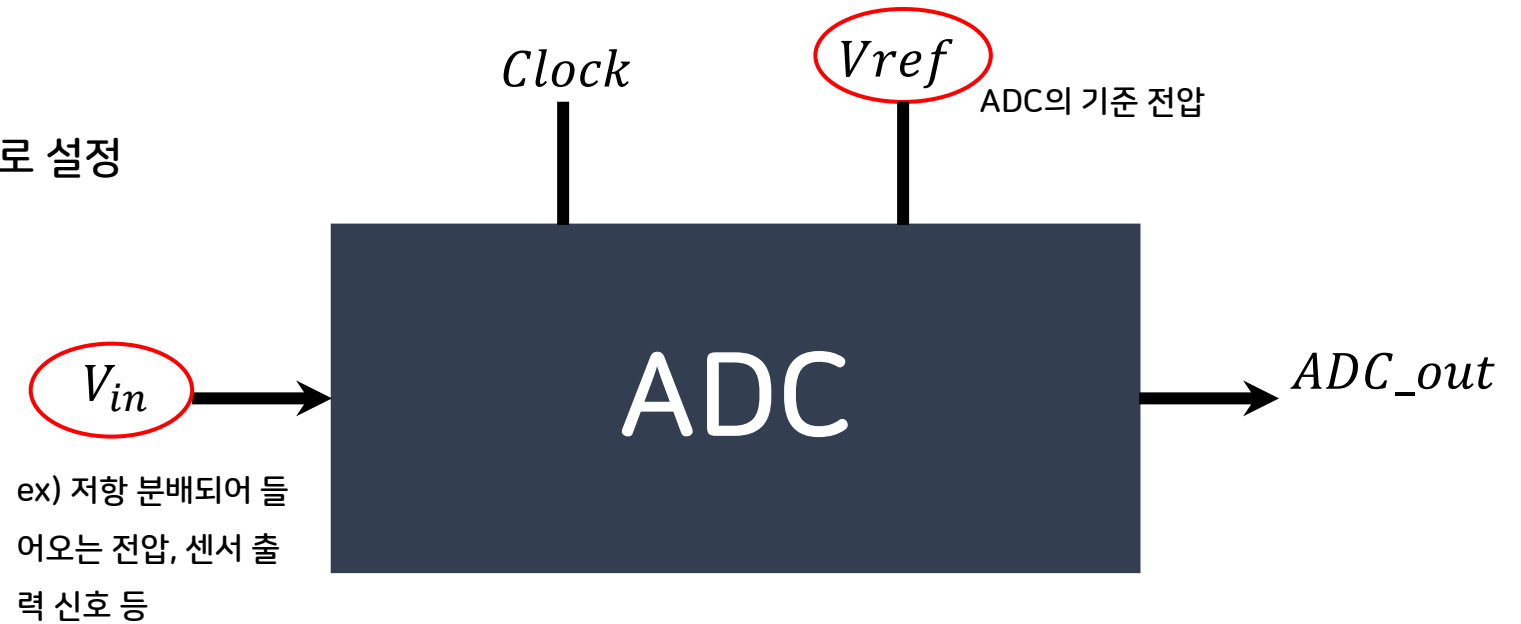
- 일정한 값을 곱한 것과 같은 기울기 오차 발생
- 소프트웨어적으로 보상 가능 → 일정한 값을 곱해주어 보상



5. ADC (Analog to Digital Converter)

■ STM32F767 ADC 분해능

- 분해능 : ADC 출력의 이진 비트 수
- ex) 12비트 : 0~4095
- ADC에 입력되는 전압의 크기는 항상 기준전압 이내로 설정
- 기준 전압은 노이즈가 적어야 함



$$ADC_out = (2^N - 1) \times \frac{V_{in}}{V_{ref}} \quad N: \text{분해능}$$

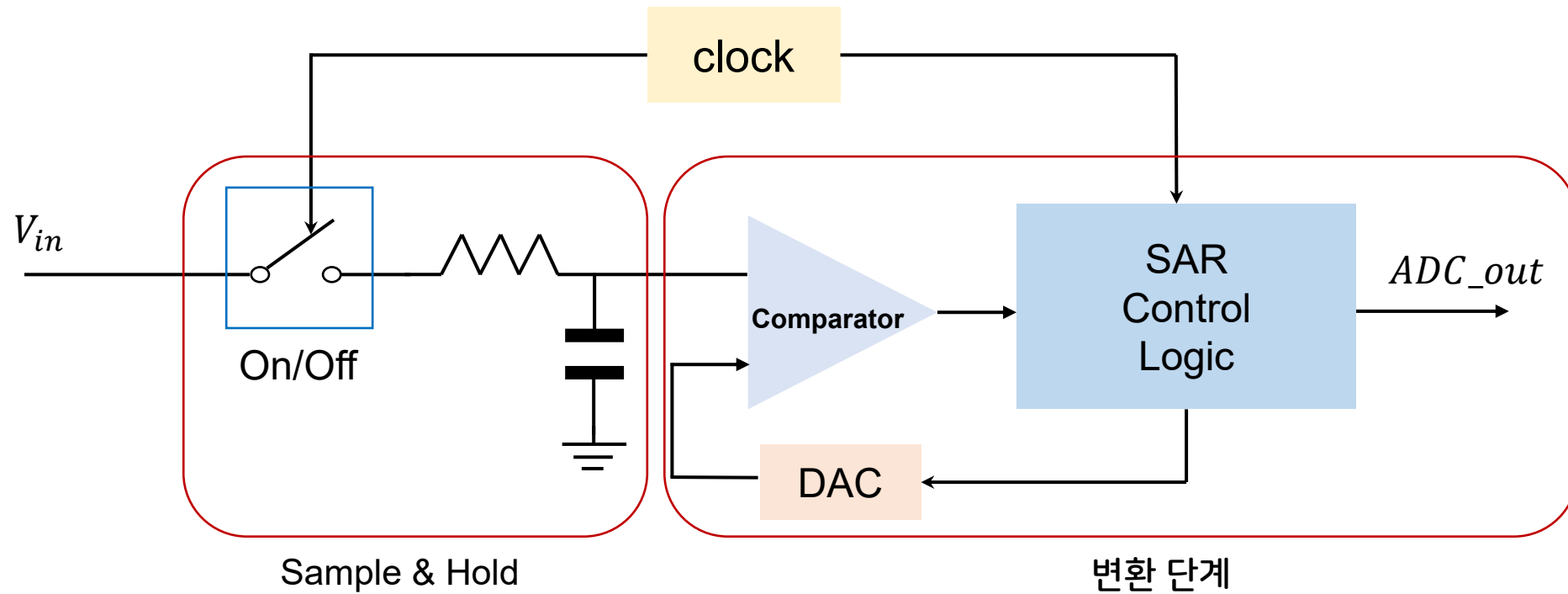
$$12\text{-bit ADC}, V_{in} = 1V, V_{ref} = 3.3V$$

$$ADC_out = (4095 - 1) \times \frac{1V}{3.3V} \cong 1240$$

✓ 역산하여 물리적인 값으로 환산 가능

5. ADC (Analog to Digital Converter)

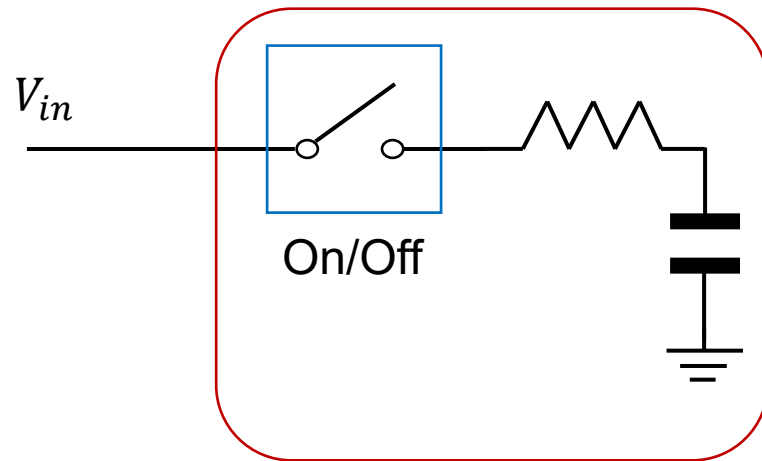
- SAR (Successive Approximation)
 - SAR ADC는 크게 S&H와 변환 단계로 나뉘져 있음
 - Clock 신호에 동기화되어 스위치를 On/Off 함
 - 변환 단계는 Off 단계에서 진행됨



5. ADC (Analog to Digital Converter)

■ SAR (Successive Approximation)

- Sampling time을 설정할 수 있음.



Sample & Hold

15.5 Channel-wise programmable sampling time

The ADC samples the input voltage for a number of ADCCLK cycles that can be modified using the SMP[2:0] bits in the ADC_SMPR1 and ADC_SMPR2 registers. Each channel can be sampled with a different sampling time.

The total conversion time is calculated as follows:

$$T_{\text{conv}} = \text{Sampling time} + 12 \text{ cycles}$$

Example:

With ADCCLK = 30 MHz and sampling time = 3 cycles:

$$T_{\text{conv}} = 3 + 12 = 15 \text{ cycles} = 0.5 \mu\text{s with APB2 at 60 MHz}$$

15.13.4 ADC sample time register 1 (ADC_SMPR1)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	SMP18[2:0]			SMP17[2:0]			SMP16[2:0]			SMP15[2:1]	
					rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SMP15_0	SMP14[2:0]			SMP13[2:0]			SMP12[2:0]			SMP11[2:0]			SMP10[2:0]		
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31: 27 Reserved, must be kept at reset value.

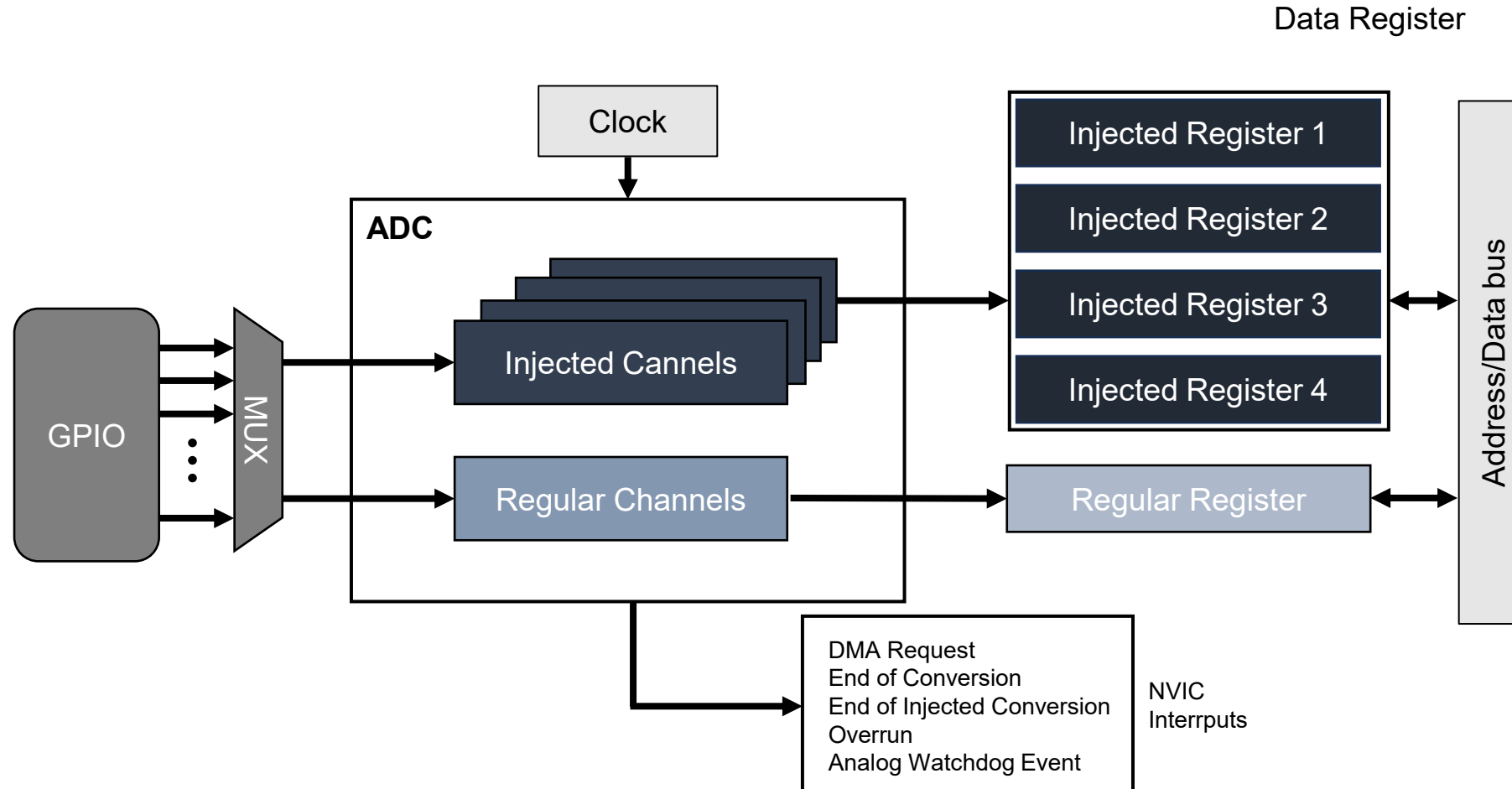
Bits 26:0 **SMPx[2:0]**: Channel x sampling time selection

These bits are written by software to select the sampling time individually for each channel. During sampling cycles, the channel selection bits must remain unchanged.

Note: 000: 3 cycles
001: 15 cycles
010: 28 cycles
011: 56 cycles
100: 84 cycles
101: 112 cycles
110: 144 cycles
111: 480 cycles

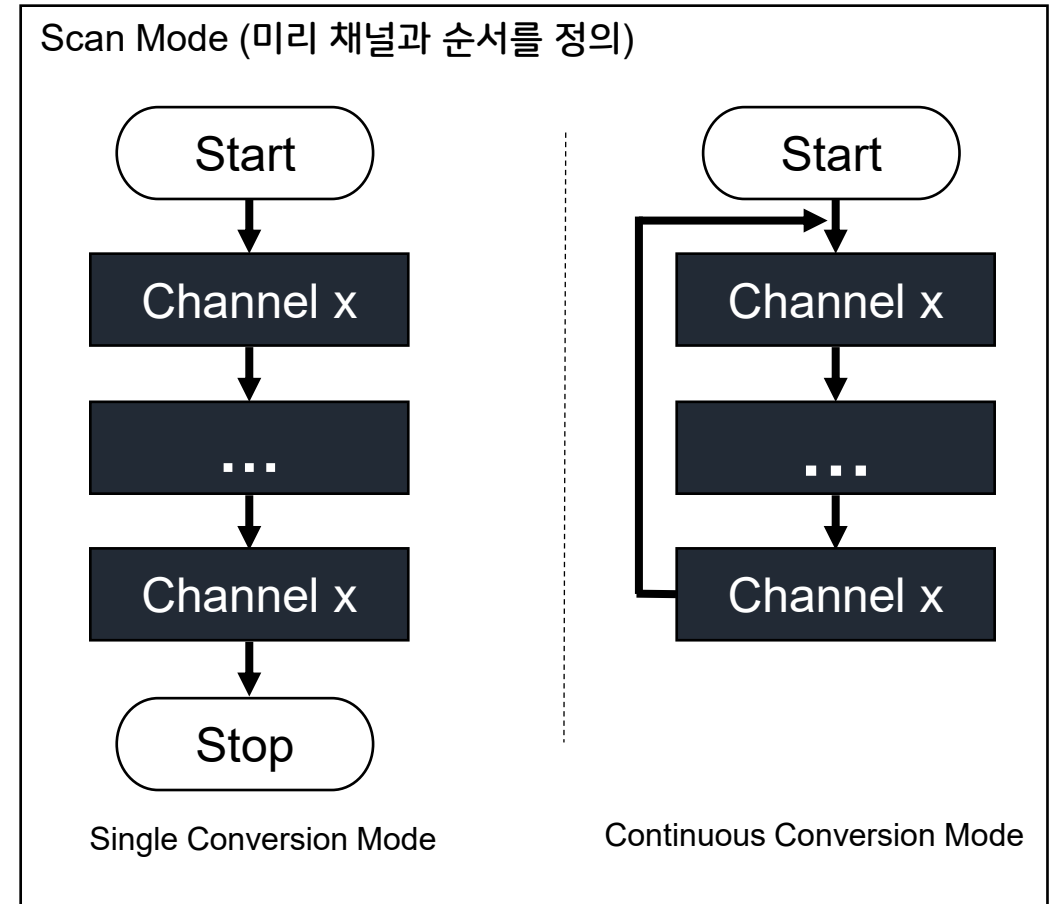
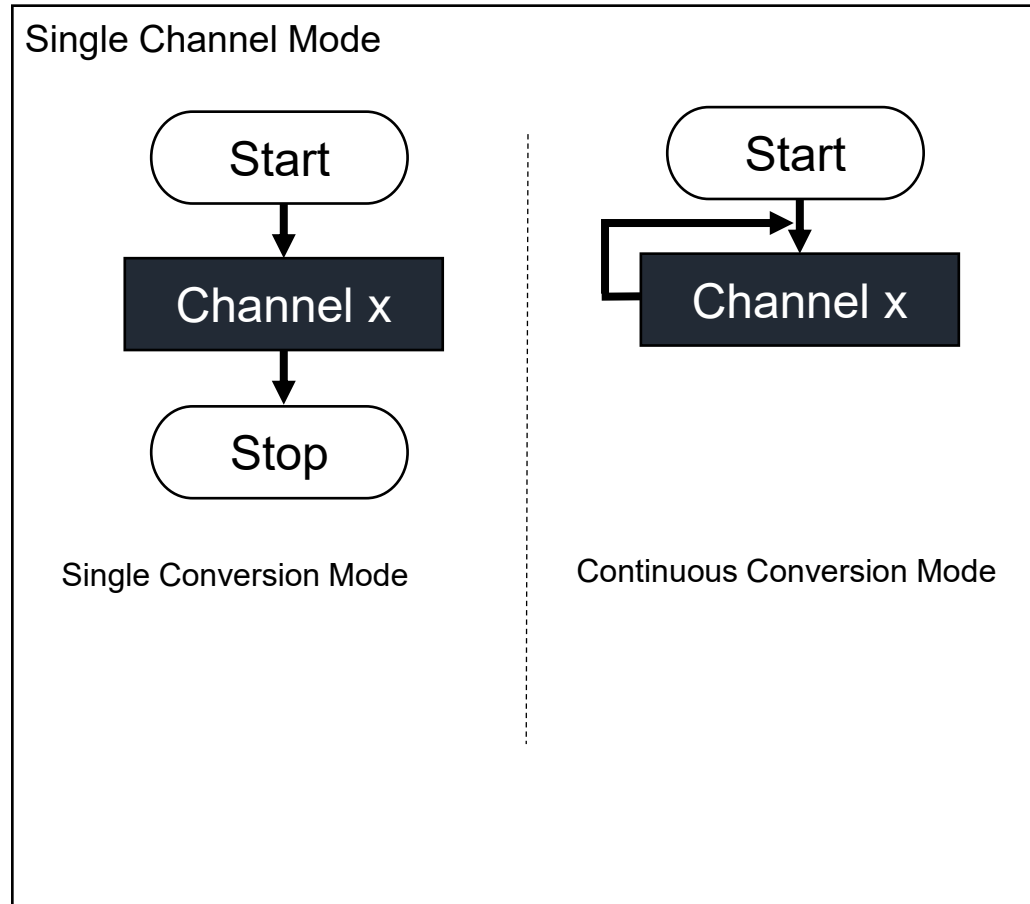
5. ADC (Analog to Digital Converter)

- Regular, Injected 그룹
 - ADC의 내부 채널들을 Regular, Injected로 그룹화 할 수 있음
 - Injected 채널은 각각의 데이터 레지스터가 존재함.



5. ADC (Analog to Digital Converter)

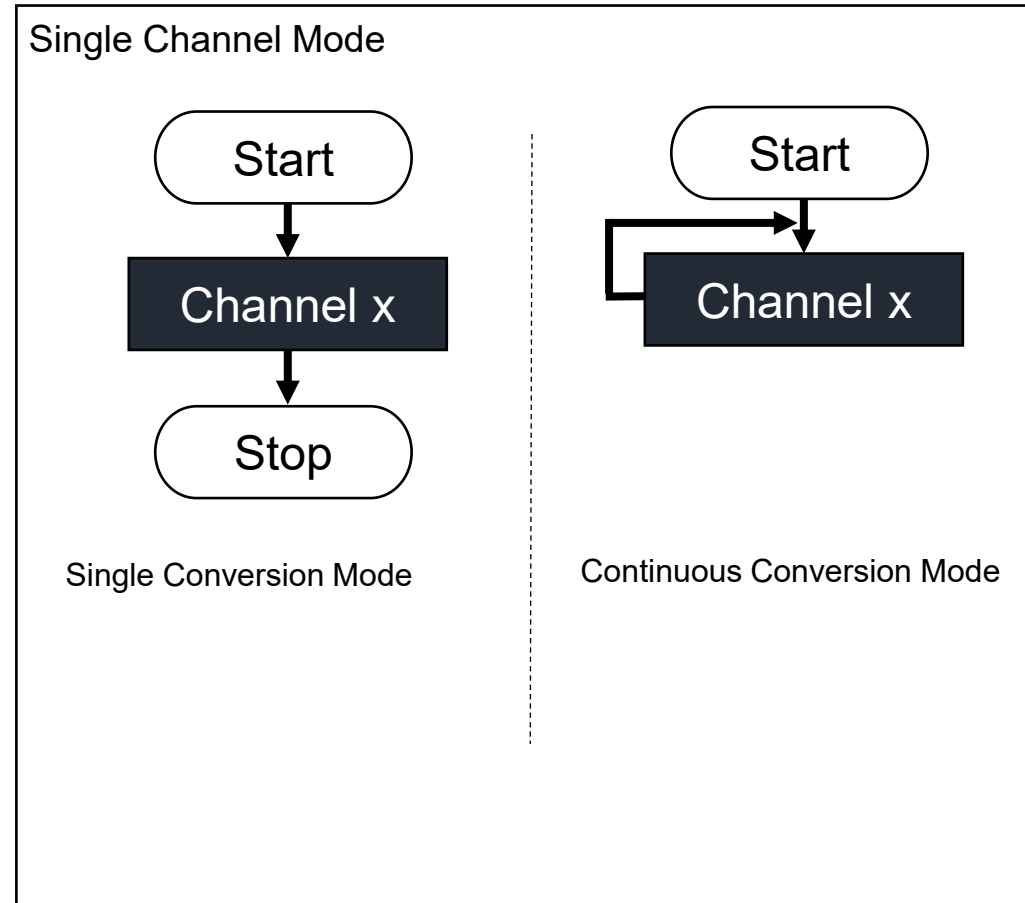
■ ADC Conversion Modes



5. ADC (Analog to Digital Converter)

■ ADC Conversion Modes

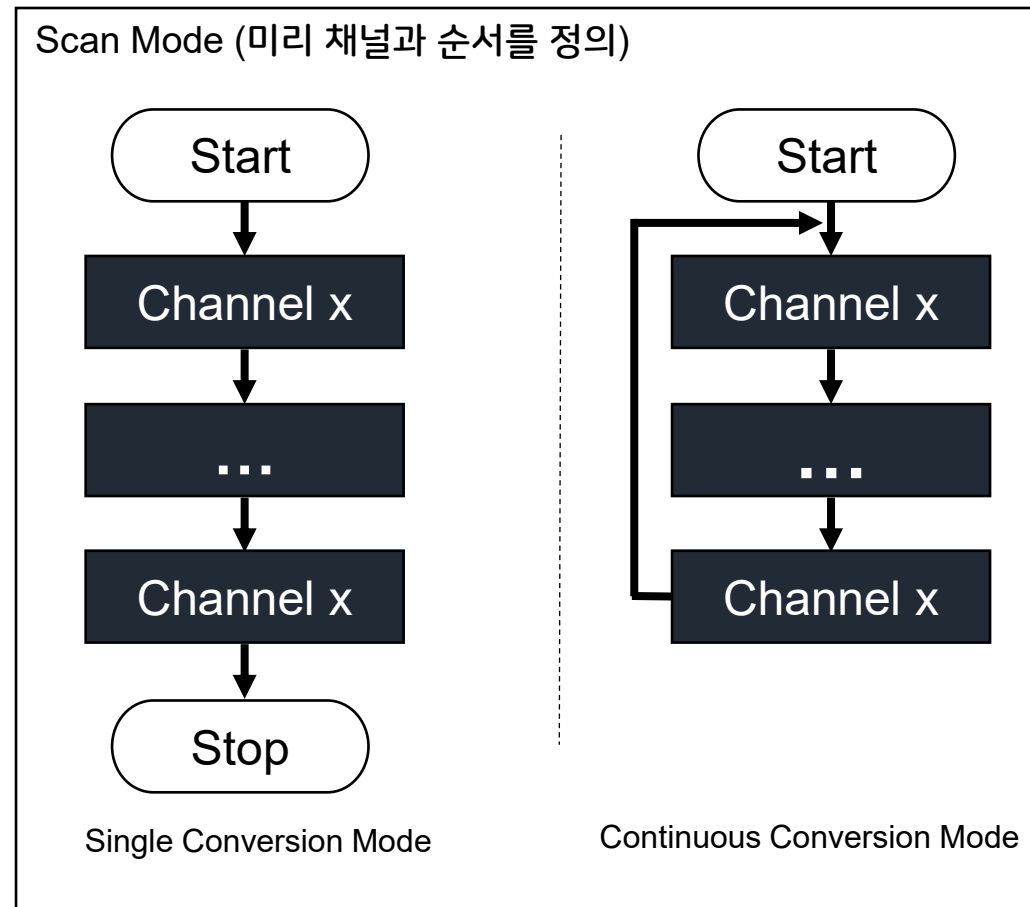
- Single Conversion Mode : 트리거가 발생하였을 때마다 1번의 ADC 변환
- Continuous Conversion Mode : 한번의 트리거에 의하여 계속 반복하여 ADC 변환 수행



5. ADC (Analog to Digital Converter)

■ ADC Conversion Modes

- 다수의 채널(16채널)을 ADC를 한번만 수행하도록 설정하거나, 계속 반복하여 수행하도록 설정할 수 있음
- 레귤러 채널의 시퀀스가 동작 중에 인젝티드 트리거가 발생하면 → 레귤러 채널의 시퀀스를 중단하고 인젝티드 채널의 변환을 먼저 처리하고 레귤러 채널 변환
- 인젝티드 채널의 시퀀스가 동작 중에 레귤러 채널 트리거가 발생하면 → 인젝티드 채널의 시퀀스가 끝난 후에 레귤러 채널 변환 동작



✓ 인젝티드 그룹의 우선순위가 높음

5. ADC (Analog to Digital Converter)

- ADC 변환 결과 저장
 - ADC 데이터는 ADCx_DR(레귤러 그룹), ADC_xJDR1~4(인젝티드 그룹) 레지스터에 저장됨
 - 인젝티드 그룹은 ADCx_JOFR1~4 레지스터에 미리 저장되어 있던 값(오프셋 설정 시)을 뺀 후에 데이터 레지스터에 저장하여 ADC_xJDR1~4이 음수가 될 수 있음
 - SEXT(sign-extended) : ADC 변환 결과가 양수 = 0 , 음수 = 1

Figure 77. Right alignment of 12-bit data

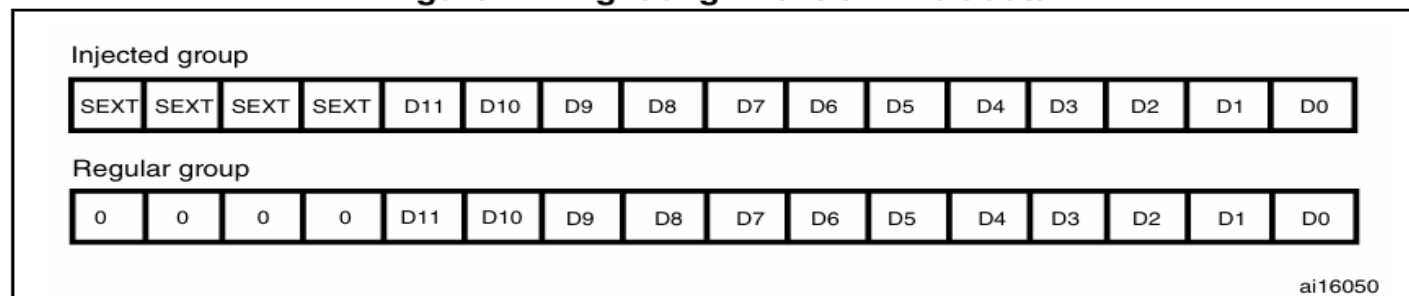
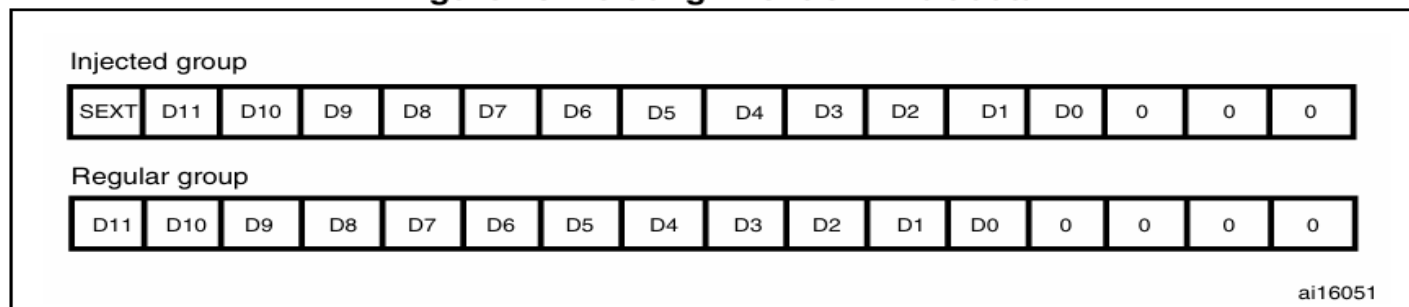


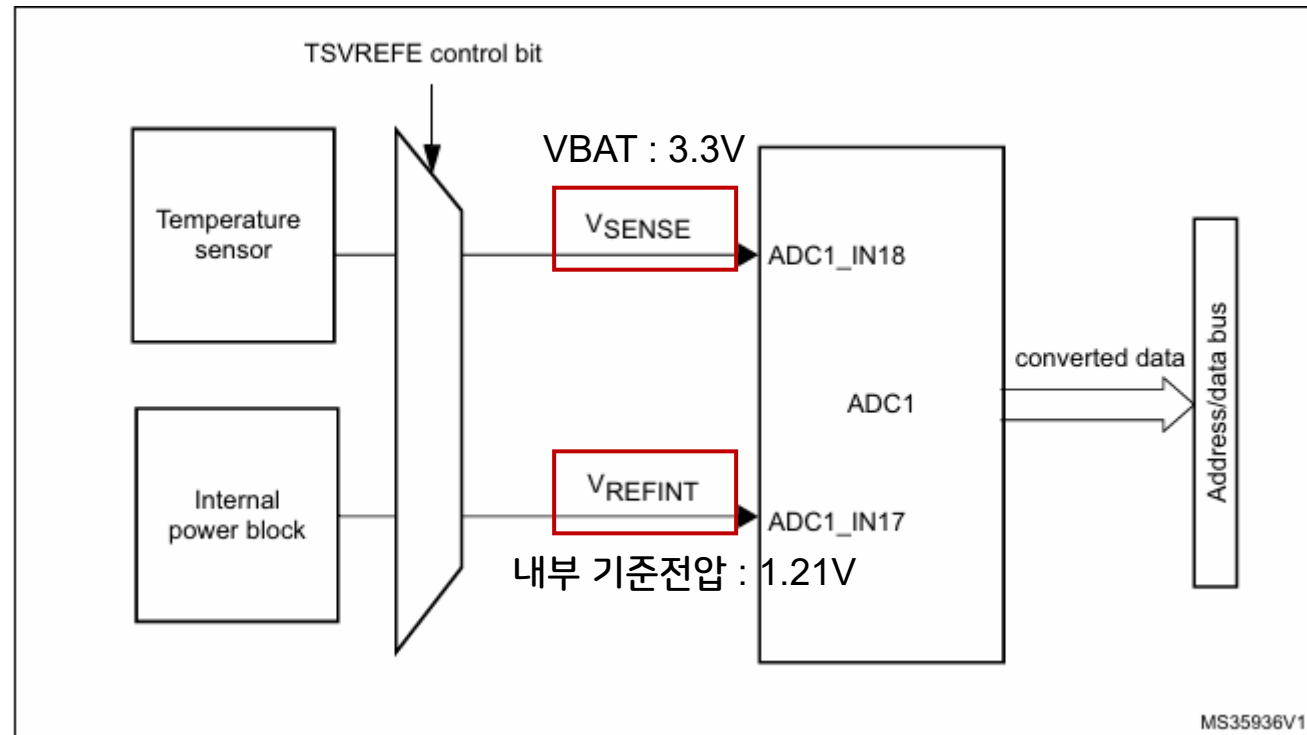
Figure 78. Left alignment of 12-bit data



<ADC 변환 결과 레지스터>

5. ADC (Analog to Digital Converter)

- ADC 컨버터 내부 채널 사용
 - STM32F767의 ADC 컨버터는 16개의 외부입력 채널과 3개의 내부입력 채널이 존재함
 - 실제 사용되는 채널은 아래 그림의 ADC1_IN17, ADC1_IN18을 사용
 - 내부 채널 사용시 샘플링 시간을 10us 이상으로 크게 설정



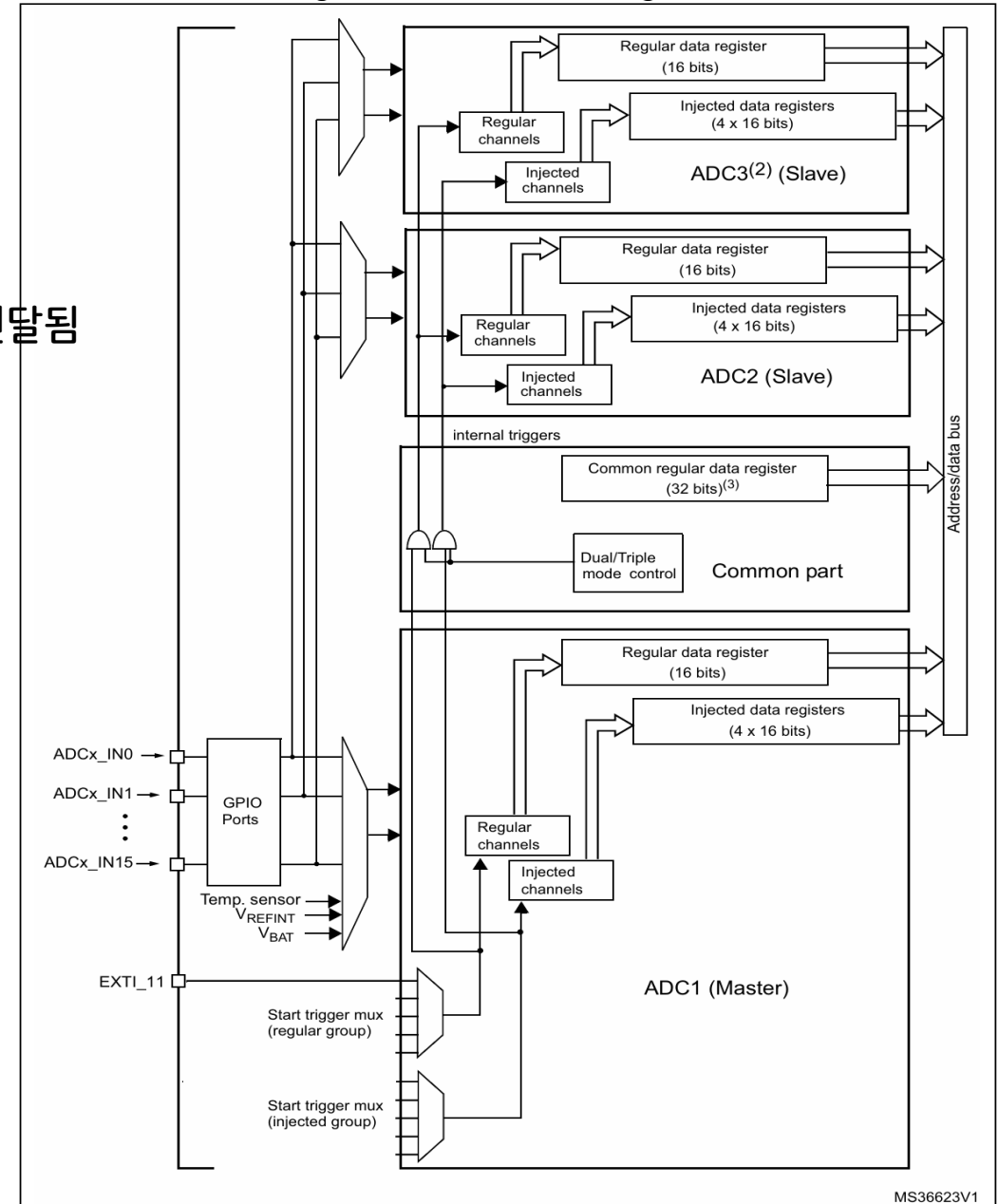
<ADC 내부 채널>

5. ADC (Analog to Digital Converter)

■ 멀티 ADC 모드

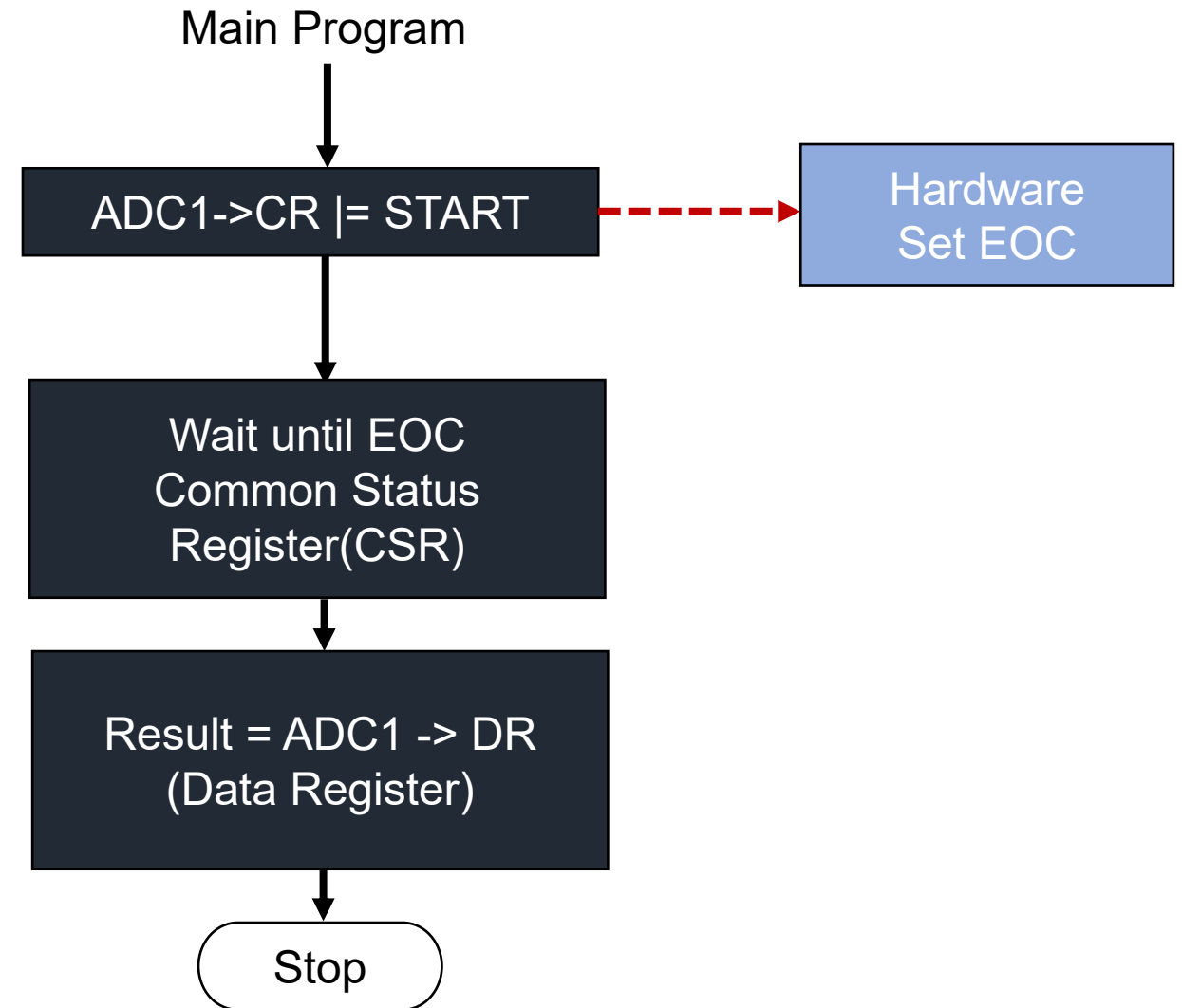
- STM32F767은 3개의 ADC(ADC1~ADC3)을 가지고 있음
- 동시에 2~3개의 ADC가 변환을 수행하도록 할 수 있음 → 멀티 ADC 모드
- ADC1이 마스터로 동작하고 ADC2, ADC3는 슬레이브로 사용
- ADC 변환 트리거 신호는 마스터인 ADC1에서 슬레이브인 ADC2, ADC3로 전달됨

Figure 80. Multi ADC block diagram⁽¹⁾



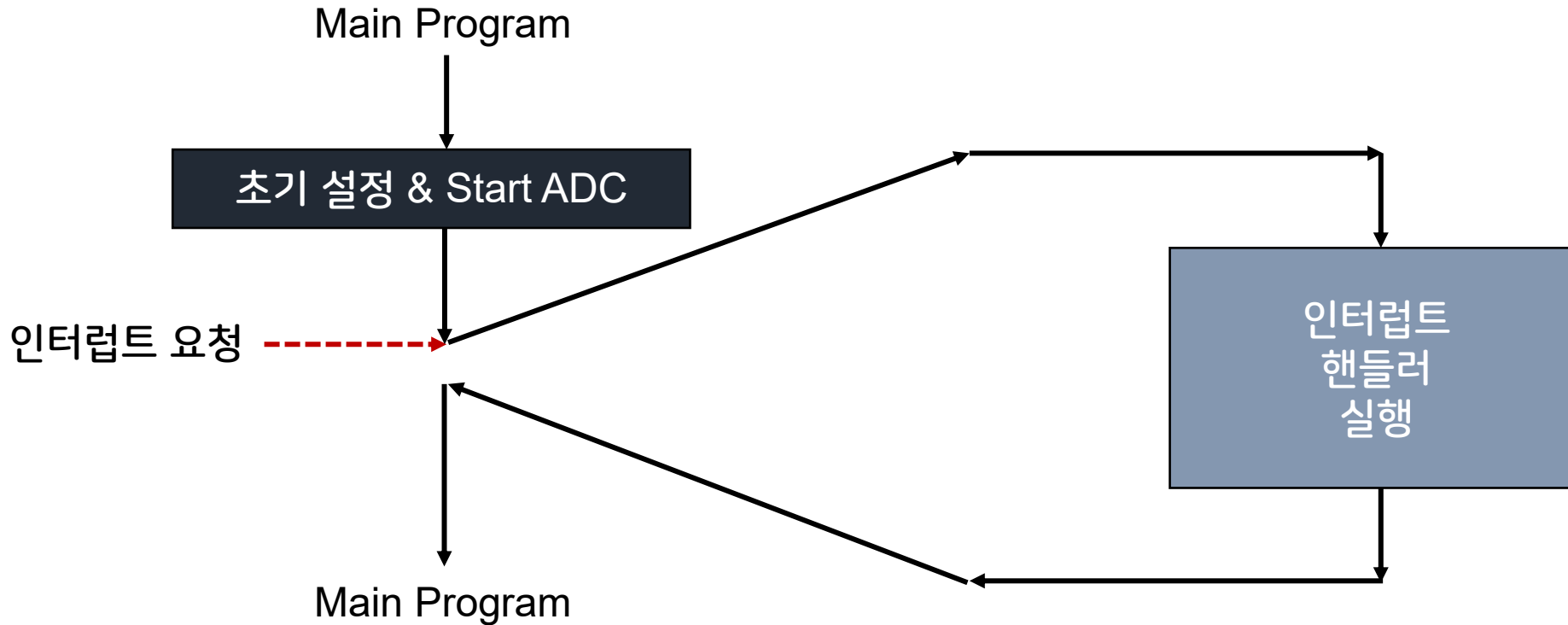
5. ADC (Analog to Digital Converter)

- Polling 방식의 ADC 데이터 획득
 - 인터럽트 방식 대비 구현이 간단
 - 항상 EOC(End Of Conversion)을 기다려야 하므로 손실 발생



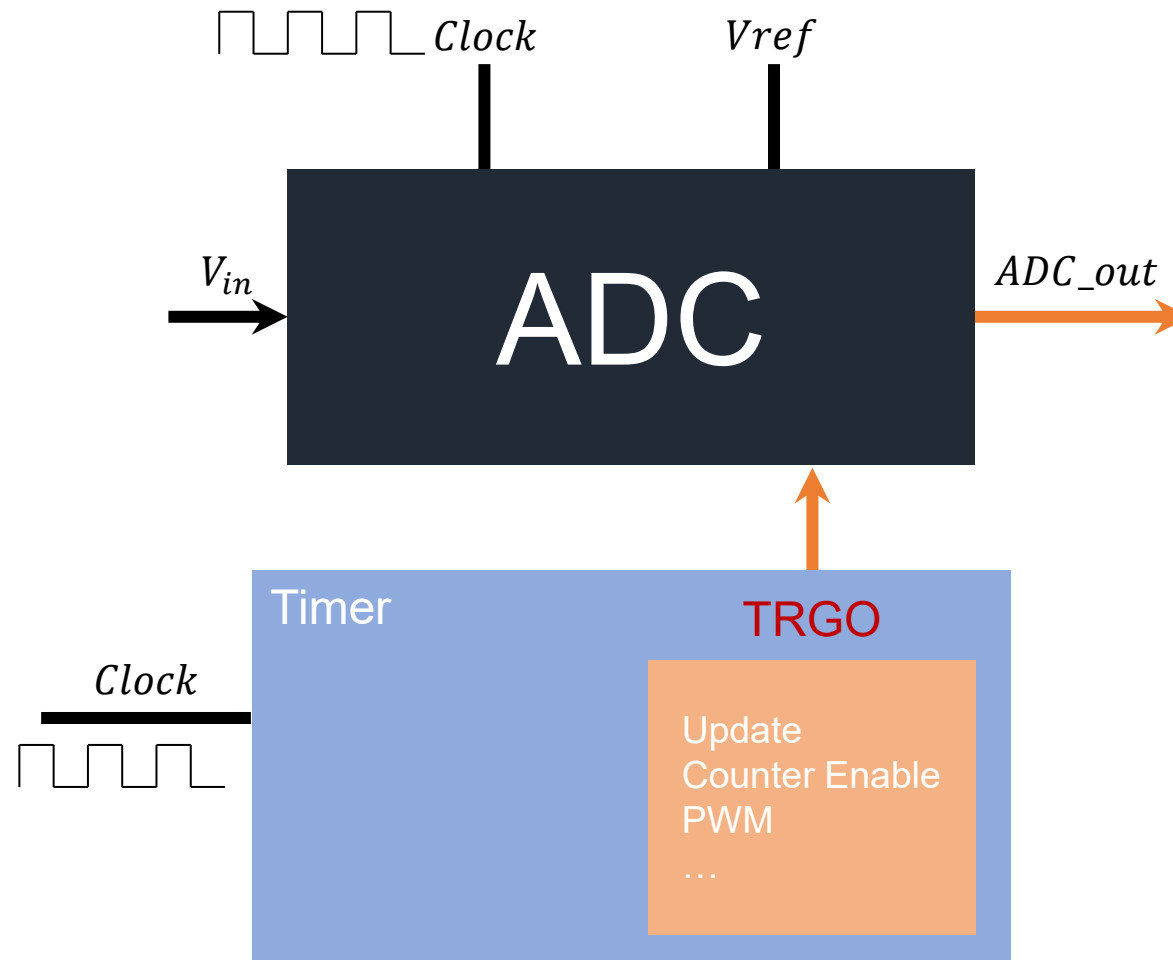
5. ADC (Analog to Digital Converter)

- 인터럽트 방식의 ADC 데이터 획득
 - 폴링 방식 대비 CPU의 load를 줄일 수 있음
 - 상대적으로 구현이 복잡함



5. ADC (Analog to Digital Converter)

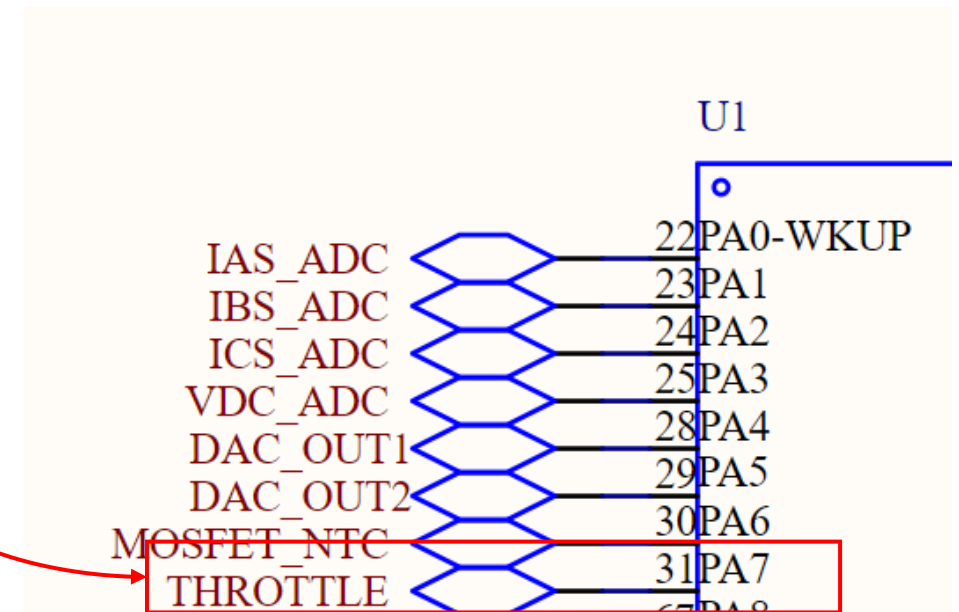
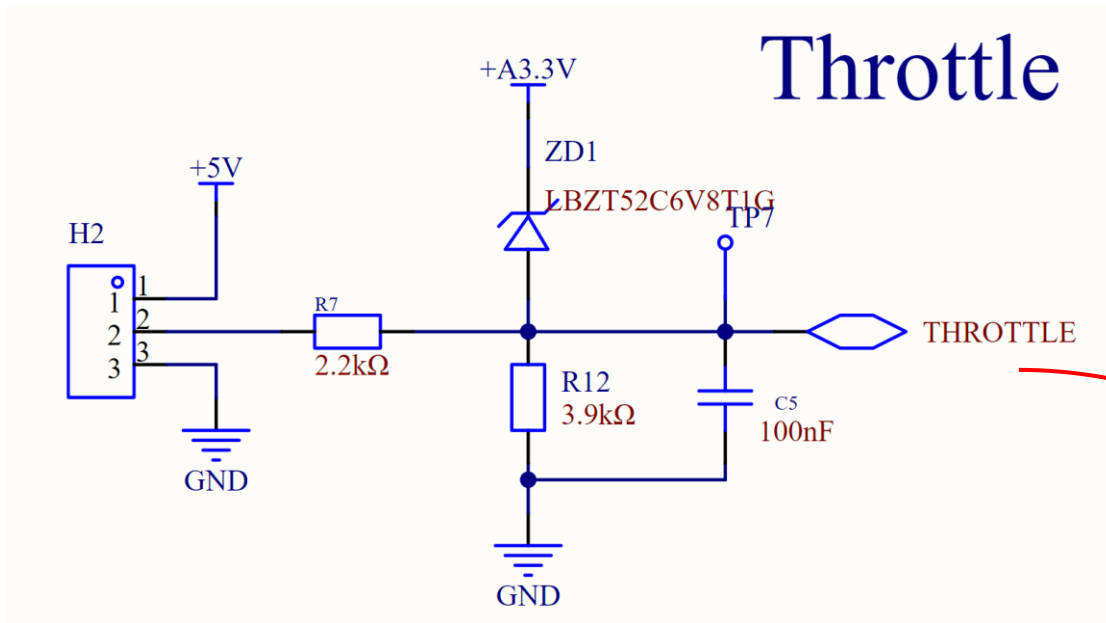
- Timer 트리거 방식
 - TRGO 신호를 이용하여 원하는 주기마다 ADC 변환 수행



5. ADC (Analog to Digital Converter)

■ STM32F767 ADC 사용

- PA7을 ADC 설정하여 쓰로틀을 돌려서 전압이 변하는 지 확인 해보기.
- Single 채널 모드, Single 컨버전 모드로 동작



5. ADC (Analog to Digital Converter)

■ STM32F767 레지스터 설정

5.3.14 RCC APB2 peripheral clock enable register (RCC_APB2ENR)

Address offset: 0x44

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	LTDC EN	Res.	Res.	SAI2EN	SAI1EN	SPI6EN	SPI5EN	Res.	TIM11 EN	TIM10 EN	TIM9 EN
					rw			rw	rw	rw	rw		rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	SYSCFG EN	SPI4 EN	SPI1 EN	SDMMC1 EN	ADC3 EN	ADC2 EN	ADC1 EN	Res.	Res.	USART6 EN	USART1 EN	Res.	Res.	TIM8 EN	TIM1 EN
	rw	rw	rw	rw	rw	rw	rw			rw	rw			rw	rw

5. ADC (Analog to Digital Converter)

■ STM32F767 ADC 클럭 & 기준전압

- PCLK=APB2=54MHz
- ADC 모듈의 클럭을 27MHz로 설정

15.13.16 ADC common control register (ADC_CCR)

Address offset: 0x04 (this offset address is relative to ADC1 base address + 0x300)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TSVREFE	VBATE	Res.	Res.	Res.	Res.	ADCPRE	
								rw	rw					rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DMA[1:0]		DDS	Res.	DELAY[3:0]				Res.	Res.	Res.	MULTI[4:0]				
rw	rw	rw		rw	rw	rw	rw				rw	rw	rw	rw	rw

Bits 17:16 **ADCPRE**: ADC prescaler

Set and cleared by software to select the frequency of the clock to the ADC. The clock is common for all the ADCs.

Note: 00: PCLK2 divided by 2

01: PCLK2 divided by 4

10: PCLK2 divided by 6

11: PCLK2 divided by 8

5. ADC (Analog to Digital Converter)

- STM32F767 ADC 클록 & 기준전압
 - ADC_CR1 레지스터 설정을 통해 12bit 해상도로 선택

15.13.2 ADC control register 1 (ADC_CR1)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	OVRIE	RES		AWDEN	JAWDEN	Res.	Res.	Res.	Res.	Res.	Res.
					rw	rw	rw	rw	rw						
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DISCNUM[2:0]			JDISCEN	DISCEN	JAUTO	AWDSGL	SCAN	JEOCIE	AWDIE	EOCIE	AWDCH[4:0]				
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 25:24 **RES[1:0]**: Resolution

These bits are written by software to select the resolution of the conversion.

00: 12-bit (minimum 15 ADCCLK cycles)

01: 10-bit (minimum 13 ADCCLK cycles)

10: 8-bit (minimum 11 ADCCLK cycles)

11: 6-bit (minimum 9 ADCCLK cycles)

5. ADC (Analog to Digital Converter)

■ STM32F767 ADC 사용

15.13.3 ADC control register 2 (ADC_CR2)

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	SWSTART	EXTEN		EXTSEL[3:0]				Res.	JSWSTART	JEXTEN		JEXTSEL[3:0]			
	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	ALIGN	EOCS	DDS	DMA	Res.	Res.	Res.	Res.	Res.	Res.	CONT	ADON
				rw	rw	rw	rw							rw	rw

Bit 0 **ADON**: A/D Converter ON / OFF
This bit is set and cleared by software.
0: Disable ADC conversion and go to power down mode
1: Enable ADC

Bit 30 **SWSTART**: Start conversion of regular channels
This bit is set by software to start conversion and cleared by hardware as soon as the conversion starts.
0: Reset state
1: Starts conversion of regular channels
Note: This bit can be set only when ADON = 1 otherwise no conversion is launched.

5. ADC (Analog to Digital Converter)

- STM32F767 ADC 사용
 - ADC_SQR1 레지스터 설정으로 시퀀스 길이를 설정 → 현재는 단일 채널만 사용함

15.13.9 ADC regular sequence register 1 (ADC_SQR1)

Address offset: 0x2C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	L[3:0]				SQ16[4:1]			
								rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SQ16_0	SQ15[4:0]					SQ14[4:0]					SQ13[4:0]				
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:24 Reserved, must be kept at reset value.

Bits 23:20 **L[3:0]**: Regular channel sequence length

These bits are written by software to define the total number of conversions in the regular channel conversion sequence.

0000: 1 conversion

0001: 2 conversions

...

1111: 16 conversions

5. ADC (Analog to Digital Converter)

- STM32F767 ADC 사용
 - ADC_SQR3 레지스터 설정으로 어떤 채널을 사용할지 설정 : PA7(채널 7)

15.13.11 ADC regular sequence register 3 (ADC_SQR3)

Address offset: 0x34

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	SQ6[4:0]					SQ5[4:0]					SQ4[4:1]			
		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SQ4_0	SQ3[4:0]					SQ2[4:0]					SQ1[4:0]				
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:30 Reserved, must be kept at reset value.

Bits 29:25 **SQ6[4:0]**: 6th conversion in regular sequence

These bits are written by software with the channel number (0..18) assigned as the 6th in the sequence to be converted.

Bits 24:20 **SQ5[4:0]**: 5th conversion in regular sequence

Bits 19:15 **SQ4[4:0]**: 4th conversion in regular sequence

Bits 14:10 **SQ3[4:0]**: 3rd conversion in regular sequence

Bits 9:5 **SQ2[4:0]**: 2nd conversion in regular sequence

Bits 4:0 **SQ1[4:0]**: 1st conversion in regular sequence

5. ADC (Analog to Digital Converter)

■ STM32F767 ADC 사용

- ADC_SR 레지스터의 EOC 비트를 모니터링하여 ADC 컨버전이 완료되었는지를 확인할 수 있음

15.13.1 ADC status register (ADC_SR)

Address offset: 0x00

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OVR	STRT	JSTRT	JEOP	EOC	AWD
										rc_w0	rc_w0	rc_w0	rc_w0	rc_w0	rc_w0

Bit 1 **EOC**: Regular channel end of conversion

This bit is set by hardware at the end of the conversion of a regular group of channels. It is cleared by software or by reading the ADC_DR register.

0: Conversion not complete (EOCS=0), or sequence of conversions not complete (EOCS=1)

1: Conversion complete (EOCS=0), or sequence of conversions complete (EOCS=1)

6. Timer & Counter

- 타이머 & 카운터 비교 출력
 - 타이머는 내부 클럭을 사용하여 일정 주기의 신호를 만들어내거나, 시간을 측정
 - 카운터는 외부 신호를 사용하여 계수 동작

Timer

내부 Clock 사용

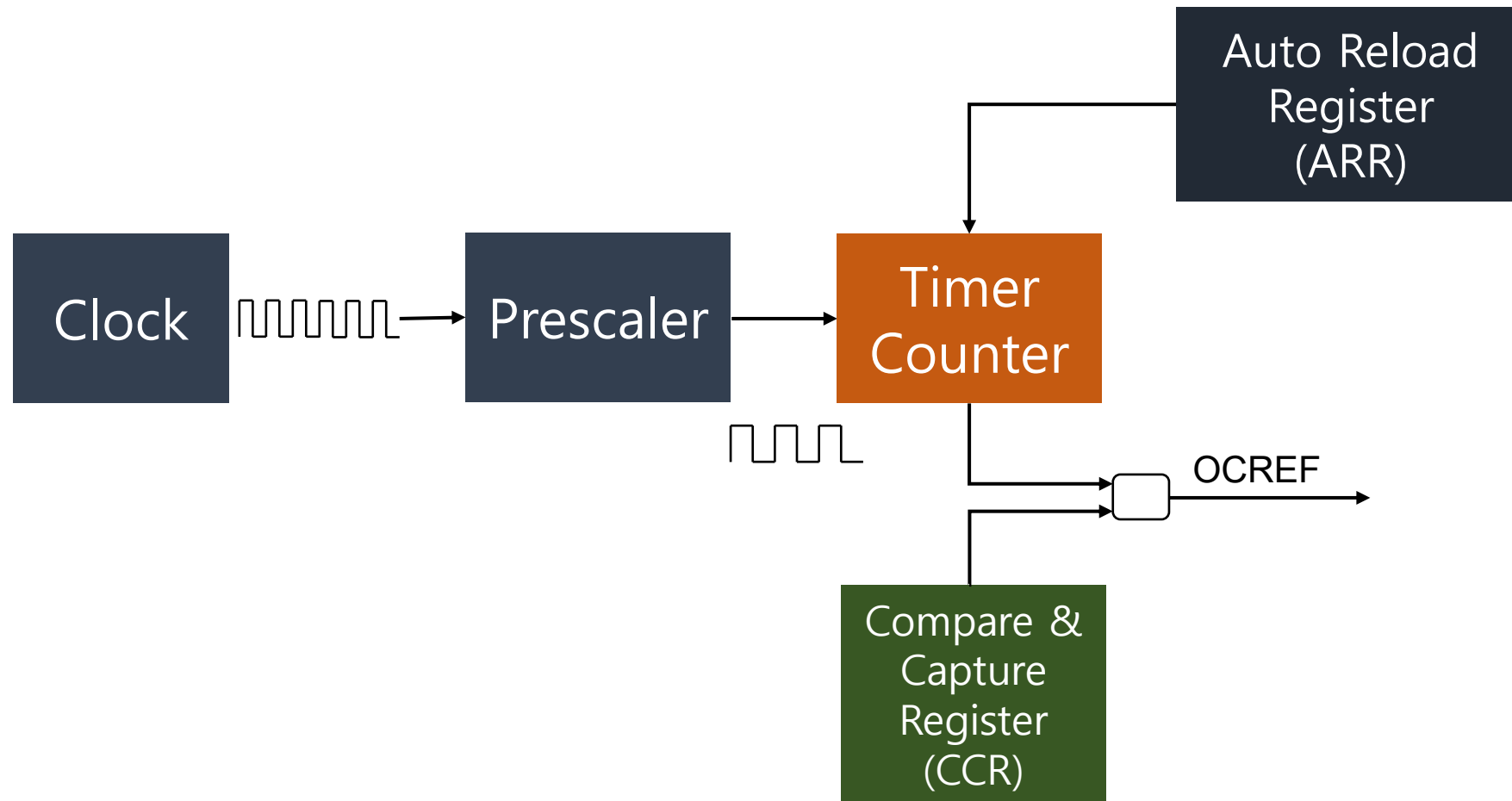
VS

Counter

외부 신호 사용

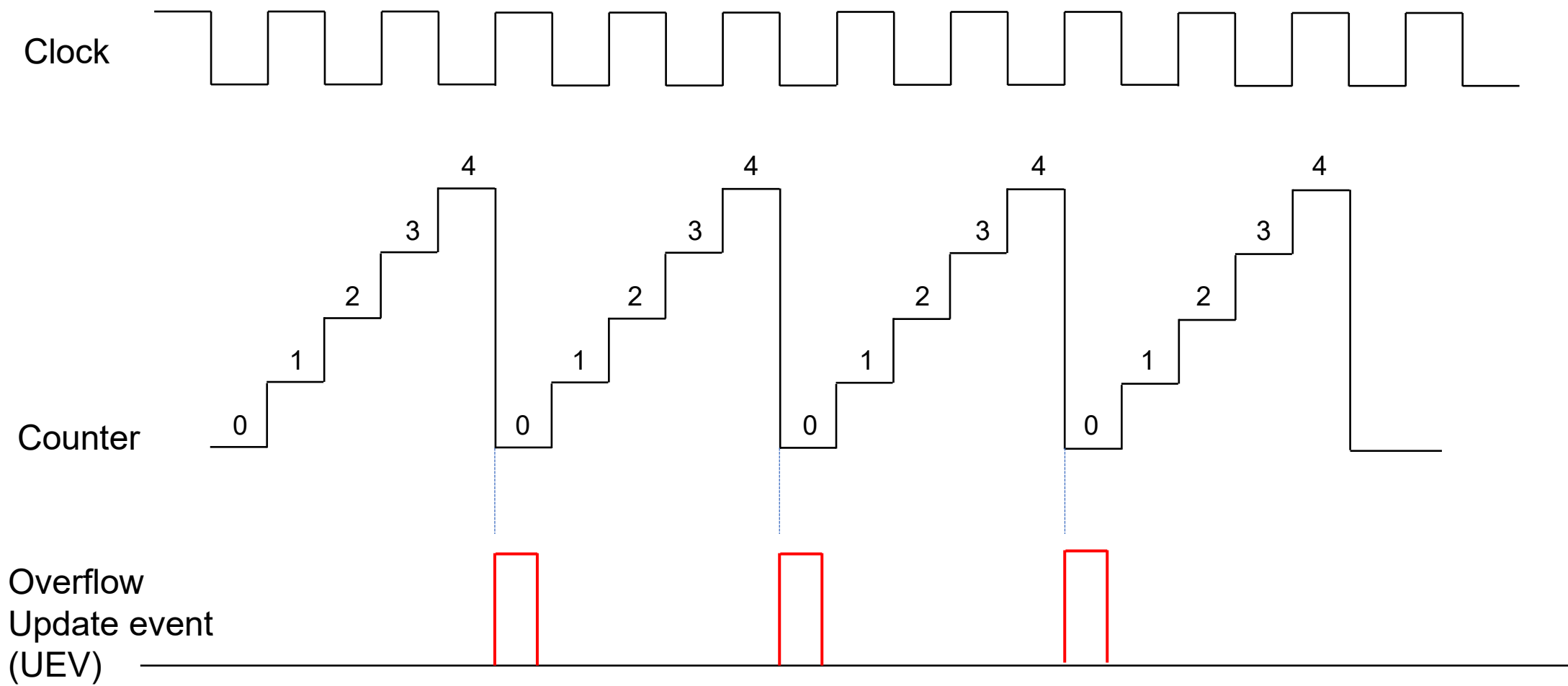
6. Timer & Counter

- 타이머 & 카운터 비교 출력



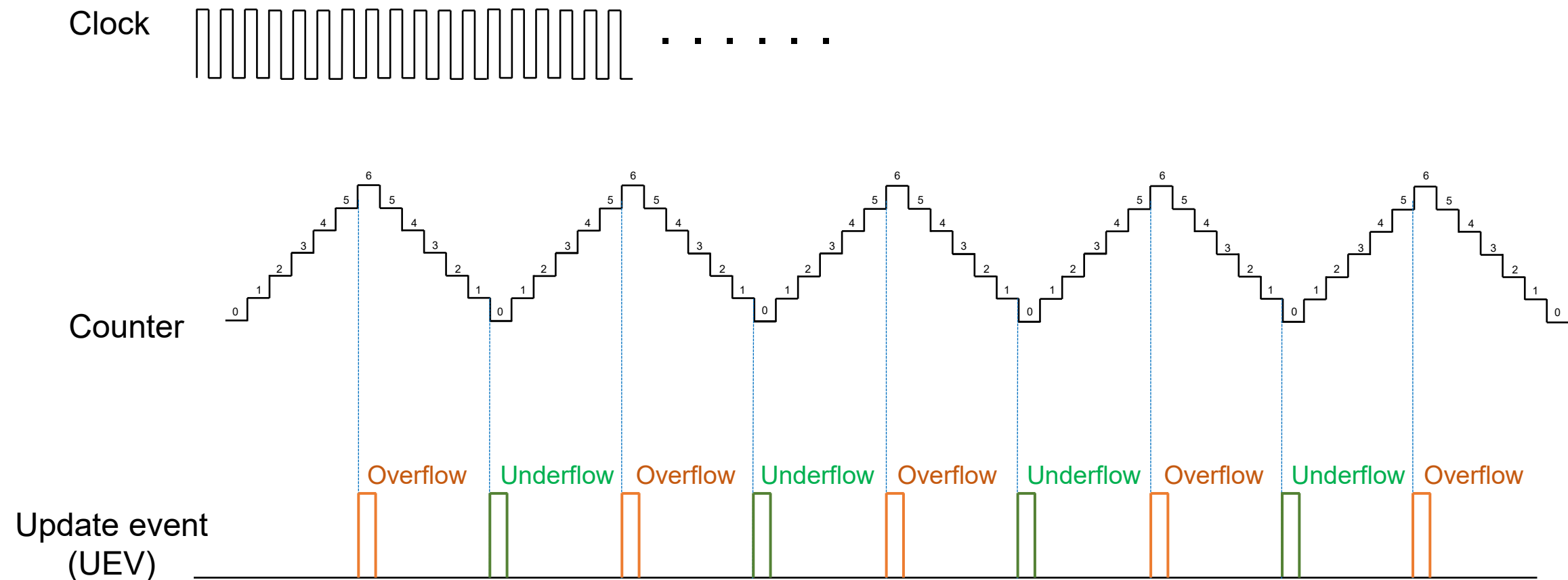
6. Timer & Counter

- 타이머 & 카운터 Up count
 - ARR : 4인 경우



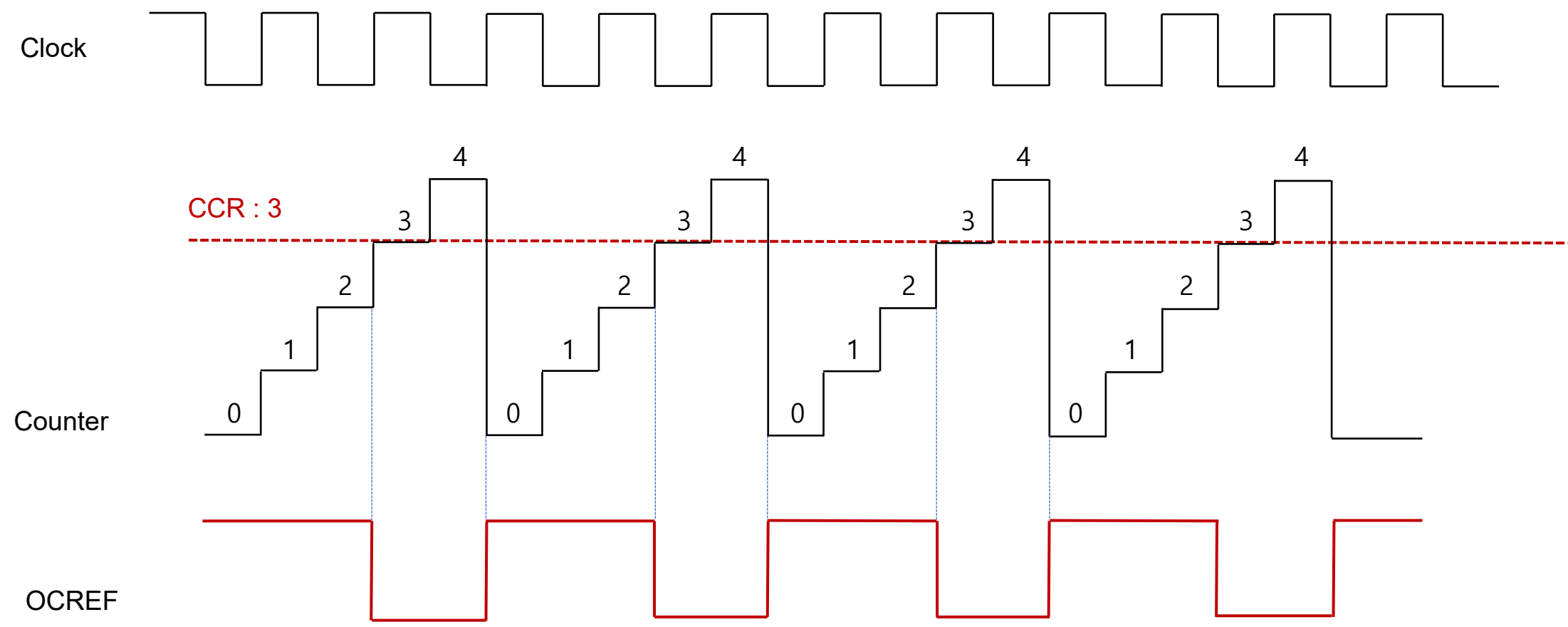
6. Timer & Counter

- 타이머 & 카운터 Up & Down count
 - ARR : 6인 경우



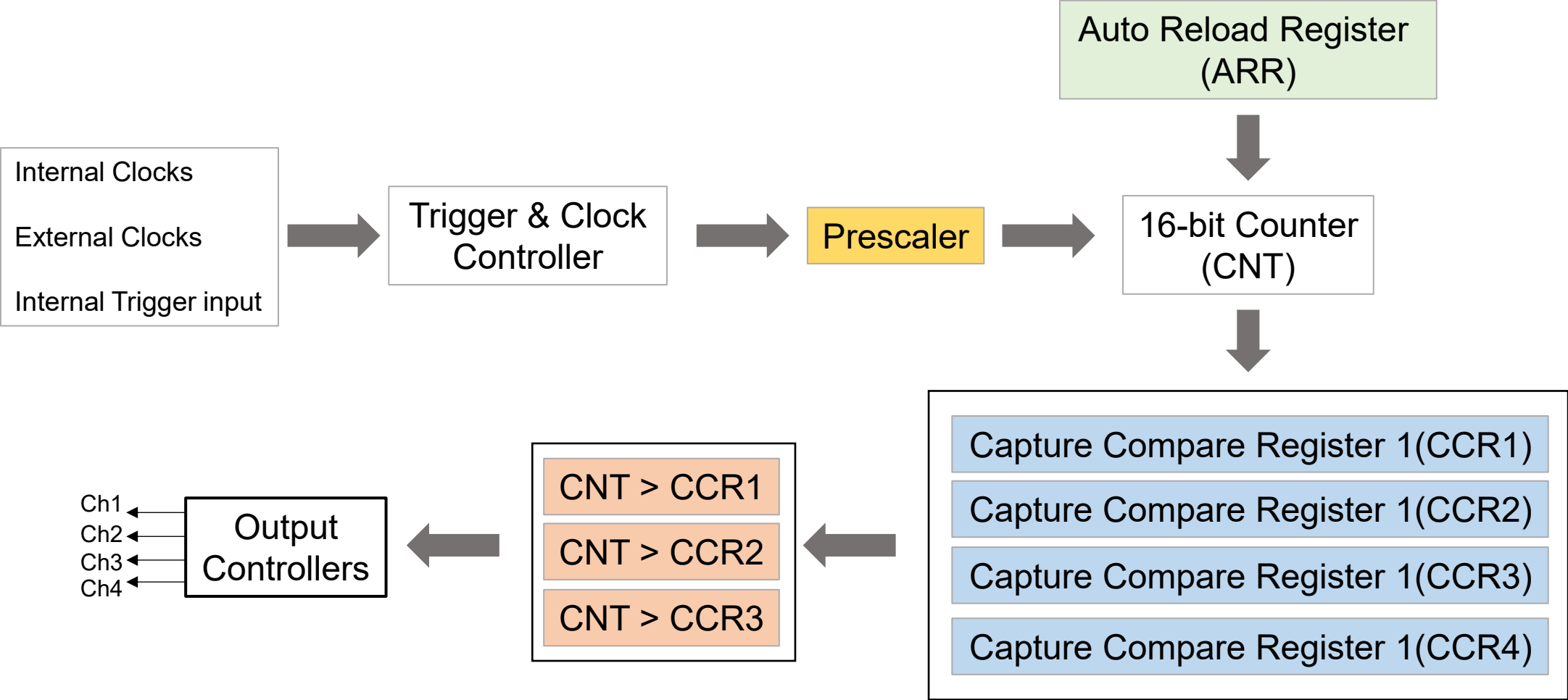
6. Timer & Counter

- 타이머 & 카운터 Up count
 - ARR : 4, CCR : 3 → PWM 출력



6. Timer & Counter

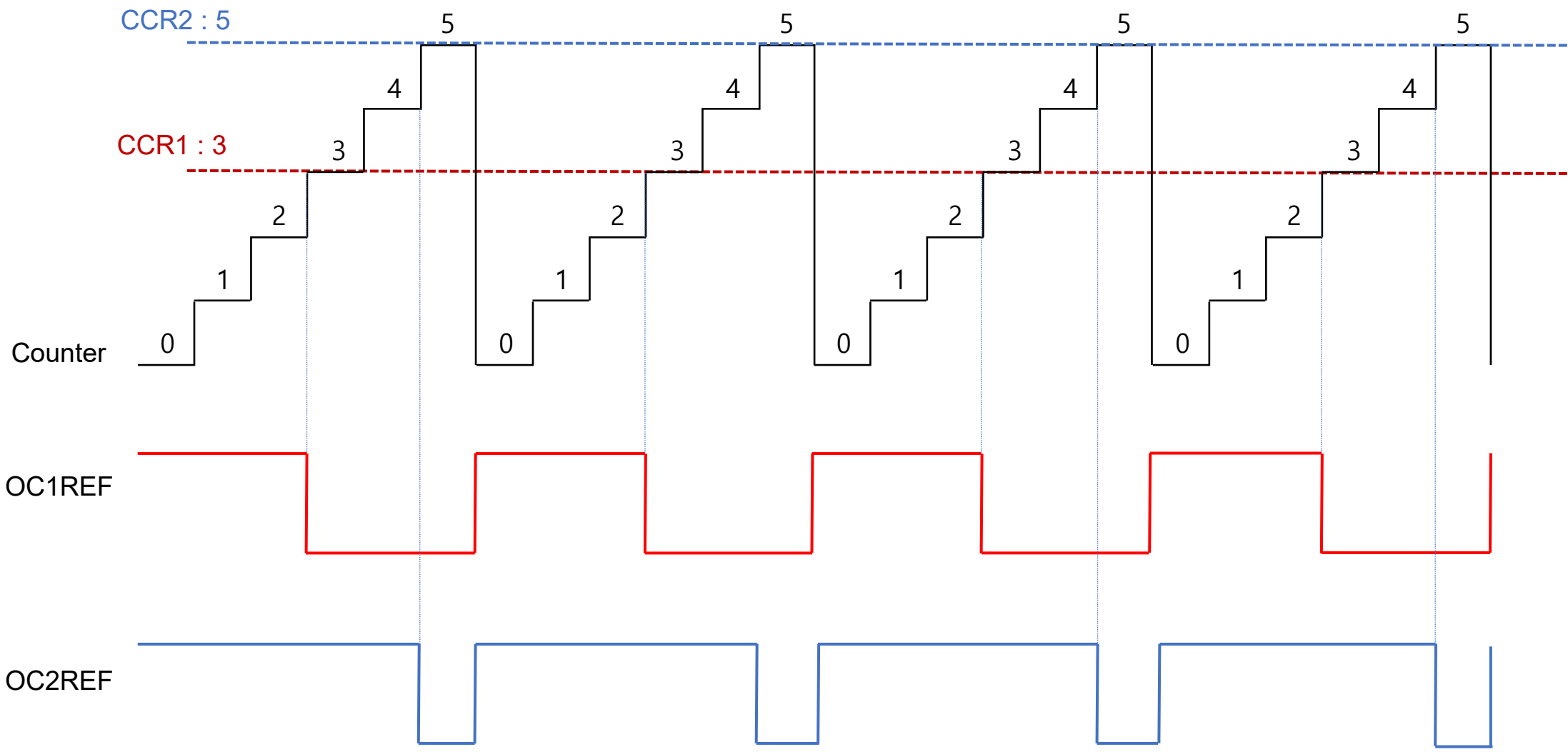
- 멀티 채널 Output



6. Timer & Counter

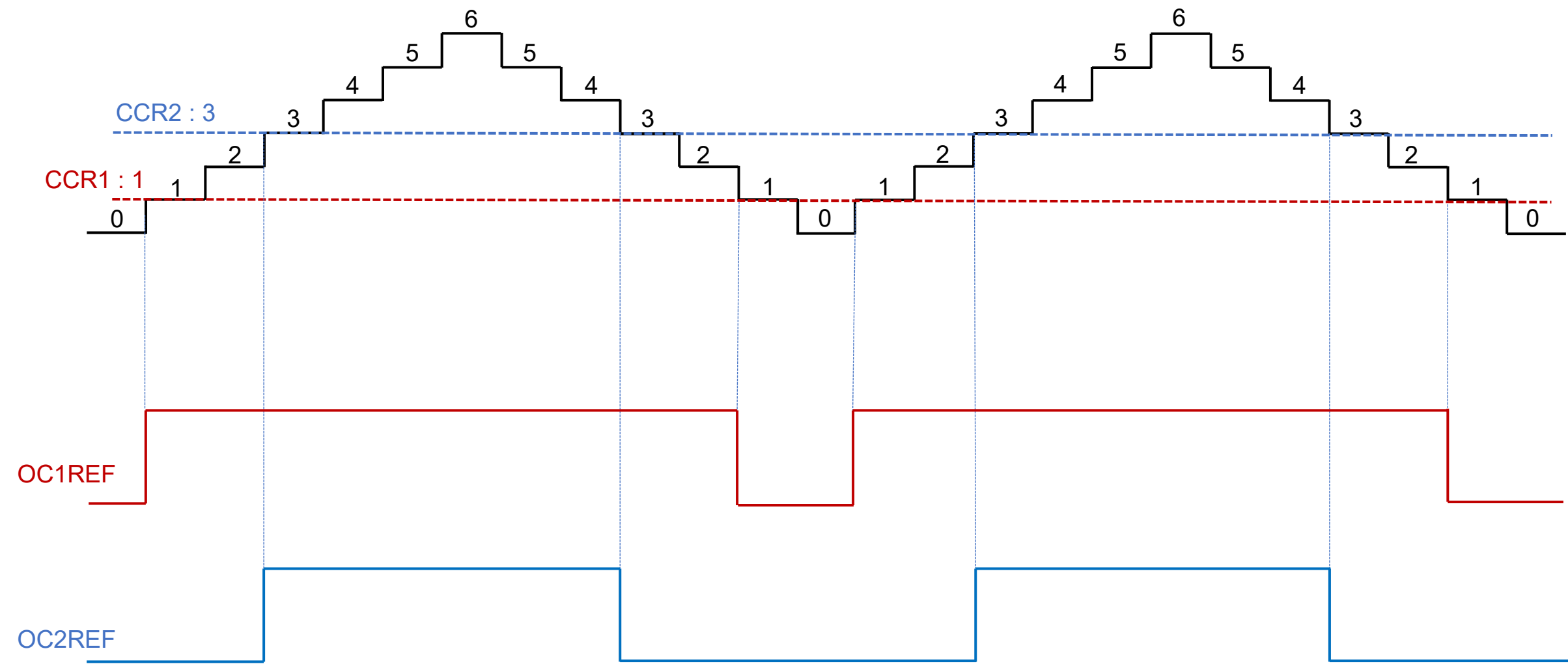
타이머 & 카운터 Edge-aligned

➤ ARR : 5, CCR1 : 3, CCR2 : 5



6. Timer & Counter

- 타이머 & 카운터 Up & Down count
 - ARR : 6 CCR1 = 3, CCR2 = 1, mode=2



- PWM 출력 극성
 - PWM 극성은 TIMx_CCER 레지스터를 이용해서 변경할 수 있음

Mode	CNT > CCR	CNT <= CCR
PWM mode 1	Active	Inactive
PWM mode 2	Inactive	Active

- TIMx_CCER 레지스터에서 설정 가능

	Active	Inactvie
Active High	High Voltage	Low Voltage
Active Low	Low Voltage	High Voltage

- Auto-Reload Register (ARR)
 - Auto-Reload Preload Enable (ARPE) 비트는 TIMx_CR1 레지스터에서 설정 가능
 - ARPE = 1 : Preload Register를 사용하여 UEV가 발생하기 전까지는 ARR을 변경하지 않음 → 글리치 현상을 방지, 부드러운 전환
 - ARPE = 0 : ARR 값을 변경하면 즉시 적용
 - ✓ 글리치 현상 : 원치 않는 짧은 펄스나 일시적인 오류 신호
 - ex) PWM 신호 생성 중 한 주기가 끝나기 전에 ARR 값이 변경되면 비정상적인 폭의 펄스가 생성 될 수 있음



Fig. ARPE = 1인 경우

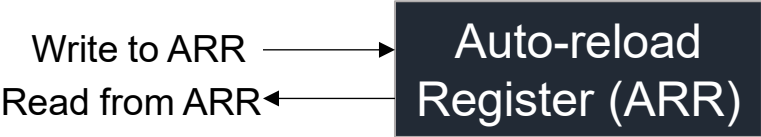
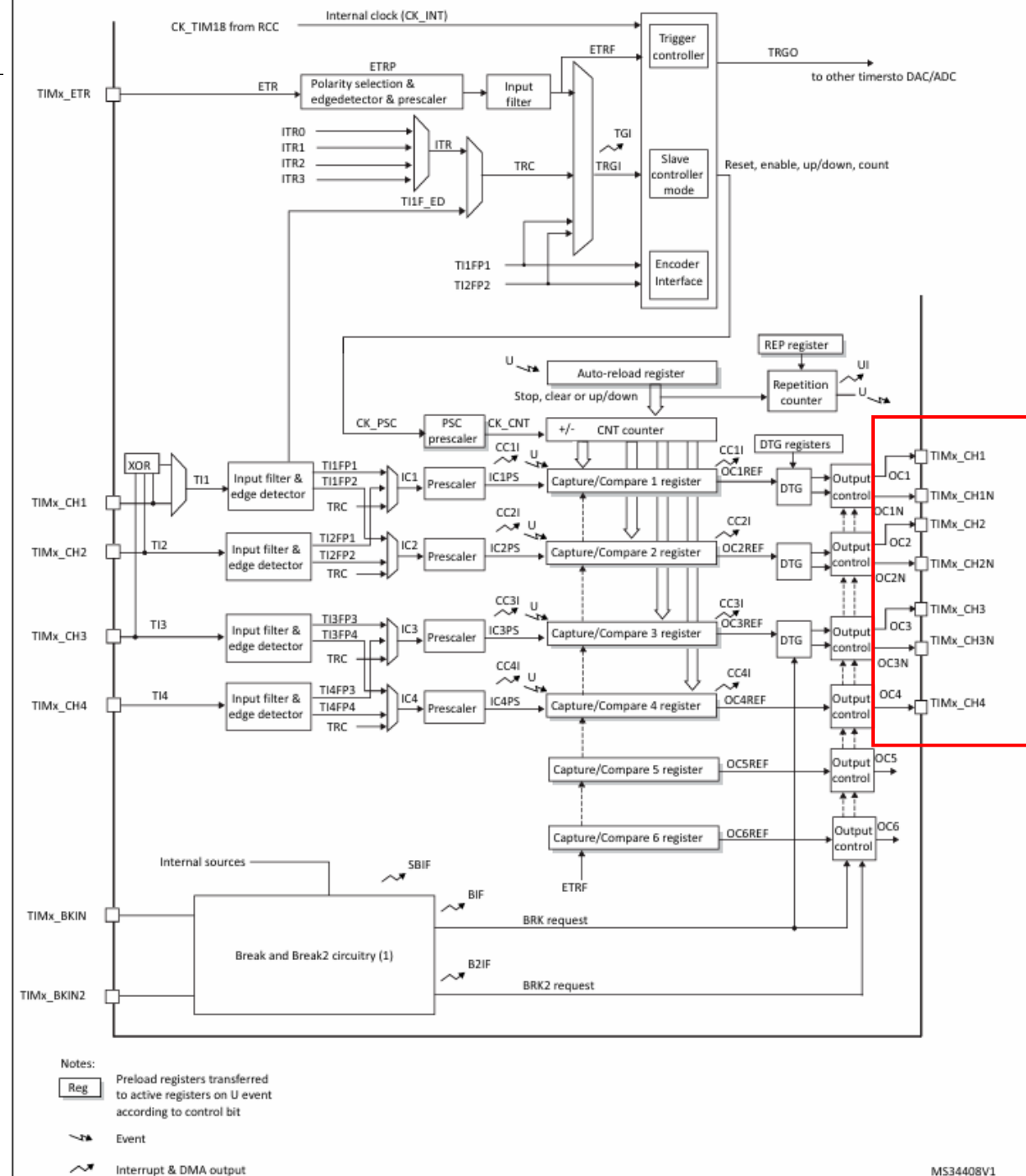


Fig. ARPE = 0인 경우

6. Timer & Counter

■ STM32F767VGT6의 고성능 TIM1/8

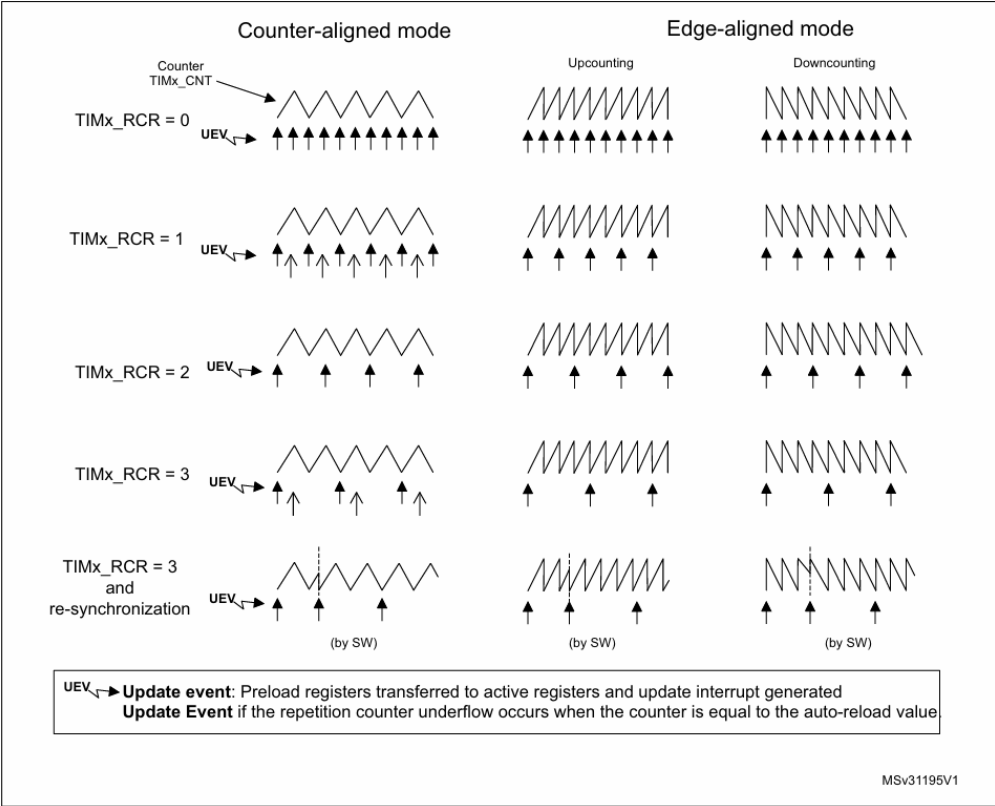
- STM32F767VGT6은 TIM1~TIM14의 타이머(14개)를 가지고 있음
- TIM1, TIM8은 PWM 채널 반전 출력 가능(모터 제어 최적)
- 채널 5,6은 입출력 신호, 입력 캡처 기능이 없고 출력 비교 기능만 가짐



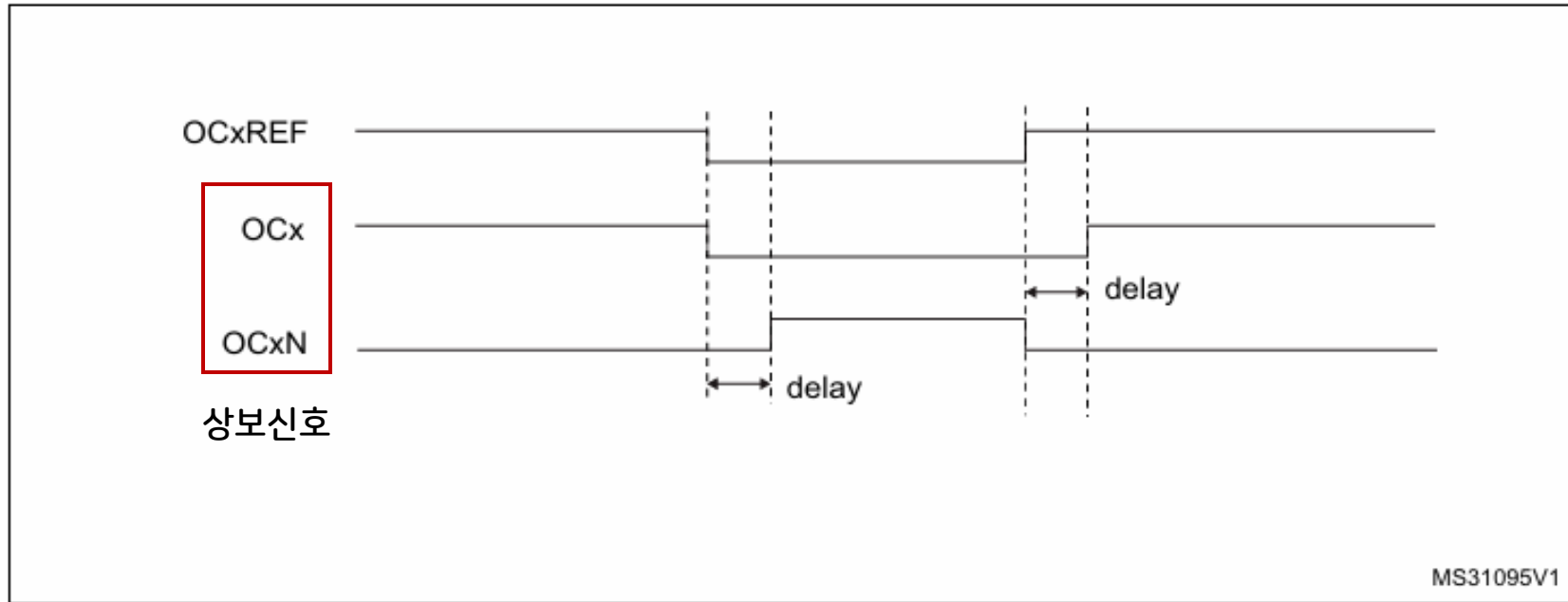
6. Timer & Counter

반복 카운터

- TIM1/8 반복 카운터는 TIMx_RCR에 지정된 횟수 만큼 다운 카운팅 동작(**Down count**)을 반복하면서 0으로 도달하면 업데이트 이벤트 발생
- 반복카운터는 Up-count 모드 에서 오버플로우, Down-count 모드에서 언더플로우, Center-aligned 모드에서 오버플로우 또는 언더플로우 가 발생하는 경우에 1 씩 감소
- Center-aligned 모드에서 TIMx_RCR이 홀수이면 오버플로우 또는 언더플로우 1가지에서만 동작
- 카운터 동작 시작 전에 TIMx_RCR 레지스터 값을 설정하면 오버플로우, 카운터 동작이 시작된 후 TIMx_RC 레지스터 값을 설정하면 언더플로우에서 업데이트 이벤트 발생 (홀수인경우)



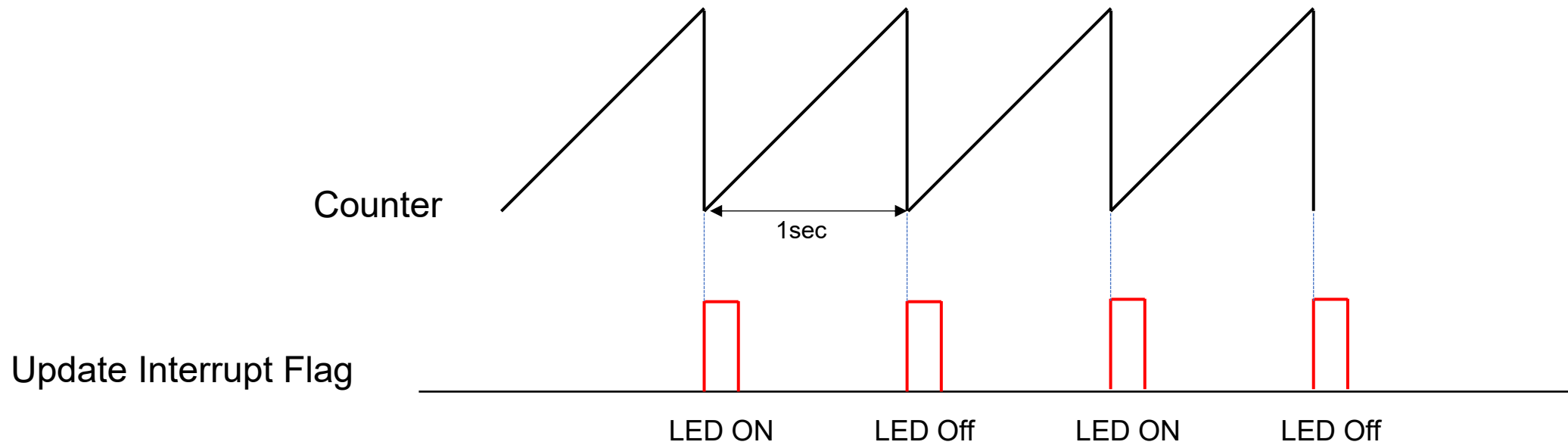
- 상보(Complementary) 출력, 데드타임 설정
 - BLDC, PMSM과 같은 3상 모터를 구동하기 위해서는 상보 출력과 데드타임 설정은 필수적
 - 타이머1/8에서는 채널1~3에 대하여 1쌍의 상보 신호 OCx, OCxN을 출력할 수 있고, 출력신호의 데드타임을 설정할 수 있음
 - 펄스폭의 크기를 주의하여 설정해야 함



<상보출력과 데드타임 설정>

6. Timer & Counter

- 타이머로 1초 주기를 만들어서 LED 토글 실습
 - 1초마다 오버플로우가 발생하도록 설정
 - 1초마다 LED를 토글 → On/Off 반복



- STM32F767 타이머/카운터 레지스터 설정 (LED토글 실습)
 - TIMx_PSC 레지스터를 설정하여 타이머/카운터 모듈의 분주비 설정

22.4.11 TIMx prescaler (TIMx_PSC)(x = 1, 8)

Address offset: 0x28

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PSC[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:0 **PSC[15:0]**: Prescaler value

The counter clock frequency (CK_CNT) is equal to $f_{CK_PSC} / (PSC[15:0] + 1)$.
PSC contains the value to be loaded in the active prescaler register at each update event (including when the counter is cleared through UG bit of TIMx_EGR register or through trigger controller when configured in “reset mode”).

- STM32F767 타이머/카운터 레지스터 설정 (LED토글 실습)
 - TIMx_ARR 레지스터를 설정하여 카운터의 Max값을 선정

22.4.12 TIMx auto-reload register (TIMx_ARR)(x = 1, 8)

Address offset: 0x2C

Reset value: 0xFFFF

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ARR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:0 **ARR[15:0]**: Auto-reload value

ARR is the value to be loaded in the actual auto-reload register.
Refer to the [Section 22.3.1: Time-base unit on page 650](#) for more details about ARR update and behavior.
The counter is blocked while the auto-reload value is null.

- STM32F767 타이머/카운터 레지스터 설정 (LED토글 실습)
 - TIMx_EGR 레지스터의 UG 비트를 1로 Set하여 카운터를 다시 초기화

이 비트는 소프트웨어로 설정할 수 있으며, 하드웨어에 의해 자동으로 클리어 됩니다.
0: 동작 없음
1: 카운터를 다시 초기화하고 레지스터를 업데이트합니다. 프리스케일러 카운터도 클리어됩니다(프리스케일러 비율에는 영향을 미치지 않습니다). 중앙 정렬 모드가 선택되었거나 DIR=0(업카운팅)이면 카운터가 클리어되고, DIR=1(다운카운팅)이면 자동 리로드 값(TIMx_ARR)을 사용합니다.

22.4.6 TIMx event generation register (TIMx_EGR)(x = 1, 8)

Address offset: 0x14

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	B2G	BG	TG	COMG	CC4G	CC3G	CC2G	CC1G	UG
							w	w	w	w	w	w	w	w	w

Bit 0 **UG**: Update generation

This bit can be set by software, it is automatically cleared by hardware.

0: No action

1: Reinitialize the counter and generates an update of the registers. Note that the prescaler counter is cleared too (anyway the prescaler ratio is not affected). The counter is cleared if the center-aligned mode is selected or if DIR=0 (upcounting), else it takes the auto-reload value (TIMx_ARR) if DIR=1 (downcounting).

- STM32F767 타이머/카운터 레지스터 설정 (LED토글 실습)
 - TIMx_CR1의 레지스터를 설정하여 카운터를 동작시킴

22.4.1 TIMx control register 1 (TIMx_CR1)(x = 1, 8)

Address offset: 0x00

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	UIFRE MAP	Res.	CKD[1:0]		ARPE	CMS[1:0]		DIR	OPM	URS	UDIS	CEN
				rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 0 **CEN**: Counter enable

0: Counter disabled

1: Counter enabled

Note: External clock, gated mode and encoder mode can work only if the CEN bit has been previously set by software. However trigger mode can set the CEN bit automatically by hardware.

- STM32F767 타이머/카운터 레지스터 설정 (LED토글 실습)
 - TIMx_SR의 UIF 비트를 이용하여 1초마다 LED 토글

22.4.5 TIMx status register (TIMx_SR)(x = 1, 8)

Address offset: 0x10

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CC6IF	CC5IF
														rc_w0	rc_w0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.		CC4OF	CC3OF	CC2OF	CC1OF	B2IF	BIF	TIF	COMIF	CC4IF	CC3IF	CC2IF	CC1IF	UIF
			rc_w0	rc_w0	rc_w0	rc_w0	rc_w0	rc_w0	rc_w0	rc_w0	rc_w0	rc_w0	rc_w0	rc_w0	rc_w0

Bit 0 **UIF**: Update interrupt flag

This bit is set by hardware on an update event. It is cleared by software.

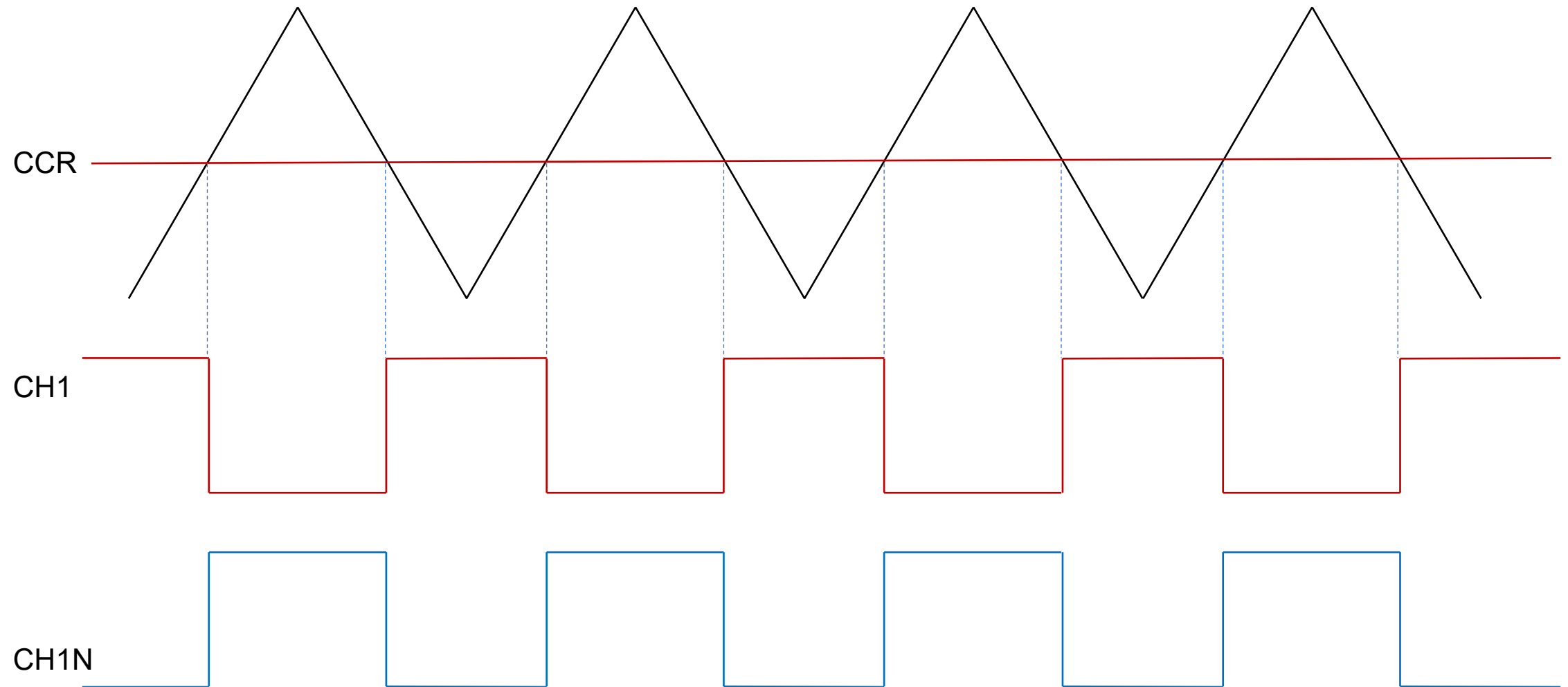
0: No update occurred.

1: Update interrupt pending. This bit is set by hardware when the registers are updated:

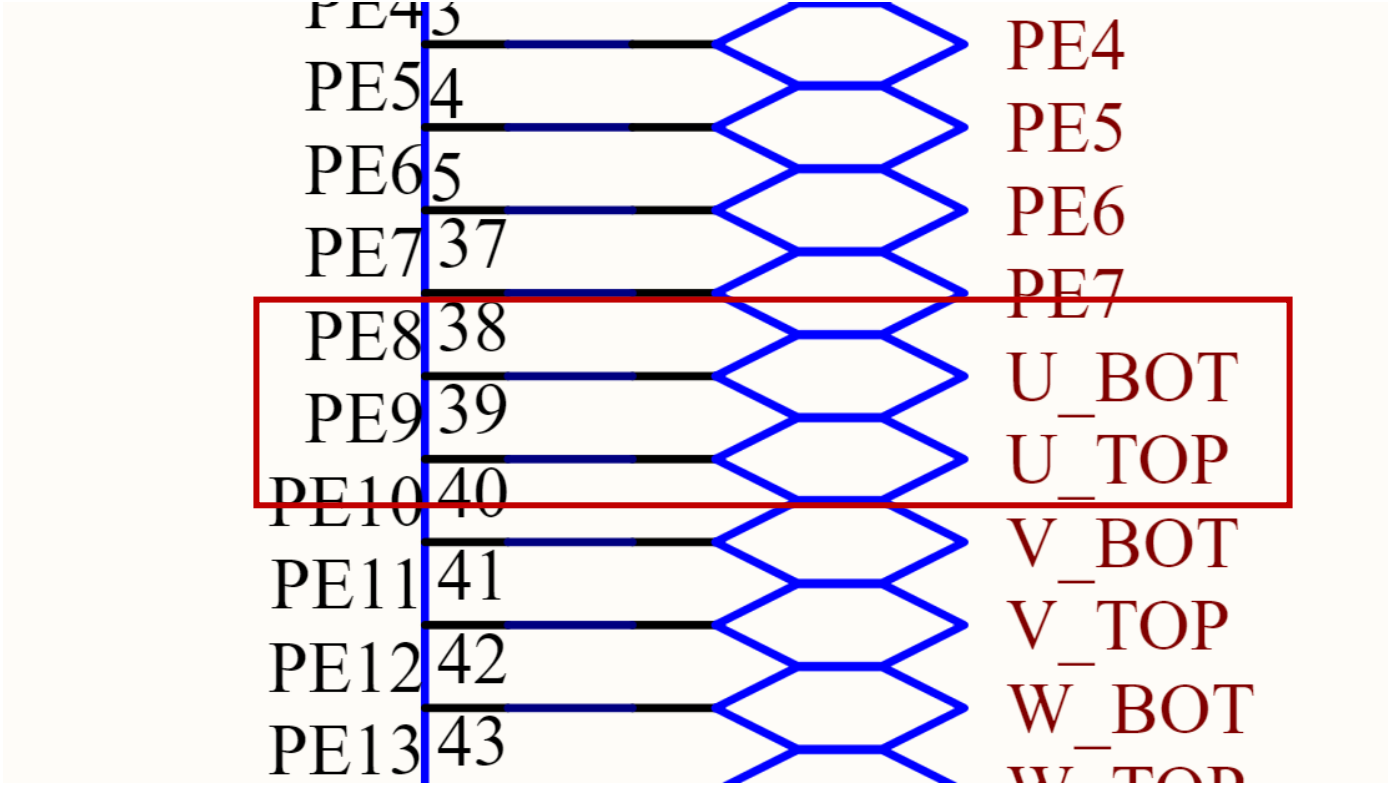
- At overflow or underflow regarding the repetition counter value (update if repetition counter = 0) and if the UDIS=0 in the TIMx_CR1 register.
- When CNT is reinitialized by software using the UG bit in TIMx_EGR register, if URS=0 and UDIS=0 in the TIMx_CR1 register.
- When CNT is reinitialized by a trigger event (refer to [Section 22.4.3: TIMx slave mode control register \(TIMx_SMCR\)\(x = 1, 8\)](#)), if URS=0 and UDIS=0 in the TIMx_CR1 register.

■ Center Align 상보 PWM 실습

- 타이머1의 CH1,CH2N을 Center Align 상보 PWM 출력(1kHz)



- Center Align 상보 PWM 실습
 - PE9(CH1), PE8(CH1N) 사용



- Center Align 상보 PWM 실습
 - GPIOx_MODER 레지스터 설정을 통해 PE9(CH1), PE8(CH1N) Alternate function mode 설정

6.4.1 GPIO port mode register (GPIOx_MODER) (x =A to K)

Address offset:0x00

Reset value:

- 0xA800 0000 for port A
- 0x0000 0280 for port B
- 0x0000 0000 for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **MODER[15:0][1:0]**: Port x configuration I/O pin y (y = 15 to 0)

These bits are written by software to configure the I/O mode.

- 00: Input mode (reset state)
- 01: General purpose output mode
- 10: Alternate function mode
- 11: Analog mode

- Center Align 상보 PWM 실습
 - TIMx_CR1 레지스터 설정을 하여 Center-aligned mode 설정

22.4.1 TIMx control register 1 (TIMx_CR1)(x = 1, 8)

Address offset: 0x00

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	UIFRE MAP	Res.	CKD[1:0]		ARPE	CMS[1:0]		DIR	OPM	URS	UDIS	CEN
				rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 6:5 **CMS[1:0]**: Center-aligned mode selection

- 00: Edge-aligned mode. The counter counts up or down depending on the direction bit (DIR).
- 01: Center-aligned mode 1. The counter counts up and down alternatively. Output compare interrupt flags of channels configured in output (CCxS=00 in TIMx_CCMRx register) are set only when the counter is counting down.
- 10: Center-aligned mode 2. The counter counts up and down alternatively. Output compare interrupt flags of channels configured in output (CCxS=00 in TIMx_CCMRx register) are set only when the counter is counting up.
- 11: Center-aligned mode 3. The counter counts up and down alternatively. Output compare interrupt flags of channels configured in output (CCxS=00 in TIMx_CCMRx register) are set both when the counter is counting up or down.

Note: It is not allowed to switch from edge-aligned mode to center-aligned mode as long as the counter is enabled (CEN=1)

Bit 0 **CEN**: Counter enable

- 0: Counter disabled
- 1: Counter enabled

Note: External clock, gated mode and encoder mode can work only if the CEN bit has been previously set by software. However trigger mode can set the CEN bit automatically by hardware.

- Center Align 상보 PWM 실습
 - TIMx_CCMR1 레지스터를 설정하여, PWM 동작 모드와 Preload를 enabled

22.4.7 TIMx capture/compare mode register 1 (TIMx_CCMR1)(x = 1, 8)

Address offset: 0x18
Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	OC2M[3]	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OC1M[3]
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
							rw								rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OC2 CE	OC2M[2:0]			OC2 PE	OC2 FE	CC2S[1:0]		OC1 CE	OC1M[2:0]			OC1 PE	OC1 FE	CC1S[1:0]	
IC2F[3:0]				IC2PSC[1:0]					IC1F[3:0]			IC1PSC[1:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 6:4 **OC1M**: Output Compare 1 mode

0110: PWM mode 1 - In upcounting, channel 1 is active as long as TIMx_CNT<TIMx_CCR else inactive. In downcounting, channel 1 is inactive (OC1REF='0') as long as TIMx_CNT>TIMx_CCR1 else active (OC1REF='1').

0111: PWM mode 2 - In upcounting, channel 1 is inactive as long as TIMx_CNT<TIMx_CCR1 else active. In downcounting, channel 1 is active as long as TIMx_CNT>TIMx_CCR1 else inactive.

Bit 3 **OC1PE**: Output Compare 1 preload enable

0: Preload register on TIMx_CCR1 disabled. TIMx_CCR1 can be written at anytime, the new value is taken in account immediately.

1: Preload register on TIMx_CCR1 enabled. Read/Write operations access the preload register. TIMx_CCR1 preload value is loaded in the active register at each update event.

Note: **1:** These bits can not be modified as long as LOCK level 3 has been programmed (LOCK bits in TIMx_BDTR register) and CC1S='00' (the channel is configured in output).

2: The PWM mode can be used without validating the preload register only in one pulse mode (OPM bit set in TIMx_CR1 register). Else the behavior is not guaranteed.

- Center Align 상보 PWM 실습
 - TIMx_CCER 레지스터를 설정하여 CH1, CH1N을 enable

22.4.9 TIMx capture/compare enable register (TIMx_CCER)(x = 1, 8)

Address offset: 0x20

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CC6P	CC6E	Res.	Res.	CC5P	CC5E
										rw	rw			rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CC4NP	Res.	CC4P	CC4E	CC3NP	CC3NE	CC3P	CC3E	CC2NP	CC2NE	CC2P	CC2E	CC1NP	CC1NE	CC1P	CC1E
rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 0 **CC1E**: Capture/Compare 1 output enable

CC1 channel configured as output:

0: Off - OC1 is not active. OC1 level is then function of MOE, OSS1, OSSR, OIS1, OIS1N and CC1NE bits.

1: On - OC1 signal is output on the corresponding output pin depending on MOE, OSS1, OSSR, OIS1, OIS1N and CC1NE bits.

CC1 channel configured as input: This bit determines if a capture of the counter value can actually be done into the input capture/compare register 1 (TIMx_CCR1) or not.

0: Capture disabled.

1: Capture enabled.

On channels having a complementary output, this bit is preloaded. If the CCPC bit is set in the TIMx_CR2 register then the CC1E active bit takes the new value from the preloaded bit only when a Commutation event is generated.

Bit 2 **CC1NE**: Capture/Compare 1 complementary output enable

0: Off - OC1N is not active. OC1N level is then function of MOE, OSS1, OSSR, OIS1, OIS1N and CC1E bits.

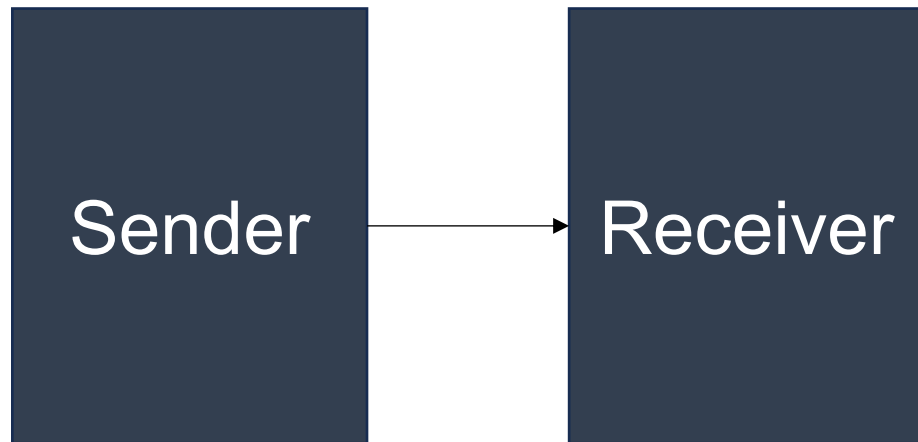
1: On - OC1N signal is output on the corresponding output pin depending on MOE, OSS1, OSSR, OIS1, OIS1N and CC1E bits.

On channels having a complementary output, this bit is preloaded. If the CCPC bit is set in the TIMx_CR2 register then the CC1NE active bit takes the new value from the preloaded bit only when a Commutation event is generated.

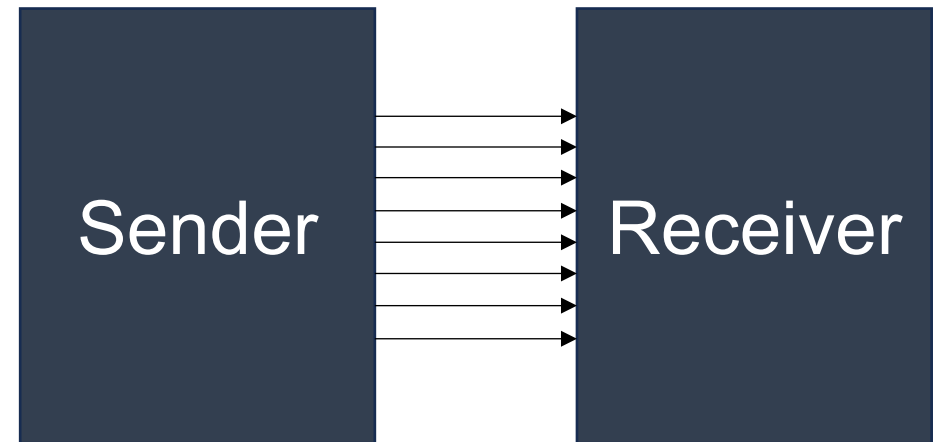
7. UART

직렬 통신, 병렬 통신

- 직렬통신은 데이터를 한 비트씩 순차적으로 전송해, 배선이 간단하고 장거리 전송이 유리하며 SPI, UART, I2C 등이 대표적이다.
- 병렬통신은 여러 비트를 동시에 전송해, 고속 전송이 가능하지만 배선이 복잡하고 거리가 길어질수록 노이즈가 커진다.
- 직렬통신 장점: 배선 수가 적고 장거리 전송에 안정적이며, 간섭이 적다.
- 병렬통신 장점: 높은 데이터 대역폭을 얻을 수 있어, 짧은 거리에서 빠른 전송이 필요할 때 유리하다.
- 사용처: 직렬은 USB, Ethernet, CAN 등 네트워크/임베디드에 폭넓게 쓰이고, 병렬은 내부 버스나 프린터 포트 등 고속·단거리 환경에 사용된다.



➤ 직렬 통신 : 비트 하나씩 송신

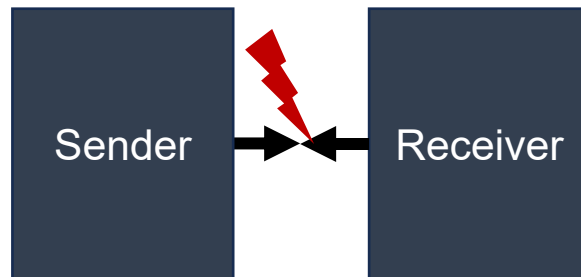
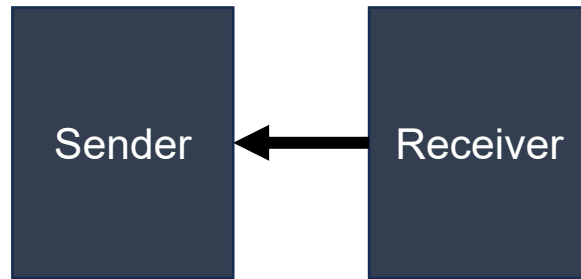
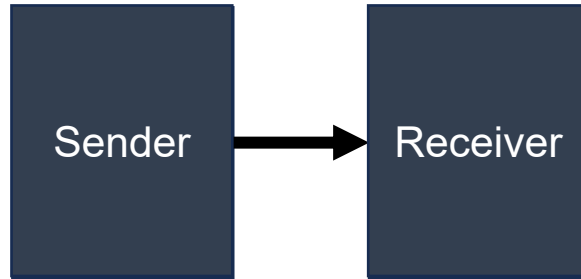


➤ 병렬 통신 : 한번에 여러 개의 비트 송신

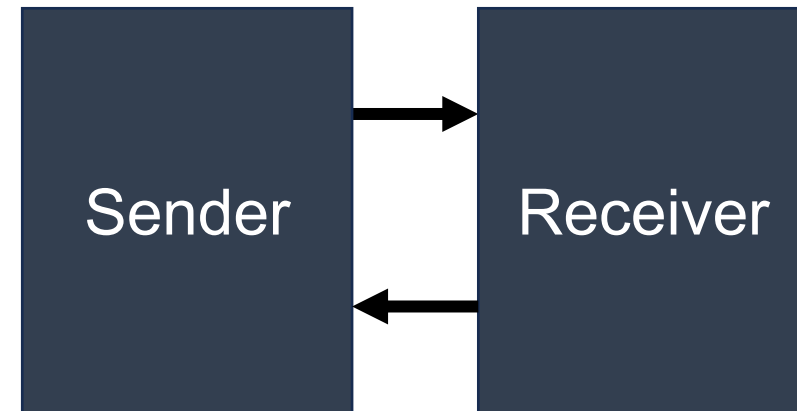
7. UART

■ 전이중, 반이중 통신

- 반이중 통신 : 양방향 통신이 가능하지만, 동시에 보내고 받는 것은 불가능 (무전기)
- 전이중 통신 : 양방향 통신이 가능하고, 동시에 주고 받을 수 있음 (스마트폰)



<반이중 통신>

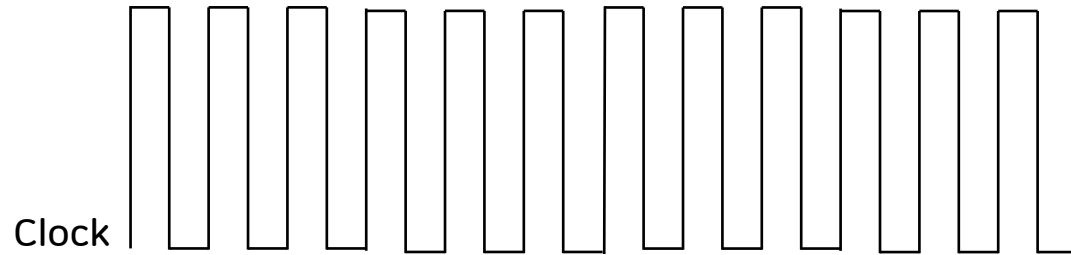


<전이중 통신>

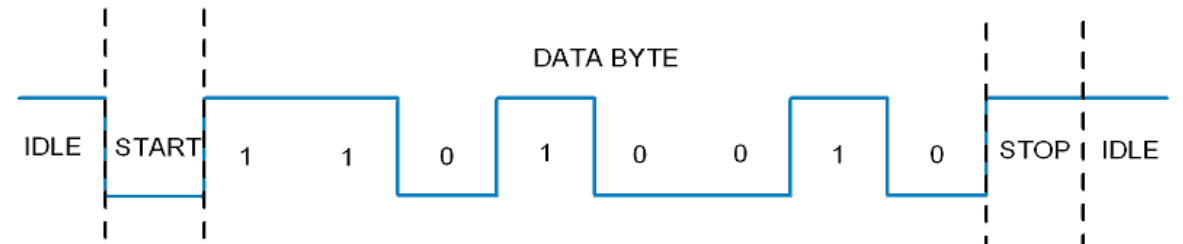
7. UART

동기통신, 비동기통신

- 동기 통신 : 클럭 신호에 동기되어 데이터를 송수신함 (I2C, SPI)
- 비동기 통신 : Tx, Rx의 통신 속도를 정해서 데이터를 송수신함 (UART 통신)



- 동기 통신 : Clock에 동기 되어 데이터 송수신



- 비동기 통신 : Tx, Rx의 통신 속도를 정해서 데이터 송수신

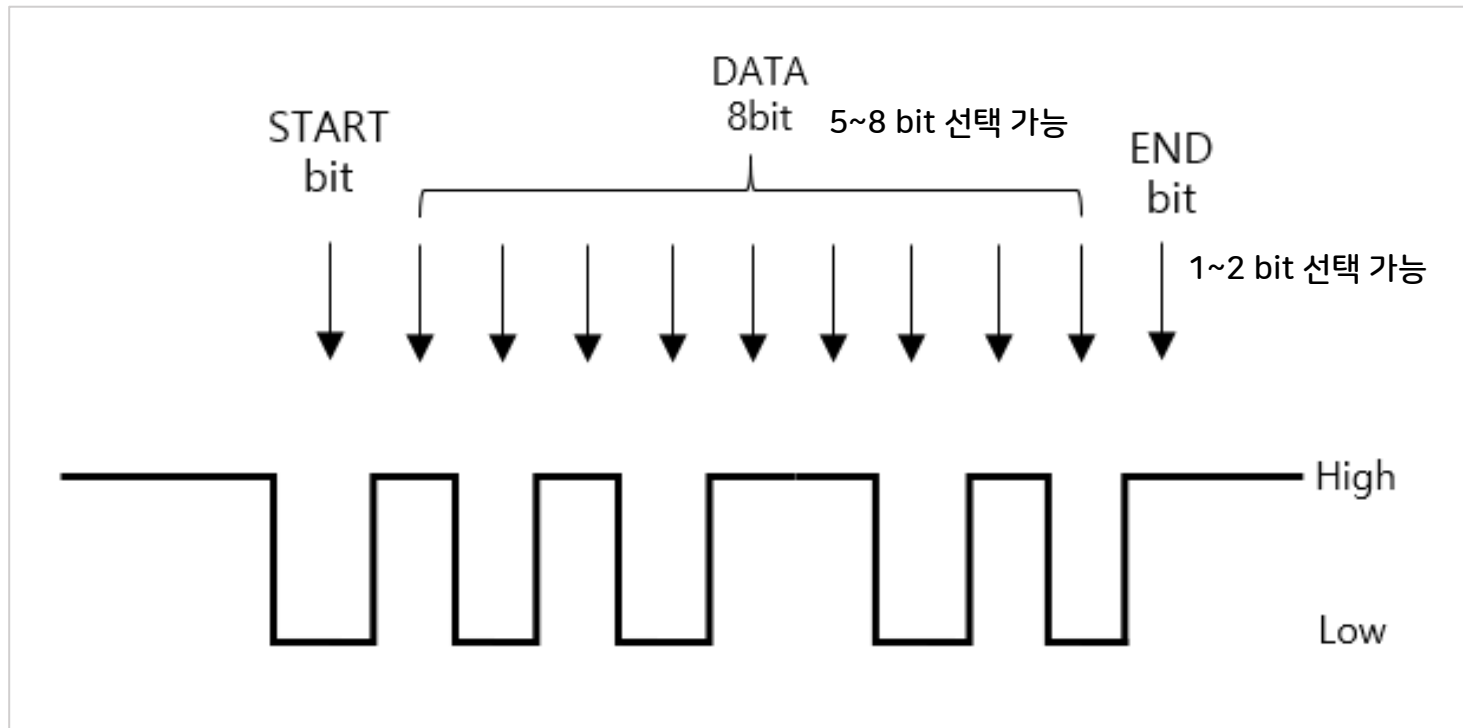
- UART vs USART

- USART : Universal Synchronous Receiver/Transmitter
클럭과 함께 데이터를 전달

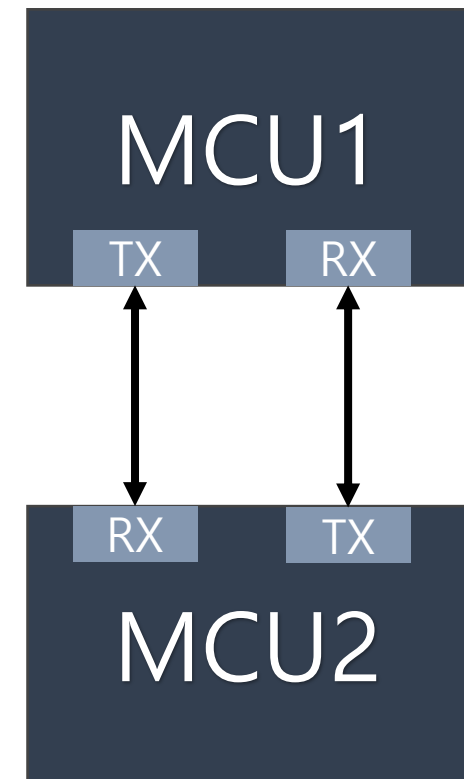
- UART : Universal Asynchronous Receiver/Transmitter
Baudrate를 이용해서 데이터를 전달

7. UART

- UART(Universal asynchronous receiver-transmitter) 통신 프로토콜
 - UART 통신은 비동기로 동작함
 - 통신을 위해선 Tx,Rx간 Start bit, Data bit, Stop bit, 통신 속도를 동일하게 설정해줘야 함
 - Tx는 Rx에, Rx는 Tx에 연결하여 통신해야 함



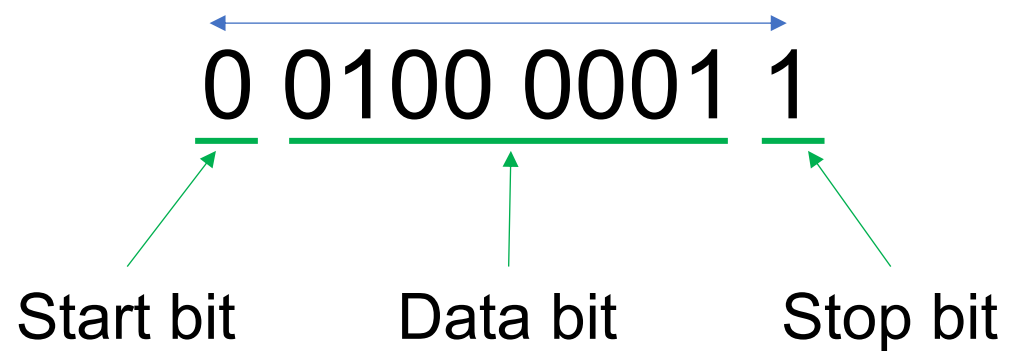
그림에는 없지만 패리티 비트 설정 가능
(송수신 오류 판별)



7. UART

- UART(Universal asynchronous receiver-transmitter) 통신 프로토콜
 - 'A'라는 문자를 보내기 위해서 아래와 같이 UART 통신 수행

ASCII 'A' = 0100 0001



7. UART

■ UART(Universal asynchronous receiver-transmitter) 통신 프로토콜

➤ BPS(Bit Per Second) 초당 비트 수를 Tx/Rx 동일하게 설정

Table 194. Error calculation for programmed baud rates at $f_{CK} = 216 \text{ MHz}$
in both cases of oversampling by 8 (OVER8 = 1)⁽¹⁾

Desired baud rate (Bps)	Actual baud rate (Bps)	BRR	Error
9600	9600	AFC4	0.00000000
19200	19200	57E2	0.00000000
38400	38400	2BF1	0.00000000
57600	57600	1D46	0.00000000
115200	115200	EA3	0.00000000
230400	230400	751	0.00000000
460800	461538.461	3A4	0.160256293
921600	923076.923	1D2	0.001602564
13500000	13500000.000	20	0.00000000
27000000	27000000.000	10	0.00000000

1. The lower the CPU clock the lower the accuracy for a particular baud rate. The upper limit of the achievable baud rate can be fixed with these data.

Table 195. Error calculation for programmed baud rates at $f_{CK} = 216 \text{ MHz}$
in both cases of oversampling by 16 (OVER8 = 0)⁽¹⁾

Desired baud rate (Bps)	Actual baud rate (Bps)	BRR	Error
9600	9600.000	57E4	0.00000000
19200	19200.000	2BF2	0.00000000
38400	38400.000	15F9	0.00000000
57600	57600.000	EA6	0.00000000
115200	115200.000	753	0.00000000
230400	230522.946	3A9	0.05336179
460800	461538.462	1D4	0.16025641
921600	923076.923	EA	0.16025641
4000000	4000000.000	36	0.00000000
6000000	6000000.000	24	0.00000000
10000000	10285714.286	15	2.85714286
13500000	13500000.000	10	0.00000000

```
68 // 예: APB1 클럭이 54MHz일 경우, BRR = 54MHz/115200 ≈ 468.75, 469로 설정
69 // 예: APB1 클럭이 54MHz일 경우, BRR = 54MHz/9600 ≈ 5625, 469로 설정
70 // USART2->BRR = 469;
71 USART2->BRR = 5625;
```

Tera Term: Serial port setup and connection

Port: COM5

Speed: 9600

Data: 8 bit

Parity: none

Stop bits: 1 bit

Flow control: none

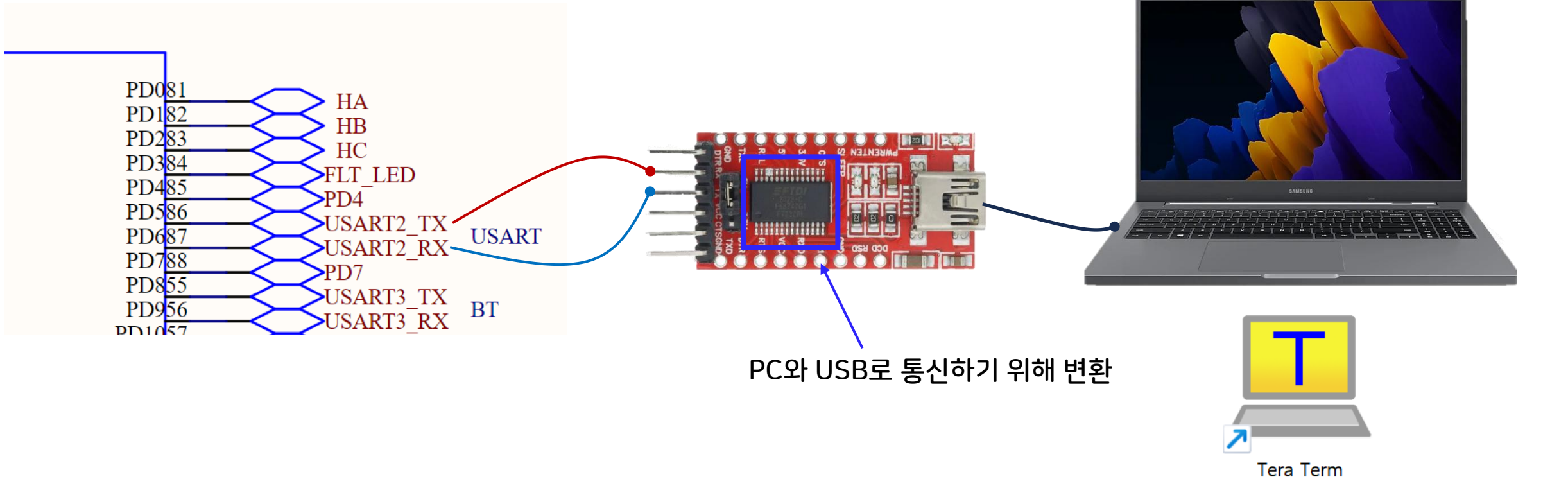
Transmit delay: 0 msec/char 0 msec/line

Device Friendly Name: USB Serial Port(COM5)
Device Instance ID: FTDIBUS\VID_0403+PID_6001+A50285BIA
Device Manufacturer: FTDI
Provider Name: FTDI
Driver Date: 7-5-2021
Driver Version: 2.12.36.4

7. UART

■ UART TX/RX 예제

- MCU와 PC의 UART 통신을 위해서 중간 역할을 해주는 모듈을 사용
- MCU ↔ 모듈 간의 Tx/Rx 연결을 주의 해야함



Tera Term 프로그램을 통해 통신 모니터링

7. UART

UART TX/RX 예제

